

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA, ĐẠI HỌC QUỐC
GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

XÂY DỰNG FRAMEWORK TẠO
DỰNG MÔ HÌNH AI CHO CÁC
ỨNG DỤNG THEO DÕI
CHUYỂN ĐỘNG CỦA CON
NGƯỜI

Chuyên ngành: Khoa học Máy tính

GVHD: TS. Lê Trọng Nhân

—o0o—

Học viên: Nguyễn Trương Minh Hoàng

Mã số học viên: 2270757

Thành phố Hồ Chí Minh, 05 /2026

XÁC NHẬN CỦA GIẢNG VIÊN

Xác nhận chữ ký

LỜI CAM ĐOAN

Tôi cam đoan rằng nghiên cứu này là công trình của riêng tôi, được thực hiện dưới sự giám sát và hướng dẫn của TS. Lê Trọng Nhân. Kết quả nghiên cứu của tôi là hợp pháp và chưa được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu được sử dụng trong nghiên cứu này đều được tôi thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong nghiên cứu này, tôi cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả đều đã được trích dẫn một cách thích hợp. Trong mọi trường hợp đạo văn, tôi hoàn toàn chịu trách nhiệm về hành động của mình. Do đó, Trường Đại học Bách Khoa - ĐHQG TP.HCM không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào được thực hiện trong nghiên cứu của tôi.

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đóng góp vào việc phát triển và hoàn thiện luận văn này.

Trước tiên, tôi xin gửi lời tri ân sâu sắc nhất đến người hướng dẫn khoa học của tôi, TS. Lê Trọng Nhân, vì sự hướng dẫn quý báu, động viên và phản hồi mang tính xây dựng trong suốt quá trình hình thành đề tài này. Chuyên môn và những hiểu biết sâu sắc của thầy đã đóng vai trò quan trọng trong việc định hướng nghiên cứu của tôi.

Tôi cũng vô cùng biết ơn Trường Đại học Bách Khoa - ĐHQG TP.HCM và Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp môi trường học thuật nuôi dưỡng và các nguồn lực cần thiết để thực hiện nghiên cứu này.

Lời cảm ơn đặc biệt gửi đến các bạn đồng nghiệp và đồng môn đã chia sẻ quan điểm và đưa ra những gợi ý sâu sắc, làm phong phú thêm phạm vi của nghiên cứu này.

Cuối cùng, tôi vô cùng biết ơn gia đình và bạn bè vì sự ủng hộ không ngừng, sự kiên nhẫn và động lực khi tôi đắm mình vào nỗ lực đầy thách thức nhưng bổ ích này.

Đề xuất nghiên cứu này là sự phản ánh của sự hỗ trợ và cống hiến tập thể, và tôi xin chân thành cảm ơn tất cả những người đã đóng góp trực tiếp hoặc gián tiếp vào việc hiện thực hóa nó.

TÓM TẮT

Sự phát triển của học máy và điện toán biên đã cách mạng hóa các ứng dụng theo dõi chuyển động, cho phép các hệ thống thời gian thực, tiết kiệm năng lượng và chính xác cao cho nhiều trường hợp sử dụng trong y tế, phân tích thể thao và tương tác người-máy. Tuy nhiên, việc phát triển các mô hình AI cho theo dõi chuyển động vẫn còn phức tạp, đòi hỏi kiến thức chuyên môn về tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và triển khai trên thiết bị biên. Bài báo này trình bày một framework toàn diện giúp đơn giản hóa quy trình end-to-end để tạo ra các mô hình AI được tùy chỉnh cho theo dõi chuyển động con người trên các thiết bị biên có tài nguyên hạn chế.

Framework giới thiệu một phương pháp tiền xử lý tương tác mới với tính năng lựa chọn cửa sổ thời gian kéo thả, cho phép người dùng chú thích và phân đoạn dữ liệu chuyển động một cách hiệu quả mà không cần kiến thức lập trình sâu rộng. Hệ thống hỗ trợ nhiều thuật toán học máy bao gồm Random Forest, Support Vector Machines và Neural Networks, với khả năng tự động sinh mã cho nhiều họ thiết bị và môi trường thực thi khác nhau như các bo tương thích Arduino, ARM Cortex-M thuần, ESP-IDF, MicroPython và Zephyr. Kiến trúc tạo mã nhận biết tối ưu hóa xem xét nhiều chiến lược triển khai—độ chính xác, tốc độ, tiêu thụ điện năng và phương án cân bằng—điều chỉnh gói mã sinh ra theo ràng buộc của từng đích triển khai.

Để kiểm chứng framework ở mức thực nghiệm, chúng tôi sử dụng bộ dữ liệu Nhận dạng Hoạt động Con Người (HAR) được thu thập ở 100Hz từ cảm biến IMU 6 trục (LSM6DS3) trên Seeed XIAO nRF52840 Sense như một nền tảng tham chiếu thuộc họ ARM Cortex-M4F. Trong nghiên cứu bằng chứng khái niệm này sử dụng dữ liệu đơn đối tượng trên năm lớp hoạt động, năm thuật toán (Random Forest, SVM, Neural Network, PyTorch MLP, PyTorch 1D-CNN) đạt độ chính xác 97,38–99,13% trên tập kiểm tra, với PyTorch MLP và 1D-CNN đạt cao nhất (99,13%). Quan trọng hơn, cùng một pipeline huấn luyện có thể được ánh xạ sang nhiều đích triển khai thông qua hệ thống generator phân tầng theo mô hình, nền tảng và backend thực thi. Dữ liệu tham khảo trên thiết bị cho thấy mô hình PyTorch 1D-CNN đạt 99,4% độ chính xác triển khai với tỷ lệ từ chối chỉ 0,6%. Phân tích so sánh với các nền tảng thương mại (Edge Impulse, SensiML) cho thấy framework mã nguồn mở của chúng tôi cung cấp tính minh bạch hoàn toàn, tính linh hoạt tiền xử lý vượt trội và chi phí bằng không.

Framework giải quyết các khoảng trống quan trọng trong các giải pháp hiện tại bằng cách cung cấp: (1) quy trình làm việc dựa trên GUI trực quan để tiếp cận cho

người không chuyên, (2) kiểm soát hoàn toàn đường ống tiền xử lý với phản hồi trực quan, (3) triển khai đa thiết bị với các tối ưu hóa đặc thù cho từng đích, và (4) kiến trúc có thể mở rộng hỗ trợ tích hợp các thuật toán, backend và nền tảng bổ sung trong tương lai. Nghiên cứu này đóng góp vào việc dân chủ hóa phát triển Edge AI cho các ứng dụng theo dõi chuyển động, với tác động tiềm năng trong phục hồi chức năng y tế, giám sát hiệu suất thể thao và công nghệ hỗ trợ.

Mục lục

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
TÓM TẮT	iv
Mục lục	vi
Danh sách bảng	xi
Danh sách hình vẽ	xii
1 Giới thiệu	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.2 Phát biểu bài toán	2
1.3 So sánh với các nền tảng hiện có	3
1.4 Mục tiêu nghiên cứu	4
1.5 Đóng góp nghiên cứu	5
1.6 Phạm vi và hạn chế	6
1.7 Tổ chức luận văn	6
2 Cơ sở lý thuyết	8
2.1 Tổng quan	8
2.2 Các lĩnh vực ứng dụng của theo dõi chuyển động	8
2.3 Theo dõi chuyển động dựa trên cảm biến	9
2.4 Theo dõi chuyển động dựa trên thị giác	9
2.5 AI trong theo dõi chuyển động	10
2.6 Các Framework tạo mô hình	10
2.7 Tiền xử lý dữ liệu và tạo cửa sổ	12
2.8 Trích xuất và biểu diễn đặc trưng	12

2.9	Triển khai biên và tối ưu hóa	13
2.10	Các khoảng trống trong công trình hiện có	14
2.11	Tóm tắt	15
3	Phương pháp luận	16
3.1	Tổng quan kiến trúc Framework	16
3.2	Dữ liệu cảm biến quán tính cho nhận dạng hoạt động	17
3.2.1	Gia tốc kế và con quay hồi chuyển: Tính bổ sung thông tin . . .	17
3.2.2	Tần số lấy mẫu: Định lý Nyquist và nội dung phổ hoạt động . .	18
3.3	Quy trình tiền xử lý tương tác	19
3.3.1	Tải lên và trực quan hóa dữ liệu	19
3.3.2	Các phương pháp lọc và làm sạch tín hiệu	19
3.3.3	Phân đoạn và sinh cửa sổ huấn luyện	23
3.4	Trích xuất đặc trưng	24
3.4.1	Tham số cửa sổ và ràng buộc nhất quán	24
3.4.2	Các loại trích xuất đặc trưng	24
3.4.3	Chiến lược đệm cửa sổ (Window Padding Strategy)	26
3.4.4	Tăng cường dữ liệu cho huấn luyện (Data Augmentation)	27
3.4.5	Ngưỡng tin cậy cho từ chối hoạt động không xác định	29
3.5	Huấn luyện mô hình học máy	30
3.5.1	Các thuật toán học máy áp dụng	30
3.5.2	Giao thức huấn luyện và đánh giá	32
3.5.3	Tối ưu hóa siêu tham số	33
3.6	Tạo mã cho triển khai biên	34
3.6.1	Kiến trúc	34
3.6.2	Lý do chọn tạo mã trực tiếp thay vì Runtime Engine	35
3.6.3	Triển khai TFLite Micro	39
3.6.4	Cấu trúc mã đã tạo	40
3.6.5	Lượng tử hóa mô hình cho triển khai biên	41
3.6.6	Kiến trúc tạo mã đa kiến trúc	44
3.6.7	Các chiến lược tối ưu hóa	45
3.6.8	Các Điều chỉnh Đặc thù Nền tảng	46
3.7	Thiết kế thực nghiệm	47
3.7.1	Tập dữ liệu	47
3.7.2	Các số liệu đánh giá	48
3.7.3	So sánh cơ sở	49

4	Thiết kế và hiện thực	50
4.1	Kiến trúc hệ thống	50
4.1.1	Ngăn xếp công nghệ và môi trường phát triển	50
4.1.2	Kiến trúc ứng dụng	51
4.1.3	Quản lý trạng thái	52
4.2	Module quản lý dữ liệu	53
4.2.1	Giao diện tải dữ liệu	53
4.2.2	Trực quan hóa tín hiệu đồng bộ	54
4.2.3	Phát hiện tần số lấy mẫu	56
4.3	Module tiền xử lý tương tác	56
4.3.1	Giao diện lựa chọn đoạn tương tác	56
4.3.2	Sinh cửa sổ trượt	57
4.3.3	Trực quan hóa lọc tín hiệu	58
4.4	Module trích xuất đặc trưng	58
4.4.1	Chế độ trích xuất đặc trưng	58
4.4.2	Tích hợp data augmentation	59
4.4.3	Phân chia train/validation/test	60
4.5	Module huấn luyện mô hình	61
4.5.1	Lựa chọn thuật toán	61
4.5.2	Tối ưu hóa siêu tham số	62
4.5.3	Theo dõi tiến trình real-time	62
4.5.4	Serialization mô hình	63
4.6	Module tạo mã triển khai	64
4.6.1	Kiến trúc Code Generator: Factory Pattern	64
4.6.2	Hệ thống phân cấp Generator	64
4.6.3	Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend	65
4.6.4	Feature reordering tại thời điểm sinh mã	66
4.6.5	Bốn chế độ tối ưu hóa	67
4.6.6	Gói đầu ra triển khai	69
4.6.7	Kiến trúc đa mô hình: CNN code generation	70
4.7	Module kiểm tra thiết bị	73
4.7.1	Giao tiếp serial	73
4.7.2	Quy trình xác minh triển khai	74
4.8	Luồng dữ liệu và quản lý trạng thái tổng thể	75
4.8.1	Luồng dữ liệu end-to-end	75
4.8.2	Cơ chế rollback và retry	76

4.8.3	Truy xuất thông tin model	77
4.9	Tổng kết chương	78
5	Kết quả thực nghiệm	79
5.1	Thiết lập thực nghiệm	79
5.2	Kết quả huấn luyện	80
5.2.1	Bài toán 3 lớp (running / still / walking)	80
5.2.2	Bài toán 5 lớp — Đánh giá chính	80
5.2.3	Hiệu suất theo từng lớp	81
5.3	So sánh hiệu suất giữa các thuật toán	83
5.4	Khả năng triển khai đa thiết bị	84
5.5	Đánh giá triển khai trên thiết bị	85
5.5.1	Tổng hợp kết quả triển khai	85
5.5.2	Phân tích theo từng mô hình	86
5.5.3	Bài học: khoảng cách huấn luyện–triển khai	87
5.6	So sánh với Edge Impulse	87
5.6.1	So sánh hiệu suất	87
5.6.2	So sánh tài nguyên	88
5.6.3	So sánh kiến trúc	89
5.7	Tóm tắt	89
6	Thảo Luận và Kết Luận	91
6.1	Diễn Giải Các Phát Hiện Chính	91
6.1.1	Hiệu Suất Mô Hình và Lựa Chọn Thuật Toán	91
6.1.2	Cân Bằng Lớp trong Thực hành	91
6.1.3	Tính Khả Thi Triển Khai Biên	91
6.1.4	Tiền Xử Lý Tương Tác: Một Thay Đổi Mô Hình	92
6.2	So Sánh với Công Trình Liên Quan	92
6.2.1	Các Nền Tảng Thương Mại	92
6.2.2	Các Hệ Thống HAR Học Thuật	93
6.3	Các Đánh Đổi Thiết Kế	93
6.3.1	ML Cổ Điển vs Học Sâu	93
6.3.2	Đặc Trưng Thủ Công vs Đặc Trưng Học	93
6.3.3	Tạo Mã Trực tiếp vs Runtime Suy luận	93
6.3.4	Các Chế Độ Tối Ưu Hóa	94
6.4	Tương Đồng Huấn Luyện-Triển Khai trong Edge ML	94
6.4.1	Các Nguồn Không Khớp Được Xác Định	94

6.4.2	Tác Động Thực Tế	95
6.4.3	Danh Sách Kiểm Tra Tương Đồng	95
6.4.4	So Sánh với Nền Tảng Thương Mại	96
6.4.5	Bộ Lọc Kalman: Đột Phá về Parity	96
6.5	Tăng Cường Dữ Liệu và Ngưỡng Tin Cây	97
6.5.1	Tăng Cường Nhận Biết Lớp	97
6.5.2	Ngưỡng Tin Cây cho Từ Chối Hoạt Động Không Xác Định . . .	97
6.6	Phân Tích Các Phát Hiện Kỹ Thuật Quan Trọng	98
6.6.1	Kiến Trúc Tạo Mã Đa Kiến Trúc	98
6.6.2	Hạn Chế TFLite và Giải Pháp Keras Surrogate	98
6.7	Các Hạn Chế	98
6.7.1	Hạn Chế Tập Dữ Liệu	98
6.7.2	Lựa Chọn Kích Thước Cửa Sổ	99
6.7.3	Vị Trí và Hướng Cảm Biến	99
6.7.4	Sự Đa Dạng Nền Tảng Tính Toán	99
6.7.5	Tính Hợp Lệ Bên Ngoài	99
6.8	Hướng Phát Triển Tương Lai	99
6.8.1	Các Mở Rộng Ngắn Hạn	99
6.8.2	Tầm Nhìn Dài Hạn	100
6.8.3	Các Cải Tiến Khả Năng Sử Dụng	101
6.9	Kết Luận	101
6.9.1	Tóm Tắt Đóng Góp	101
6.9.2	Xem Lại Các Mục Tiêu Nghiên Cứu	102
6.9.3	Các Ứng Dụng Thực Tế	102
6.9.4	Suy Nghĩ Cuối Cùng	103
Tài liệu tham khảo		104

Danh sách bảng

1.3.1	So sánh các Nền tảng Edge ML cho Theo dõi Chuyển động	3
3.3.1	So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai	22
3.4.1	Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai	25
3.6.1	Ma trận đích triển khai được hỗ trợ	35
3.6.2	So sánh phương pháp triển khai mô hình ML trên vi điều khiển	37
3.6.3	So sánh chiến lược lượng tử hóa theo phương pháp tạo mã	43
4.4.1	Các chế độ trích xuất đặc trưng và số lượng features	58
4.5.1	Thuật toán học máy được hỗ trợ	61
4.6.1	Chế độ tối ưu hóa mã triển khai	67
5.2.1	Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)	80
5.2.2	Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)	81
5.2.3	Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)	82
5.3.1	So sánh tổng hợp các thuật toán — bài toán 5 lớp	83
5.3.2	Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra	84
5.4.1	Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã	84
5.5.1	Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840	85
5.6.1	So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu	88
5.6.2	So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz	88
5.6.3	So sánh kiến trúc với các nền tảng thương mại	89
6.4.1	Danh sách kiểm tra tương đồng huấn luyện-triển khai	96

Danh sách hình vẽ

1.4.1	Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)	5
3.1.1	Tổng quan kiến trúc framework: quy trình sáu bước từ tải dữ liệu thô đến triển khai mô hình trên thiết bị biên, hỗ trợ các thuật toán RF, SVM, MLP, CNN và xuất mã đa nền tảng	16
4.2.1	Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiến xử lý.	55
4.3.1	Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.	57
4.4.1	Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.	60
4.5.1	Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.	63
4.6.1	Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.	70

5.2.1	Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down).	83
-------	---	----

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Theo dõi chuyển động (Motion tracking) là một lĩnh vực năng động và phát triển nhanh chóng, liên quan đến việc thu thập, xử lý và phân tích chuyển động của các đối tượng, cá nhân hoặc môi trường. Công nghệ này đóng vai trò then chốt trong nhiều lĩnh vực bao gồm y tế (phục hồi chức năng, phát hiện té ngã, giám sát bệnh nhân), thể thao (theo dõi hiệu suất, phòng ngừa chấn thương), trò chơi điện tử và thực tế ảo/thực tế tăng cường (trải nghiệm người dùng nhập vai), và robot học (hệ thống định vị, cơ chế điều khiển chính xác)^[43].

Thị trường toàn cầu cho công nghệ theo dõi chuyển động đạt 2,8 tỷ USD vào năm 2023 và dự kiến tăng trưởng với tốc độ tăng trưởng kép hàng năm (CAGR) là 18,4% đến năm 2030^[2]. Sự tăng trưởng này được thúc đẩy đặc biệt bởi các thiết bị đeo y tế và ứng dụng nhà thông minh. Sự quan tâm trong giới học thuật cũng tăng vọt, với các công bố nghiên cứu về nhận dạng hoạt động con người (Human Activity Recognition - HAR) tăng từ 487 bài báo vào năm 2018 lên hơn 1.500 bài vào năm 2023, tương ứng với mức tăng 3,1 lần chỉ trong năm năm^[38]. Sự hội tụ của các cảm biến IoT giá cả phải chăng, thuật toán học máy hiệu quả và khả năng tính toán biên (edge computing) đã dân chủ hóa việc theo dõi chuyển động từ các phòng thí nghiệm chuyên biệt sang các ứng dụng tiêu dùng hàng ngày.

Những tiến bộ trong công nghệ cảm biến và trí tuệ nhân tạo (AI) đã mang tính chuyển đổi, làm tăng đáng kể độ chính xác, hiệu suất và tính linh hoạt của các hệ thống theo dõi chuyển động. Đặc biệt, các mô hình AI là không thể thiếu trong việc diễn giải dữ liệu chuyển động, cho phép các ứng dụng như nhận dạng cử chỉ, phân loại hoạt động và mô hình hóa chuyển động dự đoán. Tuy nhiên, quá trình phát triển các mô hình AI này vốn dĩ phức tạp, bao gồm thu thập dữ liệu, tiền xử lý và lọc để loại bỏ

nhiều, trích xuất các đặc trưng có ý nghĩa, và huấn luyện các mô hình học máy hoặc học sâu. Sự phức tạp này đặt ra những thách thức đáng kể, đặc biệt đối với các nhà phát triển và nhà nghiên cứu có thể thiếu chuyên môn về AI hoặc công nghệ theo dõi chuyển động^[26].

Sự phát triển của tính toán biên (edge computing) đã làm phức tạp thêm bối cảnh này trong khi đồng thời mang lại những cơ hội mới. Các thiết bị biên—vi điều khiển bị giới hạn tài nguyên với bộ nhớ, sức mạnh xử lý và ngân sách năng lượng hạn chế—cho phép các ứng dụng theo dõi chuyển động theo thời gian thực, bảo vệ quyền riêng tư và tiết kiệm năng lượng. Tuy nhiên, triển khai các mô hình AI trên các thiết bị như vậy đòi hỏi kiến thức chuyên biệt về tối ưu hóa mô hình, tạo mã và các chi tiết triển khai đặc thù cho từng nền tảng. Thách thức này trở nên rõ rệt hơn khi hệ sinh thái thiết bị biên ngày càng phân mảnh: cùng một mô hình có thể cần được đóng gói khác nhau cho các bo tương thích Arduino, ARM Cortex-M thuần, SDK như ESP-IDF, MicroPython hoặc RTOS như Zephyr.

1.2 Phát biểu bài toán

Bất chấp những tiến bộ trong công nghệ theo dõi chuyển động, việc tạo ra các mô hình AI cho phân tích chuyển động vẫn là một nhiệm vụ đầy thách thức đặt ra một số rào cản quan trọng:

1. **Độ phức tạp Kỹ thuật:** Các cách tiếp cận hiện tại đòi hỏi kiến thức kỹ thuật sâu rộng trải rộng nhiều lĩnh vực—xử lý tín hiệu cho tiền xử lý dữ liệu, học máy cho phát triển mô hình và hệ thống nhúng cho triển khai biên.
2. **Thách thức Tiền xử lý:** Dữ liệu cảm biến chuyển động vốn dĩ nhiễu và đòi hỏi tiền xử lý cẩn thận. Các công cụ hiện có hoặc cung cấp kiểm soát hạn chế đối với tiền xử lý (các cách tiếp cận hộp đen tự động) hoặc đòi hỏi lập trình mở rộng để triển khai các quy trình tiền xử lý tùy chỉnh.
3. **Tính linh hoạt Hạn chế:** Nhiều framework có sẵn có đường cong học tập dốc hoặc cung cấp chức năng hạn chế để tùy chỉnh mô hình theo các yêu cầu theo dõi chuyển động cụ thể. Các nền tảng thương mại như Edge Impulse và SensiML cung cấp quy trình làm việc đơn giản hóa nhưng với chi phí đáng kể (\$20-500/tháng) và với các tùy chọn tùy chỉnh bị hạn chế.
4. **Độ phức tạp Triển khai:** Dịch cùng một mô hình đã huấn luyện thành các gói mã tối ưu hóa cho nhiều họ thiết bị biên khác nhau đòi hỏi hiểu biết sâu về kiến

1.3. So sánh với các nền tảng hiện có

trúc phần cứng đích, ràng buộc bộ nhớ, hệ sinh thái công cụ (Arduino, ESP-IDF, RTOS, MicroPython) và kỹ thuật tối ưu hóa phù hợp cho từng trường hợp triển khai.

5. **Rào cản Khả năng tiếp cận:** Sự kết hợp của các yếu tố này hạn chế đổi mới và cản trở việc áp dụng rộng rãi các giải pháp theo dõi chuyển động được hỗ trợ bởi AI, đặc biệt trong bối cảnh nghiên cứu và giáo dục.

Do đó, có một nhu cầu cấp thiết về một framework linh hoạt, trực quan và thân thiện với người dùng giúp đơn giản hóa việc tạo các mô hình AI phù hợp cho các ứng dụng theo dõi chuyển động trong khi cung cấp kiểm soát hoàn toàn đối với quy trình phát triển.

1.3 So sánh với các nền tảng hiện có

Để đặt các đóng góp trong ngữ cảnh, Bảng 1.3.1 so sánh framework này với các lựa chọn thay thế thương mại và mã nguồn mở hàng đầu cho triển khai edge ML trong các ứng dụng theo dõi chuyển động.

Bảng 1.3.1: So sánh các Nền tảng Edge ML cho Theo dõi Chuyển động

Tính năng	Edge Impulse	SensiML	TensorFlow Lite	Framework này
Chi phí	\$20-99/tháng	\$99-500/tháng	Miễn phí	Miễn phí (Mã nguồn mở)
Tiền xử lý tương tác	Hạn chế (web UI)	Có (analytics studio)	Không (chỉ mã)	Có (cửa sổ kéo thả)
Thuật toán	NN, transfer learning	RF, SVM, NN, boosting	Chỉ deep learning	RF, SVM, NN
Tạo mã	C++ (Arduino, ARM)	C (MCU-cụ thể)	C++ (TFLite Micro)	C/C++/Python đa đích
Chế độ tối ưu hóa	Accuracy/latency	AutoML	Quant. thủ công	4 chế độ (acc./speed/power/bal.)
Mất cân bằng lớp	Oversample thủ công	Cân bằng tự động	Không tích hợp	Tự động (class_weight)
Tùy chỉnh	Hạn chế	Trung bình	Cao (OSS)	Cao (modular)
Sử dụng giáo dục	Gói miễn phí	Giấy phép học thuật	Có	Có (không giới hạn)
Nền tảng triển khai	20+ boards	MCUs đã chọn	Tương thích TFLite	Arduino, ESP-IDF, ARM, MicroPython, Zephyr
Kiểm soát dữ liệu huấn luyện	Lưu trữ đám mây	Local/cloud	Chỉ local	Local (bảo mật hoàn toàn)

Các yếu tố phân biệt chính của framework này bao gồm: (1) **Tiền xử lý tương tác mới** với chọn cửa sổ thời gian kéo thả, giảm thời gian chú thích so với các cách tiếp cận dựa trên mã hoặc nhấp chuột; (2) **Hỗ trợ đa thuật toán** với các API nhất quán, cho phép thử nghiệm nhanh chóng mà không cần chuyển đổi nền tảng; (3) **Tính minh bạch hoàn toàn** thông qua tính khả dụng mã nguồn mở, cho phép các nhà nghiên cứu hiểu, xác thực và mở rộng toàn bộ quy trình; (4) **Chi phí bằng không** cho sử dụng không giới hạn, loại bỏ rào cản tài chính cho sinh viên, nhà nghiên cứu và các khu vực đang phát triển; (5) **Tạo mã nhận thức tối ưu hóa** đánh đổi rõ ràng giữa độ chính xác, tốc độ và tiêu thụ điện năng dựa trên yêu cầu ứng dụng.

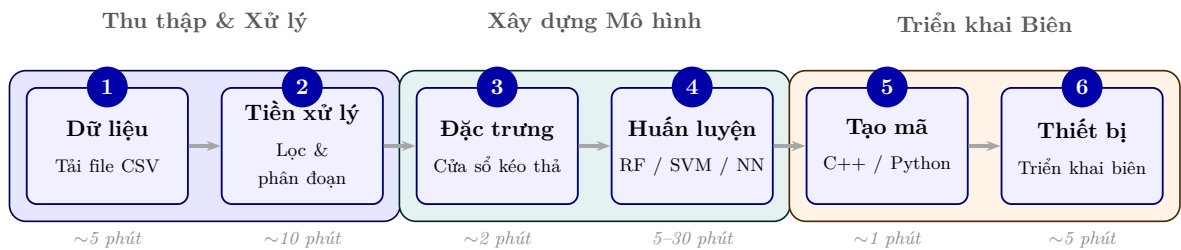
Trong khi các nền tảng thương mại cung cấp lợi thế về độ trưởng thành của hệ sinh thái (hỗ trợ 20+ board của Edge Impulse) và sự tinh vi của AutoML (lựa chọn đặc trưng tự động của SensiML), chúng áp đặt chi phí định kỳ (\$240-2.388/năm), hạn chế tùy chỉnh thông qua các API độc quyền và tạo ra sự phụ thuộc nhà cung cấp. TensorFlow Lite cung cấp khả năng học sâu mạnh mẽ nhưng đòi hỏi chuyên môn ML đáng kể và không cung cấp công cụ tiền xử lý cấp cao. Framework này nhắm đến khoảng trống giữa tính dễ sử dụng và tính linh hoạt, cung cấp một giải pháp có thể tiếp cận nhưng mạnh mẽ cho bối cảnh giáo dục và nghiên cứu^[11;14].

1.4 Mục tiêu nghiên cứu

Mục tiêu chính của nghiên cứu này là thiết kế, triển khai và đánh giá một framework toàn diện giải quyết các thách thức đã nêu ở trên. Cụ thể, nghiên cứu này nhằm mục đích:

1. **Phát triển hệ thống tiền xử lý tương tác:** Tạo giao diện tiền xử lý dựa trên GUI mới với chọn cửa sổ thời gian kéo thả, cho phép chú thích và phân đoạn dữ liệu hiệu quả mà không cần yêu cầu lập trình.
2. **Triển khai hỗ trợ đa thuật toán:** Tích hợp nhiều thuật toán học máy (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động.
3. **Thiết kế tạo mã bất khả tri nền tảng:** Xây dựng hệ thống tạo mã modular có khả năng sản xuất các triển khai C/C++/MicroPython tối ưu hóa cho nhiều họ đích khác nhau (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với xem xét các chiến lược tối ưu hóa khác nhau (độ chính xác, tốc độ, điện năng, cân bằng).

4. **Xác thực Hiệu quả Framework:** Tiến hành các thử nghiệm toàn diện sử dụng dữ liệu Nhận dạng Hoạt động Con người thực tế để đánh giá hiệu suất mô hình, kiểm tra năng lực sinh mã cho nhiều đích triển khai và đo benchmark đại diện trên phần cứng ARM Cortex-M4F tham chiếu.
5. **Đảm bảo tương đồng và độ tin cậy huấn luyện-triển khai:** Tích hợp bộ lọc Kalman và kiến trúc tạo mã nhận biết kiến trúc để đảm bảo tương đồng chính xác giữa mô hình huấn luyện và mã triển khai; tích hợp ngưỡng tin cậy để từ chối hoạt động không xác định và sử dụng tăng cường dữ liệu nhận biết lớp để đảm bảo độ tin cậy trên phần cứng thực.



Hình 1.4.1: Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)

1.5 Đóng góp nghiên cứu

Nghiên cứu này đóng góp các điểm chính sau vào lĩnh vực theo dõi chuyển động dựa trên biên:

- **Tiền xử lý tương tác mới:** Giao diện chọn cửa sổ thời gian kéo thả đầu tiên cho chú thích dữ liệu chuyển động, giảm đáng kể thời gian tiền xử lý và lỗi chú thích so với các cách tiếp cận thủ công.
- **Kiến trúc tạo mã đa thiết bị nhận thức tối ưu hóa:** Hệ thống tạo mã modular ánh xạ cùng một mô hình đã huấn luyện sang nhiều họ đích (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với các cấu hình tối ưu hóa riêng theo độ chính xác, tốc độ và tiêu thụ điện năng.
- **Cơ chế tương đồng huấn luyện-triển khai:** Quy trình xác minh đảm bảo mã C++ được tạo ra cho kết quả nhận dạng nhất quán với mô hình Python đã huấn luyện, bao gồm bộ lọc Kalman, công thức thống kê chính xác và sắp xếp lại trọng số mô hình tại thời điểm tạo mã.

- **Framework mã nguồn mở toàn diện:** Quy trình hoàn chỉnh từ đầu đến cuối từ tải dữ liệu đến triển khai biên, cung cấp lựa chọn thay thế mã nguồn mở miễn phí cho các nền tảng thương mại.

1.6 Phạm vi và hạn chế

Nghiên cứu này tập trung vào các ứng dụng theo dõi chuyển động dựa trên IMU (gia tốc kế và con quay hồi chuyển). Phạm vi bao gồm:

- Dữ liệu cảm biến IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục)
- Các phương pháp học có giám sát sử dụng các thuật toán scikit-learn
- Tạo mã cho nhiều họ nền tảng biên dựa trên vi điều khiển và môi trường thực thi nhúng
- Nhận dạng hoạt động con người là lĩnh vực xác thực chính

Các số đo phần cứng định lượng trong luận văn này tập trung vào một nền tảng tham chiếu (Seeed XIAO nRF52840), trong khi các đích còn lại được phân tích ở mức kiến trúc generator và khả năng sinh mã.

Nghiên cứu không giải quyết:

- Hợp nhất cảm biến đa phương thức (kết hợp IMU với camera, âm thanh, v.v.)
- Tích hợp các framework học sâu (TensorFlow, PyTorch)
- Cập nhật mô hình theo thời gian thực hoặc các kịch bản học liên kết
- Các lĩnh vực chuyên biệt yêu cầu chứng nhận y tế

1.7 Tổ chức luận văn

Phần còn lại của luận văn này được tổ chức như sau: Chương 2 đánh giá các công trình liên quan về hệ thống theo dõi chuyển động, framework tạo mô hình AI và công cụ triển khai biên. Chương 3 trình bày phương pháp luận chi tiết bao gồm cơ sở lý thuyết, quy trình tiền xử lý, trích xuất đặc trưng, phương pháp huấn luyện mô hình và chiến lược tạo mã. Chương 4 mô tả thiết kế và hiện thực hệ thống, bao gồm kiến trúc phần mềm, giao diện người dùng và chi tiết triển khai các module. Chương 5 báo cáo kết quả thực nghiệm bao gồm các số liệu hiệu suất mô hình, đặc điểm triển khai

và phân tích so sánh. Chương 6 thảo luận về các phát hiện, hạn chế, ý nghĩa và kết luận với tóm tắt đóng góp cùng hướng phát triển tương lai.

Chương 2

Cơ sở lý thuyết

2.1 Tổng quan

Chương này trình bày cơ sở lý thuyết trên sáu lĩnh vực: các ứng dụng theo dõi chuyển động, các phương thức cảm biến, kiến trúc AI cho HAR, các framework tạo mô hình, phương pháp tiền xử lý và trích xuất đặc trưng, và kỹ thuật tối ưu hóa triển khai biên. Phân tích dựa trên 40+ công bố từ 2001–2024, với trọng tâm vào các tiến bộ TinyML (2020–2024) và học sâu cho HAR. Dựa trên đánh giá này, các khoảng trống nghiên cứu được xác định để định vị framework đề xuất.

2.2 Các lĩnh vực ứng dụng của theo dõi chuyển động

Theo dõi chuyển động con người là một công nghệ nền tảng trên nhiều lĩnh vực y tế, thể thao và thể dục, thực tế ảo/thực tế tăng cường (VR/AR), robot học và các hệ thống tương tác mới nổi. Trong y tế, các cảm biến đeo và môi trường xung quanh cho phép phân tích dáng đi, phát hiện té ngã (đạt độ nhạy 95-98%^[36]), và giám sát phục hồi chức năng, hỗ trợ các can thiệp cá nhân hóa^[43]. Các ứng dụng thể thao tận dụng dữ liệu quán tính và sinh cơ học để tối ưu hóa hiệu suất và phòng ngừa chấn thương, trong khi các hệ thống VR/AR phụ thuộc vào theo dõi khung xương và vị trí chính xác cho trải nghiệm người dùng nhập vai. Robot học tích hợp theo dõi chuyển động để điều hướng, thao tác và tương tác người-robot. Các ứng dụng công nghiệp gần đây bao gồm giám sát an toàn lực lượng lao động^[37], đạt độ chính xác 94,3% trong phát hiện tư thế nguy hiểm trên các địa điểm xây dựng và sản xuất. Các lĩnh vực này cùng nhau tạo động lực cho nhu cầu về các framework theo dõi chuyển động được hỗ trợ bởi

AI có thể tiếp cận, thích ứng và hiệu quả, có khả năng suy luận biên theo thời gian thực với độ chính xác 90%+.

2.3 Theo dõi chuyển động dựa trên cảm biến

Các Đơn vị Đo lường Quán tính (IMUs)—kết hợp gia tốc kế và con quay hồi chuyển—được áp dụng rộng rãi cho theo dõi chuyển động do tiêu thụ điện năng thấp (dòng điện hoạt động 2-10 mA), hiệu quả chi phí (\$1-5 mỗi đơn vị) và tính phù hợp cho triển khai biên. Các khảo sát toàn diện về theo dõi chuyển động dựa trên IMU^[12] nhấn mạnh các thách thức bao gồm trôi cảm biến (độ lệch con quay hồi chuyển 0,1-1°/s), lỗi tích lũy trong ước lượng hướng, yêu cầu hiệu chuẩn và ràng buộc sinh cơ học. Các phương pháp hợp nhất đa cảm biến (ví dụ: IMU + từ kế + cảm biến áp suất) có thể cải thiện độ bền vững—đạt được mức tăng độ chính xác 2-5%^[12]—nhưng tăng độ phức tạp, tiêu thụ điện năng và chi phí. Công trình gần đây của Xia và cộng sự^[41] chứng minh các hệ thống IMU đơn đạt độ chính xác 96,8% trên các nhiệm vụ HAR 12 lớp sử dụng kiến trúc LSTM-CNN, xác thực tính khả thi của cách tiếp cận cảm biến đơn. Các tập dữ liệu HAR chuẩn như UCI-HAR^[4] và WISDM^[6] đều sử dụng IMU sáu trục gắn trên cơ thể ở tần số 50–100 Hz, qua đó xác lập cấu hình này làm thiết lập tham chiếu phổ biến cho các nghiên cứu HAR dựa trên thiết bị đeo. Framework trong luận văn này kế thừa cấu hình đó—thu thập và xử lý dữ liệu gia tốc kế và con quay hồi chuyển ba trục ở 100 Hz—và tập trung vào các quy trình IMU đơn đáng tin cậy, với khả năng mở rộng cho tích hợp đa cảm biến khi cần thiết.

2.4 Theo dõi chuyển động dựa trên thị giác

Các phương pháp thị giác máy tính sử dụng camera lập thể, theo dõi không dấu và ước lượng tư thế cung cấp các biểu diễn không gian phong phú về chuyển động con người^[42]. Trong khi các hệ thống thị giác cho phép tái tạo toàn thân (phát hiện 17+ điểm khóa tại 30 FPS^[15]) và được sử dụng trong hoạt hình và phân tích lâm sàng, chúng gặp phải các thách thức về che khuất (suy giảm độ chính xác 15-30% trong các cảnh đông đúc), biến đổi môi trường (ánh sáng, thời tiết), lo ngại về quyền riêng tư và tiêu thụ điện năng (500-2000 mW so với 20-50 mW cho hệ thống IMU). Các tiến bộ gần đây bao gồm MediaPipe pose^[15] (133 mốc tư thế, độ chính xác 95% trên các khớp nhìn thấy) và OpenPose đã cải thiện khả năng tiếp cận nhưng vẫn ít khả thi hơn cho các thiết bị đeo công suất siêu thấp. Các hệ thống kết hợp IMU-thị giác^[36] kết

hợp các điểm mạnh bổ sung—độ bền vững của IMU với che khuất, độ chính xác không gian của thị giác—nhưng yêu cầu đồng bộ hóa cảm biến và tăng độ phức tạp hệ thống. Đối với giám sát đeo liên tục ưu tiên tuổi thọ pin (ngày đến tuần), các giải pháp IMU thuần túy vẫn chiếm ưu thế.

2.5 AI trong theo dõi chuyển động

Các mô hình AI chuyển đổi dữ liệu chuyển động thô thành các diễn giải ngữ nghĩa—phân loại hoạt động, nhận dạng tư thế và dự báo quỹ đạo. Các phương pháp học máy truyền thống bao gồm Support Vector Machines (SVM)^[9] và Random Forests^[7] đã được sử dụng rộng rãi, đạt độ chính xác 88-93% trên các benchmark HAR chuẩn (UCI-HAR, WISDM) với kích thước bộ nhớ thời gian chạy nhỏ gọn (50–200 KB) phù hợp cho các thiết bị biên^[38]. Các kiến trúc học sâu đã từng bước cải thiện hiệu suất HAR: CNNs (độ chính xác 92-95%, trích xuất đặc trưng không gian từ các cửa sổ cảm biến), RNN/LSTMs (độ chính xác 94-97%, nắm bắt các phụ thuộc thời gian^[41]), và transformers dựa trên attention (độ chính xác 96-98% trên các tập dữ liệu phức tạp^[33]). Tuy nhiên, các mô hình học sâu đòi hỏi các tập dữ liệu được chú thích lớn hơn (10.000+ mẫu so với 1.000-5.000 cho ML cổ điển), bộ nhớ cao hơn (500 KB-2 MB so với 50-200 KB) và các chiến lược lượng tử hóa phức tạp hơn (số nguyên 8-bit hoặc độ chính xác hỗn hợp) trong triển khai biên^[22]. Các nghiên cứu so sánh gần đây^[38] cho thấy các mô hình ML cổ điển đạt độ chính xác 90-93% với dấu chân bộ nhớ nhỏ hơn 10× và suy luận nhanh hơn 5-20× (12-18 ms so với 60-120 ms mỗi cửa sổ) trên các mục tiêu Cortex-M4. Framework được đề xuất hỗ trợ chiến lược cả hai mô hình—mô hình cổ điển cho các kịch bản bị giới hạn tài nguyên, mạng nơ-ron nhẹ cho các ứng dụng quan trọng độ chính xác—với các chế độ tối ưu hóa đa mục tiêu giúp cân nhắc các đánh đổi một cách có căn cứ.

2.6 Các Framework tạo mô hình

Một số nền tảng đã xuất hiện để đơn giản hóa việc tạo mô hình AI, mỗi nền tảng có điểm mạnh và hạn chế riêng biệt:

- **Edge Impulse^[11]**: Cung cấp các quy trình từ đầu đến cuối cho thu thập dữ liệu, tiền xử lý (phân tích phổ, MFE/MFCC), tạo đặc trưng, huấn luyện mô hình (CNN, LSTM, ML cổ điển) và triển khai cho 85+ mục tiêu vi điều khiển. Thu hút hơn 100.000 nhà phát triển, nền tảng này vượt trội trong tự động hóa quy trình (95% sự hài lòng của người dùng cho tạo mẫu nhanh^[23]) nhưng hạn

ché tính minh bạch trong logic tiền xử lý và tính phí \$20-99/tháng cho sử dụng thương mại. Các cửa sổ tiền xử lý được cố định sau khi tải lên; tính chỉnh tương tác yêu cầu tải lên lại dữ liệu.

- **SensiML Analytics Toolkit**^[32]: Cung cấp lựa chọn đặc trưng được điều khiển bởi AutoML (200+ đặc trưng ứng viên, tối ưu hóa thuật toán di truyền) và các gói kiến thức cho các thiết bị ARM Cortex-M. Đạt độ chính xác 92-96% trên các nhiệm vụ nhận dạng cử chỉ nhưng sử dụng các định dạng độc quyền và chi phí \$99-500/tháng. Tiền xử lý không cần lập trình nhưng thiếu minh bạch (lựa chọn đặc trưng theo kiểu hộp đen).
- **MediaPipe**^[15]: Cung cấp các quy trình nhận thức modular (tư thế, tay, mặt, theo dõi đối tượng) với thực thi đồ thị tối ưu hóa trên các thiết bị di động/biên. Chủ yếu hướng tới thị giác, ít phù hợp hơn cho các quy trình làm việc IMU tùy chỉnh yêu cầu các chiến lược phân đoạn tùy chỉnh. Không có hỗ trợ sẵn có cho phân đoạn chuỗi thời gian hoặc trích xuất đặc trưng HAR.
- **Teachable Machine**^[16]: Cho phép tạo mẫu nhanh các bộ phân loại hình ảnh, âm thanh và tư thế thông qua giao diện không mã. Được sử dụng trong 1M+ dự án giáo dục nhưng thiếu công cụ cho kỹ thuật đặc trưng nâng cao, phân đoạn chuỗi thời gian và triển khai tối ưu theo từng phần cứng (chỉ xuất mô hình TensorFlow.js).
- **TensorFlow Lite Micro**^[?] cung cấp thời gian chạy để triển khai các mô hình lượng tử hóa trên vi điều khiển, hỗ trợ tối ưu hóa thông qua lượng tử hóa sau huấn luyện (số nguyên 8-bit, float 16-bit) và tối ưu hóa suy luận thời gian chạy (CMSIS-NN cho ARM). Tuy nhiên, TFLite chỉ hỗ trợ mạng nơ-ron và yêu cầu thiết kế kiến trúc mô hình thủ công.
- **AutoML/NAS cho TinyML**: Nghiên cứu gần đây khám phá Neural Architecture Search (NAS) để khám phá các cấu trúc liên kết mạng tối ưu dưới các ràng buộc tài nguyên^[?]. Dù là hướng nghiên cứu triển vọng, NAS thường đòi hỏi tài nguyên tính toán đáng kể và thiếu tính minh bạch cho các bối cảnh giáo dục và nghiên cứu.

So với các nền tảng trên, Framework này giải quyết các hạn chế đó bằng cách cung cấp: (1) **GUI tiền xử lý tương tác** với chọn cửa sổ thời gian kéo thả cho phép phân đoạn chính xác, (2) **kỹ thuật đặc trưng minh bạch** với kiểm tra từng đặc trưng, (3) **tối ưu hóa đa mục tiêu** (các chế độ độ chính xác/tốc độ/điện năng/cân bằng)

được điều chỉnh theo ràng buộc, và (4) **khả năng tiếp cận mã nguồn mở** loại bỏ rào cản chi phí.

2.7 Tiền xử lý dữ liệu và tạo cửa sổ

Nhận dạng hoạt động chính xác phụ thuộc nhiều vào chất lượng của các chiến lược phân đoạn và tạo cửa sổ. Banos và cộng sự^[6] đánh giá có hệ thống ảnh hưởng của kích thước cửa sổ đến hiệu suất HAR: cửa sổ 1 giây đạt độ chính xác 89,3%, cửa sổ 2,5 giây đạt 92,1%, trong khi cửa sổ 5 giây mang lại 93,4% nhưng giảm khả năng phản hồi. Tỷ lệ chồng lấp cũng ảnh hưởng đến hiệu suất—chồng lấp 50% cải thiện độ chính xác 2-4% so với các cửa sổ không chồng lấp^[36] với chi phí tính toán gấp đôi. Lựa chọn bộ lọc ảnh hưởng đáng kể đến giảm nhiễu: bộ lọc thông thấp Butterworth (cutoff 20 Hz) bảo tồn các đặc tính chuyển động trong khi làm suy giảm nhiễu cảm biến, cải thiện độ chính xác 3-7%^[12]. Tuy nhiên, các quy trình tiền xử lý thông thường yêu cầu sửa đổi mã thủ công để điều chỉnh lọc, làm mịn hoặc loại bỏ hiện vật—các nhà phát triển báo cáo dành 40-60% thời gian dự án cho lặp lại tiền xử lý^[23]. Các công cụ tương tác có thể giảm gánh nặng này: Nghiên cứu về học máy tương tác^[3] chỉ ra rằng các giao diện trực quan giảm thời gian lặp lại 35% và giảm lỗi 28% so với các quy trình làm việc dựa trên mã. Framework được đề xuất nâng cao khả năng sử dụng bằng cách cho phép căn chỉnh trực quan trực tiếp của các tín hiệu thô và đã xử lý, điều chỉnh cửa sổ tương tác với phản hồi tức thì và gắn nhãn lại nhanh chóng—giảm rào cản nhận thức và kỹ thuật cho lặp lại nhanh.

2.8 Trích xuất và biểu diễn đặc trưng

Các đặc trưng miền thời gian thủ công (trung bình, phương sai, RMS, độ lệch, độ nhọn, tỷ lệ qua không) và miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) vẫn hiệu quả cho các mô hình ML cổ điển khi được trích xuất một cách có hệ thống^[24]. Các bộ đặc trưng điển hình dao động từ 60-150 đặc trưng mỗi cửa sổ (10-15 mỗi trục cảm biến)^[38]. Lựa chọn đặc trưng sử dụng thông tin hỗ tương, kiểm định F ANOVA hoặc loại bỏ đệ quy có thể giảm chiều 30-50% với mất độ chính xác tối thiểu (<2%), cải thiện dấu chân bộ nhớ và tốc độ suy luận (nhanh hơn 15-30%) trên vi điều khiển^[6]. Tuy nhiên, các đặc trưng miền tần số đặt ra các thách thức triển khai: tính toán FFT trên các hệ thống nhúng không có bộ tăng tốc phần cứng yêu cầu các phép toán $O(N \log N)$ và số học dấu phẩy động, tiêu thụ 50-150 ms mỗi cửa sổ trên Cortex-M4 (64 MHz)^[5]. Các lựa chọn thay thế nhẹ bao gồm đếm qua không, đỉnh tự tương quan

hoặc các thư viện FFT đã tính toán trước (ARM CMSIS-DSP), nhưng tương đồng đặc trưng giữa huấn luyện (Python NumPy FFT) và triển khai (C++ FFT điểm cố định) yêu cầu xác thực cẩn thận. Trong khi các mạng sâu từ đầu đến cuối có thể học các đặc trưng phân cấp một cách ngầm định—không cần thiết kể đặc trưng thủ công—trích xuất đặc trưng tường minh lại cải thiện khả năng diễn giải (xác định trục/tần số cảm biến nào điều khiển dự đoán) và tính minh bạch triển khai trong các lĩnh vực được quy định hoặc quan trọng về an toàn. Framework kết hợp các mô-đun tính toán đặc trưng hoán đổi được, xác thực tính nhất quán đặc trưng huấn luyện-triển khai và hỗ trợ chẩn đoán từng cửa sổ để hướng dẫn tinh chỉnh.

2.9 Triển khai biên và tối ưu hóa

Các ràng buộc tài nguyên xác định các thách thức triển khai biên—RAM hạn chế (32-256 KB), flash (256 KB-1 MB), không có đơn vị đầu phẩy động phần cứng (hoặc hiệu suất FPU hạn chế) và ngân sách điện năng (10-50 mW cho các thiết bị đeo) [5;27]. Benchmark MLPerf Tiny^[5] thiết lập các số liệu tham chiếu: Cortex-M4F (64 MHz) đạt 10,3 suy luận/giây cho phát hiện từ khóa (mô hình 49 KB), 15,2 suy luận/giây cho phát hiện bất thường (mô hình 32 KB). Các chiến lược tối ưu hóa bao gồm:

- **Lượng tử hóa:** Chuyển đổi float 32-bit sang số nguyên 8-bit giảm kích thước mô hình $4\times$ và tăng tốc suy luận $2-3\times$ với suy giảm độ chính xác $0,5-2\%$ [22]. Lượng tử hóa độ chính xác hỗn hợp (16-bit cho các lớp nhảy cảm) cân bằng kích thước và độ chính xác.
- **Cắt tỉa mô hình:** Loại bỏ 40-70% trọng số mạng nơ-ron (cắt tỉa dựa trên độ lớn hoặc cấu trúc) mang lại tăng tốc $2-3\times$ với mất độ chính xác $<1\%$ [30].
- **Chưng cất kiến thức:** Huấn luyện các mô hình sinh viên nhỏ gọn (10-20% tham số) để bắt chước các mạng giáo viên lớn đạt 85-95% độ chính xác giáo viên với đầu chân nhỏ hơn $5-10\times$ [10].
- **Đơn giản hóa thuật toán:** Các mô hình cổ điển như Random Forests (độ sâu ≤ 10 , 50-100 cây) và SVMs (100-500 vector hỗ trợ) đạt độ chính xác 88-93% với đầu chân 50-200 KB và thời gian suy luận 12-25 ms [38].

Các khảo sát TinyML gần đây [10;27] xác định các mẫu triển khai chính: (1) các ứng dụng quan trọng độ trễ ưu tiên ML cổ điển (suy luận 5-20 ms), (2) các kịch bản quan trọng độ chính xác biện minh cho CNNs/LSTMs nhẹ (suy luận 40-80 ms, độ chính xác cao hơn 1-2%), (3) các triển khai quan trọng điện năng sử dụng lấy mẫu thưa (1-5 Hz)

và lượng tử hóa tích cực. Thành phần tạo mã của framework thực hiện xuất được điều chỉnh theo nền tảng, phát ra mã C/C++ có cấu trúc với chuyển đổi tham số nhận thức ràng buộc (ví dụ: tỷ lệ float→int16, mở rộng vòng lặp cho kích thước cửa sổ cố định). Các chế độ tối ưu hóa đa mục tiêu (độ chính xác, tốc độ, điện năng, cân bằng) cho phép ra quyết định có nguyên tắc phù hợp với mục tiêu triển khai, đánh đổi định lượng độ chính xác ($\pm 2\text{-}5\%$), độ trễ ($\pm 10\text{-}30\text{ ms}$) và điện năng ($\pm 5\text{-}15\text{ mW}$).

2.10 Các khoảng trống trong công trình hiện có

Mặc dù hệ sinh thái công cụ phát triển edge AI đã khá hoàn thiện, một số khoảng trống quan trọng vẫn còn:

1. **Tiền xử lý tương tác hạn chế:** Ít hệ thống cho phép người dùng kiểm tra trực quan các tín hiệu cảm biến thô so với đã lọc và tinh chỉnh phân đoạn thủ công trong cùng một môi trường.
2. **Thiếu minh bạch trong trích xuất đặc trưng:** Các trình tạo đặc trưng tự động che giấu các bước dẫn xuất và cản trở việc sử dụng giáo dục hoặc tái tạo nghiên cứu.
3. **Sinh mã triển khai thiếu linh hoạt:** Đầu ra tạo mã thường là nguyên khối, làm phức tạp việc tùy chỉnh hoặc thích ứng theo phần cứng cụ thể.
4. **Đánh đổi khả năng tiếp cận và kiểm soát:** Các nền tảng hiện tại thường thiên về một trong hai thái cực—dễ sử dụng như Teachable Machine, hoặc khả năng cấu hình cao như TensorFlow—ít nền tảng nào cân bằng được cả hai yêu cầu cho các quy trình xử lý dữ liệu cảm biến chuyển động.

Framework được đề xuất giải quyết các khoảng trống này bằng cách thống nhất tiền xử lý tương tác, trích xuất đặc trưng minh bạch, huấn luyện modular và tạo mã nhận thức tối ưu hóa dưới một kiến trúc mở rộng. Những lựa chọn thiết kế này, được thông báo bởi tài liệu được xem xét ở trên, định vị luận văn này như một giải pháp bổ sung bắc cầu khoảng trống giữa sự dễ sử dụng thương mại và tính linh hoạt cấp độ nghiên cứu.

2.11 Tóm tắt

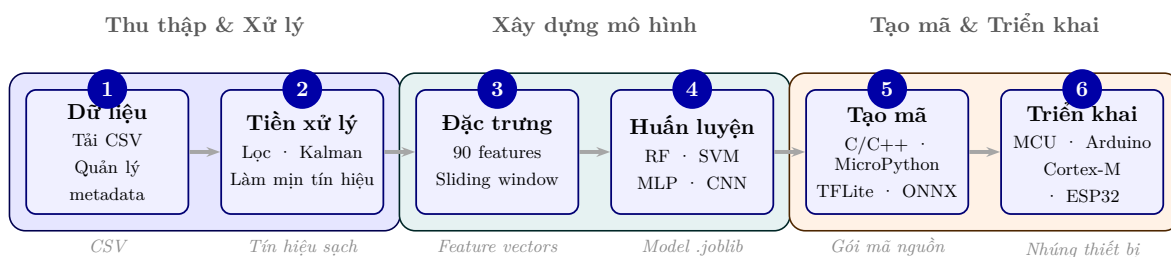
Nghiên cứu và nền tảng công nghiệp trước đây cung cấp nền tảng vững chắc trong theo dõi chuyển động, nhưng không có nền tảng nào cung cấp một quy trình làm việc tích hợp, minh bạch và thân thiện với người dùng được tối ưu hóa cho triển khai biên dựa trên cảm biến IMU, với khả năng phân đoạn dữ liệu tương tác. Công trình này xây dựng dựa trên các thuật toán đã được thiết lập và các thực hành edge ML trong khi giới thiệu các đóng góp mới dân chủ hóa phát triển, tăng tốc lặp lại và cải thiện sẵn sàng triển khai cho các ứng dụng theo dõi chuyển động thực tế.

Chương 3

Phương pháp luận

3.1 Tổng quan kiến trúc Framework

Framework được đề xuất triển khai một quy trình từ đầu đến cuối để tạo các mô hình AI được điều chỉnh cho theo dõi chuyển động trên các thiết bị biên. Kiến trúc hệ thống bao gồm năm thành phần chính: (1) tải lên và quản lý dữ liệu, (2) tiền xử lý tương tác với phân đoạn trực quan, (3) trích xuất đặc trưng, (4) huấn luyện mô hình với tối ưu hóa siêu tham số, và (5) tạo mã đặc thù cho nền tảng. Framework được triển khai bằng Python với giao diện web Dash, scikit-learn và PyTorch cho học máy, cùng các trình tạo mã tùy chỉnh cho triển khai đa nền tảng (C/C++, MicroPython, TFLite Micro, ONNX Runtime). Hình 3.1.1 minh họa quy trình làm việc tổng thể.



Hình 3.1.1: Tổng quan kiến trúc framework: quy trình sáu bước từ tải dữ liệu thô đến triển khai mô hình trên thiết bị biên, hỗ trợ các thuật toán RF, SVM, MLP, CNN và xuất mã đa nền tảng

3.2 Dữ liệu cảm biến quán tính cho nhận dạng hoạt động

Nhận dạng hoạt động con người dựa trên cảm biến đặt ra yêu cầu cơ bản: tín hiệu đầu vào phải mã hóa đủ thông tin để phân biệt các hoạt động có cấu trúc động học khác nhau. Phần này trình bày cơ sở lý thuyết giải thích tại sao cảm biến quán tính (IMU) là lựa chọn phù hợp và các nguyên tắc chi phối cách framework tiếp nhận dữ liệu đó.

3.2.1 Gia tốc kế và con quay hồi chuyển: Tính bổ sung thông tin

Cảm biến gia tốc kế đo gia tốc tuyến tính trên mỗi trục, bao gồm thành phần trọng lực chiếu lên thiết bị. Tín hiệu này phản ánh trực tiếp biên độ và tính tuần hoàn của chuyển động chi: mỗi bước chân tạo ra một xung gia tốc đặc trưng, chạy tạo biên độ lớn hơn đi bộ, trong khi trạng thái đứng yên duy trì độ lớn tín hiệu xấp xỉ $g = 9,81 \text{ m/s}^2$. Tuy nhiên, gia tốc kế đơn thuần gặp phải một hạn chế cơ bản: khi thiết bị nghiêng hoặc xoay, vectơ trọng trường chiếu lên cả ba trục thay đổi theo, khiến tín hiệu bị nhập nhằng giữa chuyển động tịnh tiến thực sự và thay đổi định hướng thiết bị. Hệ quả thực tế là leo cầu thang và xuống cầu thang tạo ra biên dạng gia tốc trục thẳng đứng tương tự nhau—cùng biên độ, cùng tần số bước—nhưng khác nhau rõ rệt ở chuyển động xoay của cổ chân và đầu gối trong mỗi bước.

Cảm biến con quay hồi chuyển đo vận tốc góc ($^\circ/\text{s}$) và không bị ảnh hưởng bởi gia tốc trọng trường. Tín hiệu này nắm bắt chính xác động học xoay của chi: leo cầu thang tạo vận tốc góc dương trong pha gập cổ chân để nâng trọng lượng, trong khi xuống cầu thang tạo vận tốc góc âm cùng trục—sự phân biệt không thể đạt được chỉ từ gia tốc kế. Nhiều cặp hoạt động có biên độ gia tốc tương tự nhau nhưng khác nhau về đặc điểm xoay, như đứng yên so với giậm chân tại chỗ, hay đi bộ trên mặt phẳng so với đi bộ trên dốc.

Tổ hợp sáu trục (ba gia tốc kế cộng ba con quay hồi chuyển) nắm bắt đồng thời cả trạng thái tịnh tiến và trạng thái xoay, cung cấp biểu diễn động học đầy đủ cho bài toán phân loại^[12]. Ưu thế này được xác nhận trên các tập dữ liệu chuẩn: UCI-HAR^[4] và PAMAP2^[28] đạt độ chính xác cao hơn so với cấu hình chỉ dùng ba trục gia tốc kế như trong WISDM^[6]. Framework tiếp nhận dữ liệu với bất kỳ số trục nào—ba trục, sáu trục hoặc nhiều hơn—nhưng đề xuất cấu hình sáu trục như điểm cân bằng giữa khả năng phân biệt hoạt động và chi phí tính toán triển khai. Mở rộng thêm trục (chín

trực với từ kế) bổ sung thông tin định hướng tuyệt đối nhưng phụ thuộc môi trường từ trường và không cải thiện đáng kể khả năng nhận dạng các hoạt động thể chất thông thường.

3.2.2 Tần số lấy mẫu: Định lý Nyquist và nội dung phổ hoạt động

Định lý lấy mẫu Shannon-Nyquist quy định: để tái tạo chính xác tín hiệu liên tục từ chuỗi rời rạc, tần số lấy mẫu f_s phải thỏa mãn $f_s \geq 2f_{\max}$, trong đó $f_{\max}$ là thành phần tần số cao nhất trong tín hiệu. Vận dụng điều này vào bài toán HAR đòi hỏi xác định nội dung phổ thực tế của tín hiệu IMU trong các hoạt động phổ biến. Phân tích phổ cho thấy phần lớn năng lượng tín hiệu tập trung trong dải tần thấp: hoạt động đứng yên và chuyển tiếp chậm dưới 1 Hz; bước đi có tần số cơ bản 1–2 Hz với các sóng hài đến khoảng 8 Hz; chạy có tần số cơ bản 2–3 Hz và các thành phần xung tác động chân (heel-strike transients) đến khoảng 20 Hz^[40]. Tần số lấy mẫu tối thiểu lý thuyết để bảo toàn toàn bộ thông tin phân biệt là khoảng 40 Hz. Các tập dữ liệu chuẩn lấy mẫu ở các mức khác nhau: WISDM ở 20 Hz (có thể aliasing sóng hài của hoạt động chạy), UCI-HAR ở 50 Hz, PAMAP2 ở 100 Hz.

Framework không áp đặt tần số lấy mẫu cố định—người dùng cấu hình theo đặc tính cảm biến và yêu cầu ứng dụng. Tuy nhiên, có một ràng buộc thiết kế không thể bỏ qua: **tần số lấy mẫu phải nhất quán giữa giai đoạn huấn luyện và giai đoạn suy luận triển khai**. Yêu cầu này xuất phát từ nguyên tắc tương đồng huấn luyện-triển khai: toàn bộ các đặc trưng thống kê miền thời gian được tính trên cửa sổ cố định theo *số mẫu*, không phải theo khoảng thời gian tuyệt đối. Nếu tần số lấy mẫu thay đổi giữa hai giai đoạn, cùng một cửa sổ 150 mẫu sẽ tương ứng với các khoảng thời gian khác nhau, làm biến dạng hệ thống toàn bộ phân phối đặc trưng và suy giảm độ chính xác mô hình mà không có bất kỳ cảnh báo nào. Để đảm bảo tính nhất quán này, framework nhúng tần số lấy mẫu đã chọn dưới dạng hằng số biên dịch trong mã triển khai được sinh ra.

Về đánh đổi thực tế: tần số lấy mẫu cao hơn cung cấp thêm chu kỳ tín hiệu trong mỗi cửa sổ, làm phong phú thêm các đặc trưng thống kê—đặc biệt là zero-crossing rate và mean-crossing rate vốn nhạy với mật độ chu kỳ—nhưng đồng thời tăng kích thước bộ đệm vòng cần thiết trên vi điều khiển và kéo dài thời gian trích xuất đặc trưng mỗi chu kỳ suy luận. Sự đánh đổi này thuộc về quyết định thiết kế của người dùng, không phải của framework.

3.3 Quy trình tiền xử lý tương tác

3.3.1 Tải lên và trực quan hóa dữ liệu

Người dùng tải lên các tập dữ liệu CSV chứa các phép đo IMU, tổ chức theo hoạt động (mỗi tệp tương ứng với một lớp). Framework tự động phát hiện tên cột cảm biến, tần số lấy mẫu và hiển thị biểu đồ đồng bộ của tín hiệu thô trên tất cả các trục, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiền xử lý (xem giao diện tại Hình 4.2.1, Chương 4).

3.3.2 Các phương pháp lọc và làm sạch tín hiệu

Dữ liệu IMU thô chứa nhiều cảm biến, nhiễu điện từ và các xung bất thường cần được loại bỏ trước khi trích xuất đặc trưng. Framework cung cấp năm phương pháp lọc, mỗi phương pháp có đặc tính khác nhau về mức độ làm mịn, tính nhân quả và—quan trọng nhất cho triển khai nhúng—*tính tương đồng huấn luyện-triển khai* (training-deployment parity). Người dùng chọn phương pháp phù hợp qua giao diện tương tác; kết quả lọc được trực quan hóa cùng tín hiệu gốc để đánh giá hiệu quả trước khi phân đoạn.

Loại bỏ ngoại lệ (outlier removal)

Xác định và loại bỏ các mẫu có giá trị vượt quá ngưỡng $k\sigma$ so với trung bình cục bộ (mặc định $k = 3$). Đây là bước làm sạch thô, xử lý các xung nhiễu điện từ hoặc bất thường phần cứng (spike artifacts) tạo ra giá trị nằm ngoài phạm vi vật lý hợp lệ của cảm biến. Phương pháp này không thay đổi hình dạng tín hiệu mà chỉ loại bỏ các điểm dữ liệu cô lập rõ ràng sai, nên không có vấn đề parity—nó chỉ áp dụng một lần trên tập dữ liệu huấn luyện (offline) và không cần triển khai trên thiết bị.

Bộ lọc Butterworth low-pass

Bộ lọc IIR Butterworth được thiết kế sao cho đáp ứng biên độ trong dải thông là cực đại phẳng (maximally flat). Hàm truyền liên tục bậc n với tần số cắt f_c :

$$|H(f)|^2 = \frac{1}{1 + (f/f_c)^{2n}}$$

Sau khi rời rạc hóa bằng biến đổi song tuyến, bộ lọc có dạng phương trình sai phân:

$$y[k] = \sum_{i=0}^n b_i x[k-i] - \sum_{j=1}^n a_j y[k-j]$$

trong đó các hệ số (b_i, a_j) được xác định từ f_c và n . Lọc một chiều thuận gây lệch pha; để khắc phục, kỹ thuật lọc hai chiều (forward-backward) áp dụng: lọc thuận thu $y_f[k]$, đảo ngược thứ tự mẫu, lọc thuận lần hai, đảo ngược lại, cho đáp ứng hiệu dụng $|H_{\text{eff}}(f)|^2 = |H(f)|^4$ với pha bằng không. Cấu hình mặc định: bậc 2, tần số cắt 5 Hz.

Parity triển khai: Suy luận HAR đệm đầy cửa sổ W mẫu trước khi bắt đầu xử lý—thiết bị không cần phản hồi theo từng mẫu mà xử lý theo lô. Khi cửa sổ đầy, thiết bị thực hiện chính xác cùng thao tác lọc hai chiều trên bộ đệm: lọc thuận, đảo ngược, lọc thuận lần hai, đảo ngược lại. Các hệ số (b_i, a_j) được tính trước tại thời điểm sinh mã nhúng (sử dụng cùng f_c và n với huấn luyện) và nhúng dưới dạng hằng số trong mã nhúng. Phép tính toán học hoàn toàn đồng nhất giữa hai môi trường.

Bộ lọc Savitzky-Golay

Bộ lọc Savitzky-Golay^[29] khớp đa thức bậc d lên cửa sổ $2m+1$ mẫu bằng bình phương tối thiểu, lấy giá trị đa thức tại điểm trung tâm làm đầu ra. Bài toán tối ưu tại mỗi điểm n :

$$\min_p \sum_{j=-m}^m (p(j) - x[n+j])^2, \quad \deg p = d$$

Nghiệm có dạng tích chập FIR tuyến tính:

$$\hat{x}[n] = \sum_{j=-m}^m c_j x[n+j]$$

trong đó $\mathbf{c} = (A^\top A)^{-1} A^\top \mathbf{e}_0$, với A là ma trận Vandermonde của cửa sổ và $\mathbf{e}_0$ chọn giá trị tại $j=0$. So với trung bình động, Savitzky-Golay bảo tồn tốt hơn các đỉnh và đặc tính hình dạng tín hiệu (peak-preserving). Cấu hình mặc định: $m=2$ (cửa sổ 5 mẫu), $d=2$.

Parity triển khai: Bộ lọc yêu cầu m mẫu ở cả hai phía điểm trung tâm—không thể áp dụng streaming theo từng mẫu. Tuy nhiên, khi cửa sổ suy luận đã đệm đầy, toàn bộ $2m+1$ mẫu lân cận đều có sẵn tại mỗi vị trí nội bộ. Các hệ số c_j được tính trước tại thời điểm sinh mã nhúng và nhúng dưới dạng mảng hằng số; tích chập được áp dụng trên bộ đệm với mở rộng biên bằng cách lặp giá trị tại biên. Phép tính đồng nhất với phía huấn luyện.

Bộ lọc FFT low-pass (miền tần số)

Phương pháp lọc trong miền tần số thực hiện ba bước: biến đổi Fourier rời rạc (DFT), triệt tiêu các bin tần số cao, rồi biến đổi ngược (IDFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1$$

$$\hat{X}[k] = X[k] \cdot \mathbf{1}[k \leq k_c], \quad k_c = \left\lfloor \frac{f_c \cdot N}{f_s} \right\rfloor$$

$$\hat{x}[n] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{X}[k] e^{j2\pi kn/N}$$

Đặc điểm nổi bật: đáp ứng tần số lý tưởng (brick-wall, không rolloff) và không gây méo pha, khác với Butterworth. Phương pháp này về bản chất yêu cầu toàn bộ N mẫu của cửa sổ trước khi lọc (không áp dụng theo từng mẫu streaming được) và có chi phí bộ nhớ $\mathcal{O}(N)$.

Parity triển khai: Khi cửa sổ suy luận đã đệm đầy W mẫu, thiết bị thực thi cùng ba bước DFT $\rightarrow$ masking $\rightarrow$ IDFT trực tiếp trên bộ đệm với cùng k_c . Để đảm bảo độ chính xác số, phép tính sử dụng số dấu phẩy động 32-bit nhất quán, tránh các xấp xỉ điểm cố định có thể gây sai lệch kết quả. Thuật toán hoàn toàn đồng nhất với phía huấn luyện.

Bộ lọc Kalman (constant-velocity)

Bộ lọc Kalman constant-velocity 1D^[20] áp dụng cho mỗi kênh cảm biến IMU được đề xuất như giải pháp lọc có *khoảng cách parity bằng không*. Bộ lọc Kalman hoạt động hoàn toàn theo chiều thuận (forward-only, causal) trong cả hai môi trường: Python huấn luyện và C++ triển khai—loại bỏ lớp lỗi mà tất cả bốn phương pháp trên gặp phải.

Mô hình toán học. Mỗi kênh cảm biến được mô hình hóa bằng trạng thái gồm vị trí (giá trị cảm biến) và vận tốc:

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}, \quad F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Ma trận hiệp phương sai nhiễu quá trình:

$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

3.3. Quy trình tiền xử lý tương tác

Nhiều đo: $R = r$ (scalar). Hai tham số điều chỉnh được: q (process noise, mặc định 10^{-3} —nhỏ hơn = mượt hơn) và r (measurement noise, mặc định 10^{-1} —lớn hơn = mượt hơn). Kalman gain K_k tự động thích ứng theo tín hiệu: tăng khi tín hiệu biến đổi nhanh (bám sát), giảm khi tín hiệu ổn định (lọc mạnh hơn).

So sánh tổng hợp các phương pháp lọc

Bảng 3.3.1 tổng hợp năm phương pháp theo tiêu chí quan trọng nhất cho hệ thống triển khai nhúng:

Bảng 3.3.1: So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai

Phương pháp	Causal?	Parity	Ghi chú triển khai
Loại bỏ ngoại lệ	—	✓	Chỉ offline (không cần trên thiết bị)
Butterworth LP	Không [†]	✓	Per-window two-pass filt-filt trên bộ đệm cửa sổ
Savitzky-Golay	Không [†]	✓	FIR convolution với hệ số tiền tính
FFT low-pass	Không [†]	✓	DFT trực tiếp trên bộ đệm cửa sổ
Kalman filter	Có	✓	Forward-only; hoạt động cả streaming lẫn windowed

[†]Non-causal trong streaming; parity đạt được nhờ HAR đệm toàn bộ cửa sổ trước khi trích xuất đặc trưng, cho phép áp dụng bộ lọc hai chiều trên bộ đệm.

Kết luận thiết kế: bốn phương pháp lọc tín hiệu (Butterworth, Savitzky-Golay, FFT, Kalman) đều đạt parity hoàn toàn chính xác khi suy luận HAR được thực hiện theo lô cửa sổ. Chìa khóa là nhận thức rằng quy trình suy luận đã đệm toàn bộ W mẫu trước khi bắt đầu trích xuất đặc trưng—ràng buộc non-causal chỉ áp dụng cho streaming theo từng mẫu, không áp dụng cho xử lý theo lô trên bộ đệm đầy. Kalman filter vẫn có lợi thế bổ sung: causal theo bản chất nên hoạt động được trong cả chế độ streaming lẫn xử lý theo lô—lý tưởng cho hiển thị tín hiệu thời gian thực trong quá trình thu thập dữ liệu. Riêng loại bỏ ngoại lệ không được nhân bản trên thiết bị vì lý do cơ bản: phương pháp cần thống kê toàn cục từ toàn bộ bản ghi, trong khi mỗi chu kỳ suy luận chỉ có một cửa sổ 1,5 giây đơn lẻ không đủ để ước lượng phân phối tín hiệu.

3.3.3 Phân đoạn và sinh cửa sổ huấn luyện

Máy học trên tín hiệu liên tục đòi hỏi chuyển đổi chuỗi thời gian thành các vector có độ dài cố định phù hợp làm đầu vào mô hình. Framework thực hiện điều này qua hai thao tác bổ sung nhau: người dùng xác định các đoạn tín hiệu sạch đại diện cho từng hoạt động, và hệ thống tự động sinh các cửa sổ huấn luyện từ đó bằng giải thuật cửa sổ trượt.

Giải thuật cửa sổ trượt tự động

Cho đoạn tín hiệu N mẫu, kích thước cửa sổ W mẫu và độ chồng lấp o (%), bước trượt $S = W \cdot (1 - o/100)$ mẫu, số cửa sổ sinh được là:

$$n_w = \left\lfloor \frac{N - W}{S} \right\rfloor + 1 \quad (3.3.1)$$

Ví dụ: một đoạn 5 giây (500 mẫu ở 100 Hz) với $W = 150$, $o = 50\%$ (tức $S = 75$) sinh ra $\lfloor (500 - 150)/75 \rfloor + 1 = 6$ cửa sổ huấn luyện, mỗi cửa sổ mang nhãn hoạt động của đoạn nguồn. Với dữ liệu thực tế (bản ghi 30–60 giây mỗi hoạt động), giải thuật này sinh hàng chục đến hàng trăm cửa sổ từ mỗi phiên thu thập mà không cần chú thích thủ công từng cửa sổ.

Cơ sở lý thuyết cho lựa chọn $o = 50\%$: độ chồng lấp làm tăng gấp đôi số mẫu huấn luyện từ cùng lượng dữ liệu thô, đồng thời giảm hiệu ứng biên—cùng một sự kiện chuyển động (ví dụ: va chạm gót chân trong bước đi) xuất hiện ở vị trí tương đối khác nhau trong các cửa sổ kế tiếp, giúp mô hình học đặc trưng bất biến vị trí trong cửa sổ. Độ chồng lấp cao hơn ($>75\%$) tạo ra nhiều mẫu hơn nhưng với tương quan cao (các cửa sổ liên tiếp gần như giống hệt nhau), làm tăng nguy cơ rò rỉ thông tin giữa tập huấn luyện và kiểm tra nếu chia không cẩn thận.

Lựa chọn đoạn tương tác và sinh dữ liệu

Không phải toàn bộ bản ghi đều có chất lượng đồng đều: giai đoạn bắt đầu và kết thúc thường chứa các chuyển tiếp hoạt động, nhiễu điện, hoặc đoạn nghỉ ngắn không đại diện cho hoạt động mục tiêu. Giao diện lựa chọn đoạn tương tác cho phép người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để đánh dấu các vùng sạch, bỏ qua các đoạn chuyển tiếp hoặc nhiễu. Nhiều đoạn không chồng lấp trên cùng một tệp được hỗ trợ—cho phép trích xuất nhiều chu kỳ hoạt động riêng biệt từ một bản ghi dài mà không cần cắt tệp thủ công. Sau khi xác nhận, cửa sổ trượt chạy tự động trên từng đoạn theo Phương trình 3.3.1, sinh toàn bộ tập huấn luyện trong một thao tác.

Thiết kế này tách bạch rõ hai quyết định: *đâu là dữ liệu đáng tin cậy* (người dùng kiểm soát) và *cách biến dữ liệu đó thành mẫu huấn luyện* (hệ thống tự động hóa hoàn toàn). Nhân hoạt động được kế thừa từ tệp nguồn nhưng có thể gán lại thủ công—ví dụ, trích xuất các đoạn *standing* từ tệp *walking* trong các giai đoạn chuyển tiếp. So với chú thích dấu thời gian thủ công từng khung, cách tiếp cận này giảm đáng kể công sức thu thập trong khi vẫn bảo tồn kiểm soát chất lượng dữ liệu (xem giao diện kéo thả tại Hình 4.3.1, Chương 4).

3.4 Trích xuất đặc trưng

3.4.1 Tham số cửa sổ và ràng buộc nhất quán

Trích xuất đặc trưng thực hiện trên các cửa sổ được sinh theo giải thuật trình bày ở Mục 3.3.3, với cấu hình đã xác thực: kích thước $W = 150$ mẫu (1,5 giây ở $f_s = 100$ Hz) và bước trượt $S = 75$ mẫu (50% chồng lấp). Tại đây, điểm mấu chốt không phải là lựa chọn tham số mà là **ràng buộc nhất quán huấn luyện-triển khai**: (W, S, f_s) phải đồng nhất giữa giai đoạn huấn luyện và suy luận. Thay đổi W sau khi huấn luyện làm thay đổi số mẫu trên mỗi đặc trưng thống kê, dẫn đến phân phối đặc trưng lệch khỏi phân phối đã học—tương tự như thay đổi độ phân giải ảnh giữa huấn luyện và suy luận trong thị giác máy tính. Framework đảm bảo điều này bằng cách nhúng W và f_s dưới dạng hằng số vào mã suy luận được sinh ra.

3.4.2 Các loại trích xuất đặc trưng

Có sáu loại trích xuất đặc trưng (feature extraction type) tiêu biểu, được thiết kế để đáp ứng các yêu cầu khác nhau về số chiều vector (dimensionality), khả năng triển khai nhúng (embedded deployment), và bất biến hướng (orientation invariance). Bảng 3.4.1 so sánh toàn diện sáu loại.

Các loại bất biến hướng (Orientation-Invariant)

Khi thiết bị đeo không cố định (đặt trong túi, đeo ở các vị trí khác nhau trên cơ thể), hướng của các trục cảm biến biến đổi giữa người dùng và lần đo. Hai loại bất biến hướng giải quyết vấn đề này bằng cách tính đặc trưng trên biên độ tổng hợp (vector magnitude)—đại lượng bất biến với phép xoay (rotation-invariant).

Loại `orientation_invariant_time_only` (41 đặc trưng): Sử dụng ba tín hiệu tổng hợp (`acc_mag`, `gyro_mag`, `acc_jerk_mag`) để tính thống kê thời gian (time-

3.4. Trích xuất đặc trưng

Bảng 3.4.1: Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai

Loại (Type)	Tín hiệu nguồn	Đặc trưng tính	#	Nhúng	OI*
orientation_invariant	acc_mag, gyro_mag, acc_jerk_mag	15 thống kê thời gian / tín hiệu	41	✓	✓
orientation_invariant	Như trên + DFT(acc_mag), DFT(gyro_mag)	Như trên + 10 đặc trưng phổ / DFT	63	✓	✓
time_domain	aX, aY, aZ, gX, gY, gZ	15 thống kê thời gian / trục	90	✓	×
all	aX, aY, aZ, gX, gY, gZ	15 thời gian + 11 tần số / trục	156	×	×
frequency_domain	aX, aY, aZ, gX, gY, gZ	11 đặc trưng phổ / trục	66	×	×
raw	aX, aY, aZ, gX, gY, gZ	Giá trị cảm biến thô	6	✓	×

*OI = Orientation-Invariant (bất biến hướng). Tín hiệu tổng hợp: $\text{acc_mag} = \sqrt{a_X^2 + a_Y^2 + a_Z^2}$; $\text{gyro_mag} = \sqrt{g_X^2 + g_Y^2 + g_Z^2}$; $\text{acc_jerk_mag} = \frac{d}{dt} \text{acc_mag}$.

domain statistics). Đây là loại tối thiểu khả dụng (minimum viable), phù hợp cho vi điều khiển hạn chế bộ nhớ và khi hướng gắn cảm biến không thể kiểm soát.

Loại orientation_invariant (63 đặc trưng): Mở rộng bằng 20 đặc trưng phổ DFT (Discrete Fourier Transform) tính trên acc_mag và gyro_mag. Đây là *loại khuyến nghị mặc định*: cân bằng tốt giữa tính phong phú đặc trưng và khả năng triển khai nhúng—bộ sinh mã phát sinh đoạn tính DFT bằng tổng sin/cos trực tiếp, không yêu cầu thư viện FFT bên ngoài.

Loại mỗi trục, miền thời gian (Per-Axis Time-Domain), 90 đặc trưng

Loại time_domain áp dụng 15 đặc trưng thống kê (statistical features) lên từng trục cảm biến riêng lẻ ($6 \times 15 = 90$ đặc trưng). Mỗi đặc trưng là biểu thức dạng đóng xác định (closed-form deterministic expression) chỉ phụ thuộc vào W mẫu của cửa sổ—không đệ quy, không phụ thuộc trạng thái—đảm bảo kết quả tính toán bit-identical trên mọi nền tảng:

- **Xu hướng trung tâm (central tendency):** Trung bình (mean), trung vị (median)

- **Phân tán (dispersion):** Độ lệch chuẩn (std), phương sai (variance), phạm vi (range = max – min), IQR (interquartile range)
- **Cực trị (extrema):** Tối thiểu (min), tối đa (max), Q1 (25th percentile), Q3 (75th percentile)
- **Hình dạng (shape):** Độ lệch (skewness) $\gamma = \frac{E[(X-\mu)^3]}{\sigma^3}$, độ nhọn (kurtosis) $\kappa = \frac{E[(X-\mu)^4]}{\sigma^4} - 3$
- **Năng lượng (energy):** $\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$, năng lượng tín hiệu (signal energy) $E = \sum_{i=1}^N x_i^2$
- **Đặc trưng xấp xỉ tần số (frequency proxy):** Tỷ lệ qua không (zero-crossing rate), tỷ lệ qua trung bình (mean-crossing rate)

Biểu diễn thống kê này bắt giữ biên độ, động lực học, đặc tính phân phối và thông tin giả tần số đủ để phân biệt các hoạt động có dấu hiệu sinh cơ học riêng biệt. Loại này phù hợp khi hướng cảm biến cố định và cần khai thác tối đa thông tin từng trục.

Các loại tần số và thô (Frequency-Domain & Raw)

Loại **all (156 đặc trưng)** và **frequency_domain (66 đặc trưng)** bổ sung đặc trưng phổ (spectral features) mỗi trục (năng lượng phổ, tần số thống trị, entropy phổ). Mặc dù thường được sử dụng trong tài liệu HAR^[6], các loại này **không hỗ trợ triển khai nhúng** vì ba lý do: (i) các giá trị phổ nhạy với sai lệch số hơn nhiều so với tín hiệu đã lọc—sai lệch nhỏ trong biên độ phổ khuếch đại khi tính argmax hoặc tỷ lệ, làm tăng rủi ro không khớp huấn luyện-triển khai^[5]; (ii) tính DFT N -điểm mỗi cửa sổ tăng đáng kể chi phí tính toán và bộ nhớ trên vi điều khiển hạn chế tài nguyên; (iii) các đặc trưng miền thời gian đã chứng minh hiệu quả tương đương trên các tập HAR chuẩn. Hai loại này phù hợp cho nghiên cứu so sánh trên máy tính.

Loại raw (6 đặc trưng): Truyền trực tiếp các giá trị cảm biến thô mà không trích xuất đặc trưng. Phù hợp làm đường cơ sở (baseline) hoặc cung cấp đầu vào cho bộ phân loại học sâu bên ngoài.

3.4.3 Chiến lược đệm cửa sổ (Window Padding Strategy)

Trong quá trình phân đoạn dữ liệu, các cửa sổ thời gian được trích xuất từ tín hiệu thô có thể ngắn hơn kích thước mục tiêu (150 mẫu ở 100Hz = 1,5 giây). Điều này xảy ra khi người dùng chọn các phân đoạn hoạt động ngắn thông qua giao diện kéo thả, hoặc khi cửa sổ trượt cuối cùng trong một đoạn không đủ dài.

Vấn đề với Zero-Padding Phương pháp zero-padding—thường được sử dụng trong xử lý tín hiệu số^[24]—tạo ra các hàng dữ liệu có tất cả giá trị cảm biến bằng 0. Đối với gia tốc kế, điều này có nghĩa $a_x = a_y = a_z = 0$, dẫn đến magnitude gia tốc:

$$\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2} = 0 \quad (3.4.1)$$

Tuy nhiên, trong thực tế, magnitude gia tốc *luôn lớn hơn 0* do gia tốc trọng trường ($\approx 9,81 \text{ m/s}^2$), kể cả khi thiết bị hoàn toàn đứng yên. Do đó, zero-padding tạo ra các giá trị **vật lý bất khả thi** (physically impossible) gây nhiều nghiêm trọng cho phân phối đặc trưng thống kê: $\min = 0$ thay vì $\approx 9,81$, median và phần trăm thứ 25 bị kéo về 0.

Giải pháp: Lặp giá trị biên (Edge-Value Replication) Framework sử dụng chiến lược lặp giá trị cuối cùng của cửa sổ để đệm đến kích thước mục tiêu. Cho cửa sổ $\mathbf{W} = [w_1, w_2, \dots, w_k]$ với $k < N$ (kích thước mục tiêu), cửa sổ đệm trở thành:

$$\mathbf{W}' = [w_1, w_2, \dots, w_k, \underbrace{w_k, w_k, \dots, w_k}_{N-k \text{ lần}}] \quad (3.4.2)$$

Phương pháp này: (1) bảo toàn các đặc tính thống kê tín hiệu gốc, (2) không tạo ra artifact vật lý bất khả thi, và (3) đảm bảo tính nhất quán với dữ liệu suy luận trên thiết bị biên—nơi bộ đệm trượt luôn chứa đầy đủ dữ liệu cảm biến thực. Ngoài ra, các cửa sổ có ít hơn 30% số mẫu mục tiêu được loại bỏ hoàn toàn, vì không chứa đủ thông tin chuyển động để trích xuất đặc trưng có ý nghĩa.

Ý nghĩa thực nghiệm: Trong quá trình phát triển, zero-padding khiến mô hình đã triển khai luôn dự đoán sai lớp *walking_downstairs* bất kể hoạt động thực tế, do các đặc trưng đầu vào nằm ngoài phân phối huấn luyện (xem Mục 6.4 để phân tích chi tiết).

3.4.4 Tăng cường dữ liệu cho huấn luyện (Data Augmentation)

Để cải thiện tính tổng quát hóa của mô hình và giảm hiện tượng quá khớp (overfitting), đặc biệt với các tập dữ liệu nhỏ điển hình của thu thập dữ liệu đối tượng đơn, các kỹ thuật tăng cường dữ liệu (data augmentation) được thiết kế riêng cho dữ liệu IMU có thể được áp dụng^[19;35]. Tăng cường được thực hiện ở cấp cửa sổ—trước khi trích xuất đặc trưng—để tạo ra các biến thể thực tế của tín hiệu gốc mà không làm thay đổi bản chất hoạt động.

Các phương pháp tăng cường

Năm phương pháp biến đổi được triển khai, mỗi phương pháp mô phỏng một nguồn biến thiên thực tế khác nhau trong dữ liệu cảm biến quán tính:

1. **Nhiều Jitter**: Thêm nhiễu Gaussian $\mathcal{N}(0, \sigma^2)$ vào mỗi mẫu, mô phỏng nhiễu cảm biến tự nhiên và biến thiên môi trường. Tham số σ (mặc định: 0,05) kiểm soát cường độ nhiễu^[35].
2. **Tỷ lệ (Scaling)**: Nhân toàn bộ cửa sổ với hệ số tỷ lệ ngẫu nhiên $s \sim \mathcal{N}(1, \sigma_s^2)$, mô phỏng sự khác biệt về cường độ chuyển động giữa các lần thực hiện. Ví dụ, $s = 1,15$ tương ứng với bước đi mạnh hơn 15%.
3. **Xoay (Rotation)**: Áp dụng ma trận xoay ngẫu nhiên lên các trục gia tốc kế và con quay hồi chuyển, mô phỏng sự thay đổi vị trí đặt cảm biến trên cơ thể—một nguồn biến thiên phổ biến và quan trọng trong ứng dụng HAR thực tế^[34]. Góc xoay tối đa có thể cấu hình (mặc định: 15°).
4. **Biến dạng thời gian (Time Warping)**: Áp dụng biến đổi phi tuyến lên trục thời gian qua nội suy cubic spline, mô phỏng sự biến thiên tốc độ tự nhiên trong thực hiện hoạt động—ví dụ, bước đi bắt đầu chậm và tăng tốc dần^[21].
5. **Hoán vị (Permutation)**: Chia cửa sổ thành k đoạn (mặc định: $k = 4$) và hoán vị thứ tự ngẫu nhiên, giúp mô hình học các đặc trưng thống kê cục bộ thay vì phụ thuộc vào thứ tự thời gian toàn cục.

Tăng cường nhận biết lớp (Class-Aware Augmentation)

Một đóng góp quan trọng của framework là chiến lược tăng cường *nhận biết lớp*, giải quyết một vấn đề tinh tế nhưng nghiêm trọng: việc áp dụng các phép biến đổi mạnh (jitter, rotation, scaling) lên các hoạt động tĩnh (static activities) như *still*, *standing* hay *sitting* tạo ra tín hiệu nhân tạo có các dao động nhỏ giống với hoạt động cường độ thấp (ví dụ: *walking_downstairs* với rung nhẹ). Điều này gây **nhầm lẫn lớp** (class confusion), làm suy giảm đáng kể hiệu suất phát hiện hoạt động tĩnh.

Thiết kế giải pháp: Phân loại tĩnh/động được xác định theo ngưỡng biên độ dao động của tín hiệu gia tốc kế trong cửa sổ và có thể được người dùng chỉ định thủ công, sau đó áp dụng chiến lược tăng cường phân biệt:

- **Hoạt động động (dynamic)**: Áp dụng đầy đủ cả năm phương pháp với tham số tiêu chuẩn ($\sigma_{\text{jitter}} = 0,05$)

- **Hoạt động tĩnh** (static): Chỉ áp dụng nhiều vi mô (micro-jitter) với $\sigma = 0,01$ —đủ để tạo tính đa dạng cảm biến thực tế mà không phá vỡ đặc tính “gần bất động” (near-stationary) vốn xác định hoạt động tĩnh

Người dùng cũng có thể xác định thủ công danh sách nhãn tĩnh qua giao diện framework, phù hợp với các ứng dụng có nhãn không thuộc bộ từ khóa mặc định.

Cơ sở lý thuyết: Chiến lược này tuân thủ nguyên tắc rằng tăng cường phải bảo toàn các đặc tính phân biệt (discriminative properties) của mỗi lớp [8]. Với hoạt động tĩnh, đặc tính phân biệt chính là biên độ dao động cực thấp ($\sigma_{acc} < 0,1 \text{ m/s}^2$); các phép biến đổi mạnh phá hủy chính xác đặc tính này, trong khi micro-jitter bảo toàn nó.

3.4.5 Ngưỡng tin cậy cho từ chối hoạt động không xác định

Trong triển khai thực tế, thiết bị biên thường ghi nhận các chuyển động không thuộc bất kỳ lớp nào đã huấn luyện (ví dụ: ngồi xuống, cúi người, hoặc đeo thiết bị ở vị trí bất thường). Các hệ thống phân loại truyền thống bắt buộc phải gán một nhãn cho mọi đầu vào, bất kể mức độ tin cậy, dẫn đến các dự đoán sai với độ tin cậy cao—hiện tượng được gọi là *overconfident misclassification* [17;18].

Cơ chế từ chối: Ngưỡng tin cậy (confidence threshold) có thể được tích hợp trực tiếp vào mã suy luận sinh ra cho thiết bị nhúng. Mỗi mô hình tính toán xác suất từng lớp, và nếu xác suất tối đa $\max_c P(y = c|\mathbf{x})$ thấp hơn ngưỡng τ (mặc định: $\tau = 0,6$), dự đoán trả về “unknown” thay vì ép buộc một nhãn cụ thể:

$$\hat{y} = \begin{cases} \arg \max_c P(y = c|\mathbf{x}) & \text{nếu } \max_c P(y = c|\mathbf{x}) \geq \tau \\ \text{unknown} & \text{nếu } \max_c P(y = c|\mathbf{x}) < \tau \end{cases} \quad (3.4.3)$$

Tính toán tin cậy theo thuật toán: Mỗi thuật toán sử dụng cơ chế tính xác suất riêng có sẵn:

- **Neural Network/CNN:** Hàm softmax trên logits đầu ra: $P(c) = \frac{e^{z_c}}{\sum_j e^{z_j}}$
- **Random Forest:** Tỷ lệ bỏ phiếu: $P(c) = \frac{\text{số cây bỏ phiếu cho } c}{\text{tổng số cây}}$
- **SVM:** Softmax trên điểm quyết định One-vs-Rest (OvR)

Giá trị $\tau = 0,6$ được chọn để cân bằng giữa tỷ lệ từ chối và tỷ lệ dương tính giả: với bài toán 5 lớp, xác suất ngẫu nhiên là 20%, do đó $\tau = 0,6$ yêu cầu mô hình tin cậy gấp 3 lần ngẫu nhiên.

So sánh với nền tảng thương mại: Edge Impulse cung cấp chức năng “anomaly detection” yêu cầu huấn luyện một khối riêng biệt trên dữ liệu bổ sung. Cách tiếp cận được đề xuất trích xuất tin cậy trực tiếp từ đầu ra mô hình hiện có—không tốn thêm bộ nhớ, không yêu cầu dữ liệu huấn luyện bổ sung, và tích hợp tự nhiên vào quy trình suy luận đã có.

3.5 Huấn luyện mô hình học máy

3.5.1 Các thuật toán học máy áp dụng

Bốn nhóm thuật toán học máy dưới đây, bao gồm cả học máy cổ điển và học sâu, đã được chứng minh hiệu quả trong nhận dạng hoạt động và phù hợp cho triển khai biên. Mỗi nhóm thể hiện đặc tính đánh đổi khác nhau giữa độ chính xác, yêu cầu dữ liệu và chi phí tài nguyên—các yếu tố cần cân nhắc khi lựa chọn cho ứng dụng cụ thể:

Random Forest (RF) Phương pháp ensemble kết hợp nhiều cây quyết định với bagging và ngẫu nhiên hóa đặc trưng^[7]. Cấu hình điển hình cho bài toán HAR:

- Số cây: 50–200 (tham chiếu: 100 cây), độ sâu tối đa 10–30
- Cân bằng lớp: `class_weight='balanced'` tự động trọng số mẫu tỷ lệ nghịch với tần suất lớp
- Tiêu chí phân nhánh: Độ bất thuần Gini hoặc entropy thông tin

RF đặc biệt phù hợp cho HAR vì không yêu cầu chuẩn hóa đặc trưng; chịu đựng tốt với đặc trưng không liên quan (mỗi lần phân nhánh chỉ xét $\sqrt{D}$ đặc trưng ngẫu nhiên); kết quả giải thích được qua độ quan trọng đặc trưng; ổn định với nhiễu nhờ cơ chế bỏ phiếu đa số.

Support Vector Machine (SVM) Bộ phân loại phân biệt tìm siêu phẳng tối đa hóa biên phân tách giữa các lớp. Cấu hình điển hình:

- Kernel: Radial Basis Function (RBF), phù hợp cho dữ liệu phi tuyến; tham số gamma kiểm soát bán kính ảnh hưởng của từng điểm huấn luyện
- Chiến lược đa lớp: One-vs-One (OvO) hoặc One-vs-Rest (OvR)
- Cân bằng lớp: `class_weight='balanced'`

- Chính quy hóa: Tham số C kiểm soát đánh đổi biên rộng—vi phạm biên ($C \in \{0,1; 1; 10\}$)

SVM phù hợp khi số chiều đặc trưng cao so với số mẫu—điển hình trong HAR với hàng chục đến hàng trăm đặc trưng và vài trăm mẫu. Yêu cầu chuẩn hóa đặc trưng (StandardScaler) trước huấn luyện để đảm bảo các chiều đóng góp đồng đều vào hàm kernel.

Neural Network (NN) Perceptron đa lớp (MLP) với lan truyền ngược, phù hợp học các ánh xạ phi tuyến phức tạp. Một kiến trúc tham chiếu cho bài toán 5 lớp với 63 đặc trưng đầu vào:

- Kiến trúc: [63 đầu vào] $\rightarrow$ [100 ẩn, ReLU] $\rightarrow$ [num_classes, Softmax]
- Trình tối ưu: Adam ($\eta = 0,001$, $\beta_1 = 0,9$, $\beta_2 = 0,999$)
- Chính quy hóa: Phạt L2 $\alpha = 0,0001$; dừng sớm (patience=10)
- Lịch trình: Tối đa 500 epoch, batch đầy đủ

Kiến trúc một lớp ẩn vừa đủ năng lực học mẫu phi tuyến vừa nhỏ gọn cho triển khai nhúng (ma trận trọng số cố định, không phân bổ bộ nhớ động). Kiến trúc tùy chỉnh với nhiều lớp ẩn hơn có thể xây dựng qua PyTorch, với cơ chế xuất trọng số sang định dạng tương thích để tạo mã triển khai.

Convolutional Neural Network (CNN) Mạng tích chập 1D xử lý trực tiếp cửa sổ cảm biến thô (end-to-end), học biểu diễn phân cấp từ tín hiệu mà không cần trích xuất đặc trưng thủ công. Một kiến trúc tham chiếu 1D-CNN:

- Đầu vào: Cửa sổ thô $W \times C$ ($W = 150$ mẫu, $C = 6$ kênh cảm biến)
- Khối tích chập: 2–3 lớp Conv1D (kernel 3–5, bộ lọc 32–64) + BatchNorm + MaxPool
- Phân loại: Flatten + Dense + Dropout (0,3–0,5) + Softmax
- Trình tối ưu: Adam với OneCycleLR scheduler
- Triển khai: Xuất trọng số thành mảng C/C++ hoặc chuyển đổi sang TFLite Micro (INT8)

CNN phù hợp khi tập dữ liệu đủ lớn và các đặc trưng thống kê thủ công chưa đủ phân biệt—mô hình học trực tiếp các biểu diễn phân cấp từ tín hiệu thô. Đổi lại, CNN yêu cầu nhiều dữ liệu huấn luyện hơn và bộ nhớ triển khai lớn hơn (500 KB–2 MB so với 50–200 KB cho mô hình cổ điển).

Convolutional Neural Network 2D (2D-CNN) Biểu thể hai chiều xử lý của số cảm biến như ảnh $W \times C$ (thời gian $\times$ kênh), cho phép học tương quan chéo kênh (cross-channel correlation) giữa các trục cảm biến thông qua kernel hai chiều. Một kiến trúc tham chiếu 2D-CNN:

- Đầu vào: Ma trận $W \times C$ ($W = 150$ bước thời gian, $C = 6$ kênh cảm biến)
- Kernel tích chập: Conv2D($3 \times C$) học đồng thời mẫu thời gian và tương quan giữa các trục cảm biến
- Khối tiếp theo: BatchNorm + MaxPool(thời gian) + Flatten + Dense + Dropout + Softmax
- Trình tối ưu: Adam với OneCycleLR scheduler
- Triển khai: Cùng đường dẫn xuất mã với 1D-CNN

2D-CNN phù hợp khi tương quan không gian giữa các trục cảm biến mang thông tin phân biệt—ví dụ, leo cầu thang tạo mẫu tương quan đặc trưng giữa gia tốc thẳng đứng và con quay hồi chuyển. So với 1D-CNN, 2D-CNN học các đặc trưng liên kênh một cách tường minh, nhưng đòi hỏi nhiều dữ liệu hơn để hội tụ do không gian tham số lớn hơn.

3.5.2 Giao thức huấn luyện và đánh giá

Chiến lược chia dữ liệu

Để ngăn rò rỉ thông tin và đảm bảo ước lượng hiệu suất không thiên vị, giao thức phân chia ba chiều là phương pháp được khuyến nghị trong học máy^[13]:

1. **Tập huấn luyện (60%)**: Dùng để khớp các tham số mô hình
2. **Tập xác thực (20%)**: Dùng để điều chỉnh siêu tham số, lựa chọn mô hình và giám sát tiến trình huấn luyện
3. **Tập kiểm tra (20%)**: Được giữ lại hoàn toàn; chỉ đánh giá một lần duy nhất sau khi huấn luyện hoàn tất

Tất cả các phân chia nên sử dụng lấy mẫu phân tầng để bảo tồn tỷ lệ lớp, đặc biệt quan trọng với tập HAR có phân phối không đồng đều.

Phân biệt Tập Xác thực so với Kiểm tra: Tập xác thực cho phép phát triển mô hình lặp lại—người dùng có thể điều chỉnh siêu tham số, thử các thuật toán khác nhau hoặc điều chỉnh các bước tiền xử lý trong khi giám sát độ chính xác xác thực mà không làm nhiễu tập kiểm tra. Tập kiểm tra được đánh giá *chính xác một lần* cho mỗi mô hình đã huấn luyện để báo cáo hiệu suất cuối cùng, cung cấp một ước lượng không thiên vị về khái quát hóa cho dữ liệu chưa thấy. Phương pháp luận này tuân theo các thực hành tốt nhất trong nghiên cứu học máy^[13] và loại bỏ rò rỉ thông tin tinh tế xảy ra khi đánh giá lặp lại trên một tập giữ lại duy nhất.

Xác thực Chéo: Đối với phân tích khám phá khi dữ liệu hạn chế, xác thực chéo 5-fold có thể thực hiện trên tập huấn luyện+xác thực kết hợp (80% dữ liệu), trong khi vẫn bảo tồn tập kiểm tra (20%) là hoàn toàn chưa thấy. Phương pháp này cung cấp ước lượng hiệu suất mạnh mẽ cho các nghiên cứu so sánh thuật toán mà không yêu cầu phân chia xác thực riêng biệt.

Xử lý mất cân bằng lớp

Các tập dữ liệu HAR thường thể hiện mất cân bằng lớp (ví dụ: nhiều mẫu *standing* hơn *walking_downstairs*), khiến mô hình huấn luyện trực tiếp thiên vị về các lớp đa số và suy giảm khả năng nhận dạng lớp thiểu số. Hai kỹ thuật chính được áp dụng để xử lý vấn đề này:

- **Cân bằng Trọng số Lớp:** Đối với RF và SVM, tham số `class_weight='balanced'` gán trọng số $w_c = \frac{n_samples}{n_classes \times n_samples_c}$ trong đó $n_samples_c$ là số lượng của lớp c . Điều này tăng đóng góp mất mát của các phân loại sai lớp thiểu số trong quá trình huấn luyện.
- **Phân chia Phân tầng:** Các phân chia huấn luyện-kiểm tra bảo tồn tỷ lệ lớp sử dụng lấy mẫu phân tầng.

Cân bằng lớp là quan trọng cho triển khai thực tế nơi tất cả các lớp hoạt động phải được phát hiện một cách đáng tin cậy, không chỉ những lớp thường xuyên nhất.

3.5.3 Tối ưu hóa siêu tham số

Điều chỉnh siêu tham số có hệ thống là bước quan trọng để đạt hiệu suất tốt nhất. GridSearchCV là một phương pháp phổ biến, duyệt toàn bộ lưới tổ hợp tham số và

đánh giá từng tổ hợp qua xác thực chéo 5-fold. Các không gian tìm kiếm điển hình cho từng thuật toán:

Random Forest:

- `n_estimators`: [50, 100, 200]
- `max_depth`: [10, 20, 30, None]
- `min_samples_split`: [2, 5, 10]
- `class_weight`: ['balanced', 'balanced_subsample']

SVM:

- `C`: [0.1, 1, 10, 100]
- `gamma`: ['scale', 'auto', 0.001, 0.01, 0.1]
- `class_weight`: ['balanced']

Neural Network:

- `hidden_layer_sizes`: [(50,), (100,), (100, 50)]
- `alpha`: [0.0001, 0.001, 0.01]
- `learning_rate_init`: [0.001, 0.01]

Tối ưu hóa xác định các cấu hình tối đa hóa độ chính xác xác thực chéo, thường cải thiện hiệu suất cơ sở 2–5% (xem giao diện cấu hình huấn luyện tại Hình 4.5.1, Chương 4).

3.6 Tạo mã cho triển khai biên

3.6.1 Kiến trúc

Hệ thống con tạo mã dịch các mô hình đã huấn luyện thành các gói triển khai phù hợp với nhiều họ thiết bị và môi trường thực thi khác nhau, thay vì nhắm đến một bo mạch đơn lẻ. Kiến trúc tuân theo mẫu thiết kế Factory, trong đó generator được chọn theo ba trục: *loại mô hình*, *họ nền tảng đích* và *cách tiếp cận triển khai*. Danh sách khóa nền tảng và backend cụ thể được tóm tắt trong Bảng 3.6.1.

Kiến trúc gồm ba lớp chính:

3.6. Tạo mã cho triển khai biên

- **Lớp cơ sở (Base Generator):** cung cấp phần dùng chung gồm sinh tệp, trích xuất đặc trưng, tuần tự hóa tham số, kiểm tra tương đồng và ví dụ tích hợp.
- **Lớp đặc thù thuật toán:** mỗi thuật toán (RF, SVM, NN, CNN) có generator riêng hiện thực logic suy luận tương ứng.
- **Lớp đặc thù nền tảng:** các generator cho ARM Cortex-M, MicroPython, Zephyr, cùng các backend TFLite Micro và ONNX Runtime, chịu trách nhiệm điều chỉnh mã theo môi trường đích.

Bảng 3.6.1: Ma trận đích triển khai được hỗ trợ

Nhóm đích	Khóa nền tảng/backend	Gói sinh ra	Mục đích sử dụng chính
Arduino-compatible	arduino, seeed_xiao, esp32, m5stack, teensy	.h + .cpp + .ino	Quy trình Arduino IDE/PlatformIO cho các bo vi điều khiển phổ biến
ARM Cortex-M thuần	arm_cortex_m	.c	Tích hợp bare-metal hoặc CMSIS trên các vi điều khiển ARM
SDK/RTOS native	generic_c, generic_cpp, esp_idf, zephyr	.h + .c/.cpp + ví dụ	Ứng dụng nhúng native, SDK hãng và RTOS
MicroPython	micropython	module .py + ví dụ .py	Nguyên mẫu nhanh hoặc giáo dục trên board hỗ trợ MicroPython
Backend runtime thay thế	tflite_micro, onnx_runtime	mã glue + model nhị phân	Mở rộng cho các pipeline cần runtime engine chuẩn hóa

Chi tiết hiện thực và giao diện tạo mã được trình bày trong Chương 4 (Hình 4.6.1).

3.6.2 Lý do chọn tạo mã trực tiếp thay vì Runtime Engine

Việc lựa chọn phương pháp triển khai mô hình ML lên vi điều khiển có ảnh hưởng trực tiếp đến mức sử dụng tài nguyên, tính minh bạch và phạm vi thuật toán được hỗ trợ. Phần này phân tích ba cách tiếp cận chính và luận giải cho quyết định thiết kế trong bối cảnh HAR biên.

Các phương pháp triển khai mô hình trên vi điều khiển

Có ba cách tiếp cận chính để triển khai mô hình ML trên vi điều khiển, mỗi cách thể hiện đánh đổi khác nhau giữa tính tiện lợi, hiệu suất và kiểm soát:

Phương pháp 1: Runtime Engine (TensorFlow Lite Micro, ONNX Micro Runtime) Mô hình được chuyển đổi sang định dạng trung gian (`.tflite`, `.onnx`), sau đó được tải bởi một runtime engine chung trên thiết bị. Runtime đọc biểu đồ tính toán và thực thi từng toán tử (operator) theo thứ tự.

Ưu điểm: Hỗ trợ nhiều kiến trúc mô hình (CNN, LSTM, Transformer); hệ sinh thái lớn với cộng đồng hỗ trợ mạnh; các tối ưu hóa lượng tử hóa tích hợp (INT8, FP16).

Nhược điểm: Runtime chiếm 100–300KB flash bổ sung^[39]—đáng kể trên vi điều khiển có 256KB–1MB flash. Interpreter overhead thêm 15–40% thời gian suy luận so với mã gốc^[22]. Quan trọng nhất, runtime hoạt động như hộp đen: người dùng không thể kiểm tra hoặc xác minh quá trình trích xuất đặc trưng và tính toán bên trong.

Phương pháp 2: Trình biên dịch mô hình (Edge Impulse EON Compiler, Apache TVM, microTVM) Mô hình được biên dịch trước (ahead-of-time compilation) sang mã C tối ưu hóa, loại bỏ runtime nhưng vẫn hoạt động qua chuỗi công cụ đóng gói của nhà cung cấp.

Ưu điểm: Không có interpreter overhead; mã tối ưu hóa cho vi kiến trúc cụ thể; Edge Impulse EON Compiler giảm flash 35–55% so với TFLite Micro (theo số liệu nhà cung cấp^[11]).

Nhược điểm: Chuỗi công cụ đóng gói—mã đã tạo bởi EON Compiler khó kiểm toán và không thể sửa đổi. Apache TVM/microTVM yêu cầu kiến thức biên dịch cấp chuyên gia. Hầu hết các trình biên dịch mô hình bắt buộc mô hình đầu vào theo định dạng ONNX hoặc TFLite, loại trừ các mô hình scikit-learn cổ điển (RF, SVM) mà không qua bước chuyển đổi trung gian phức tạp.

Phương pháp 3: Tạo mã trực tiếp (Direct Code Generation) Phương pháp này trích xuất trực tiếp các tham số đã huấn luyện từ đối tượng scikit-learn/PyTorch—trọng số và bias của mạng nơ-ron, support vectors và hệ số kernel của SVM, cấu trúc cây nhị phân của Random Forest, trọng số convolution và batch normalization của CNN—rồi tạo mã nguồn (C/C++ hoặc MicroPython) triển khai thuật toán suy luận tương ứng.

Ưu điểm: Không phụ thuộc bên ngoài—mã tự chứa hoàn toàn; dấu chân bộ nhớ tối thiểu (chỉ chứa trọng số + logic suy luận); minh bạch hoàn toàn—mọi phép tính

có thể kiểm tra và xác minh; hỗ trợ trực tiếp các thuật toán ML cổ điển mà không cần chuyển đổi định dạng; xuất được nhiều ngôn ngữ đích (C, C++, MicroPython).

Nhược điểm: Yêu cầu triển khai thủ công logic suy luận cho mỗi thuật toán; không hưởng các tối ưu hóa toán tử tự động cấp vi kiến trúc; khó mở rộng sang các kiến trúc phức tạp (CNN nhiều lớp, attention).

Phân tích so sánh định lượng

Bảng 3.6.2 tóm tắt so sánh ba phương pháp trên các tiêu chí quan trọng cho triển khai vi điều khiển bị giới hạn tài nguyên:

Bảng 3.6.2: So sánh phương pháp triển khai mô hình ML trên vi điều khiển

Tiêu chí	Runtime	Compiler	Trực tiếp
Overhead flash	100–300KB	15–50KB	0KB
Overhead suy luận	15–40%	<5%	0%
Hỗ trợ RF/SVM	Hạn chế [†]	Không [‡]	Đầy đủ
Minh bạch	Thấp	Trung bình	Hoàn toàn
Cần ONNX/TFLite	Có	Có	Không
Phức tạp triển khai	Thấp	Cao	Trung bình
Hỗ trợ DNN sâu	Tốt	Tốt	Hạn chế

[†]TFLite Micro chỉ hỗ trợ NN/CNN; RF/SVM không thể chuyển đổi trực tiếp do onnx2tf không hỗ trợ ONNX-ML TreeEnsembleClassifier. Keras surrogate có thể sử dụng nhưng là xấp xỉ (80–90% thỏa thuận).

[‡]EON Compiler và TVM tập trung vào mạng nơ-ron; không hỗ trợ RF/SVM gốc.

Lập luận cho lựa chọn thiết kế

Quyết định sử dụng tạo mã trực tiếp được lập luận bởi bốn yếu tố:

(1) Dấu chân bộ nhớ tối thiểu Trên các vi điều khiển lớp wearable như Seed XIAO nRF52840 (1MB flash, 256KB RAM), mỗi KB đều quan trọng. TFLite Micro interpreter chiếm khoảng 150–250KB flash^[27;39]—tương đương 15–25% tổng flash khả dụng—chỉ riêng runtime, trước khi tính trọng số mô hình. Phương pháp trực tiếp loại bỏ hoàn toàn overhead này: một mô hình Neural Network chỉ chiếm 95KB (trọng số + logic suy luận + trích xuất đặc trưng), để lại >900KB cho firmware ứng dụng và giúp việc chuyển sang các board cùng lớp tài nguyên trở nên khả thi hơn.

(2) Minh bạch và khả năng xác minh Một yêu cầu quan trọng trong thiết kế hệ thống suy luận nhúng là *khả năng kiểm tra và xác minh mã đã tạo* (xem Phần 6.4). Khi

phát hiện lỗi zero-padding và sai lệch công thức kurtosis, có thể truy vết trực tiếp đến mã C++ đã tạo, so sánh từng dòng với triển khai Python, và sửa chữa. Với runtime đóng gói (TFLite Micro) hoặc trình biên dịch đóng (EON Compiler), quá trình gỡ lỗi này sẽ không thể thực hiện—lỗi sẽ biểu hiện như “mô hình hoạt động kém trên thiết bị” mà không có cách xác định nguyên nhân gốc.

Khả năng xác minh này đặc biệt quan trọng cho ứng dụng y tế và an toàn, nơi các tiêu chuẩn quy định (FDA, IEC 62304) yêu cầu khả năng truy vết và kiểm toán đầy đủ cho mọi thành phần phần mềm liên quan đến quyết định lâm sàng.

(3) Hỗ trợ trực tiếp các thuật toán ML cổ điển Hệ sinh thái TFLite Micro và ONNX Runtime Micro được thiết kế chủ yếu cho mạng nơ-ron. Triển khai Random Forest hoặc SVM qua các nền tảng này yêu cầu chuyển đổi nhiều bước: `scikit-learn` → `ONNX (skl2onnx)` → `TFLite (onnx-tf)` → `TFLite Micro`. Mỗi bước chuyển đổi giới thiệu rủi ro sai lệch số và mất thông tin (ví dụ: ONNX không hỗ trợ chính xác tham số `class_weight` của `scikit-learn`). Phương pháp trực tiếp đọc các thuộc tính nội bộ của `scikit-learn` (`model.coefs_`, `model.support_vectors_`, `model.estimators_`) mà không qua định dạng trung gian, loại bỏ hoàn toàn chuỗi chuyển đổi dễ lỗi này.

(4) Các thuật toán cổ điển là lựa chọn phù hợp cho HAR biên Lựa chọn tạo mã trực tiếp liên kết chặt chẽ với lựa chọn thuật toán. Cho bài toán HAR 5–10 lớp với <1000 vectors đặc trưng và 63 đặc trưng bất biến hướng, các thuật toán cổ điển đạt hiệu suất cạnh tranh (97–99%) với chi phí tài nguyên thấp hơn đáng kể. Các mô hình này có cấu trúc suy luận đơn giản—nhân ma trận cho NN, duyệt cây cho RF, tính kernel cho SVM—có thể triển khai chính xác trong vài trăm dòng C++ mà không cần framework tính toán phức tạp. Điều này tạo thành một *sweet spot* giữa hiệu suất mô hình và tính khả thi triển khai cho các thiết bị đeo bị giới hạn tài nguyên.

Hạn chế và các phương pháp bổ sung

Tạo mã trực tiếp không phải giải pháp phổ quát. Nó không phù hợp cho:

- **Mạng sâu nhiều lớp:** LSTM, Transformer, hoặc CNN với hơn 4–5 lớp convolution yêu cầu các toán tử phức tạp (attention, bidirectional) mà triển khai thủ công dễ lỗi và khó bảo trì
- **Lượng tử hóa nâng cao:** INT8 inference với hiệu chỉnh phạm vi per-channel yêu cầu hạ tầng runtime mà TFLite Micro cung cấp sẵn

- **Mô hình thay đổi thường xuyên:** Nếu mô hình cần cập nhật OTA (over-the-air) mà không biên dịch lại firmware, runtime interpreter là lựa chọn tốt hơn

Đường dẫn TFLite Micro (Mục 3.6.3) tồn tại song song với tạo mã trực tiếp; người dùng chọn phương pháp phù hợp: tạo mã trực tiếp cho ML cổ điển và CNN nhỏ, TFLite Micro cho các mô hình cần lượng tử hóa INT8 đầy đủ.

3.6.3 Triển khai TFLite Micro

Đường dẫn triển khai TFLite Micro là phương pháp bổ sung phù hợp cho các kiến trúc mô hình neural network. Quá trình chuyển đổi tuân theo pipeline ONNX → TensorFlow → TFLite với các hạn chế quan trọng về phạm vi hỗ trợ^[1].

Phạm vi hỗ trợ theo loại mô hình

Nghiên cứu thực nghiệm cho thấy khả năng chuyển đổi khác nhau đáng kể giữa các loại mô hình:

Neural Network và CNN: Chuyển đổi trực tiếp (hỗ trợ) Các mô hình neural network (MLP) và CNN từ PyTorch có thể chuyển đổi thành công qua pipeline:

1. PyTorch model → ONNX (`torch.onnx.export`)
2. ONNX → TensorFlow (`onnx-tf`)
3. TensorFlow → TFLite (`tf.lite.TFLiteConverter`)
4. TFLite → TFLite Micro (quantization INT8)

Random Forest và SVM: Hạn chế ONNX-ML (không hỗ trợ trực tiếp) Các mô hình Random Forest và SVM gặp rào cản chuyển đổi do hạn chế trong hệ sinh thái ONNX-ML:

- **Scikit-learn → ONNX:** `skl2onnx` tạo các toán tử ONNX-ML như `TreeEnsembleClassifier` và `SVMClassifier`
- **ONNX-ML → TensorFlow:** `onnx2tf` không hỗ trợ các toán tử ONNX-ML, chỉ hỗ trợ các toán tử ONNX chuẩn (operator domain "ai.onnx")
- **Kết quả:** Pipeline bị gián đoạn, không thể chuyển đổi RF/SVM sang TFLite

Giải pháp Keras Surrogate cho RF/SVM

Để khắc phục hạn chế ONNX-ML, *phương pháp Keras surrogate* có thể được áp dụng:

1. **Sinh dữ liệu huấn luyện surrogate:** Sử dụng mô hình RF/SVM gốc để tạo soft predictions trên tập dữ liệu đại diện
2. **Huấn luyện Keras MLP:** Mạng neural đa lớp học ánh xạ từ features → soft predictions của RF/SVM
3. **Chuyển đổi MLP → TFLite:** Keras MLP có thể chuyển đổi thành công qua pipeline tiêu chuẩn

Hạn chế Surrogate: Keras surrogate là *xấp xỉ*, không phải triển khai chính xác. Độ thỏa thuận kỳ vọng 80-90% giữa mô hình gốc và surrogate. Cho độ chính xác tối đa, *tạo mã trực tiếp C++ vẫn là phương pháp được khuyến nghị cho RF/SVM*.

Khuyến nghị sử dụng

- **NN/CNN:** Sử dụng TFLite Micro để tận dụng runtime tối ưu hóa và hỗ trợ quantization INT8
- **RF/SVM:** Ưu tiên tạo mã trực tiếp C++ (chính xác, nhanh, nhỏ gọn). TFLite surrogate chỉ khi yêu cầu tích hợp runtime chuẩn hóa

3.6.4 Cấu trúc mã đã tạo

Đối với mỗi mô hình đã huấn luyện, trình tạo sản xuất một gói triển khai gồm các tệp tham số, tệp cài đặt và tệp ví dụ tích hợp. Cụ thể:

1. **Tệp Header (.h):** Chứa các tham số mô hình (ví dụ: vectors hỗ trợ, cấu trúc cây, ma trận trọng số), khai báo hàm, các macro số lượng đặc trưng/lớp
2. **Tệp Triển khai (.c/.cpp/.py):** Triển khai trích xuất đặc trưng, logic dự đoán (đặc thù cho thuật toán) và các hàm tiện ích, với phần mở rộng phụ thuộc nền tảng đích
3. **Tệp Ví dụ Tích hợp:** Có thể là Arduino sketch (.ino), chương trình C/C++ ví dụ hoặc script MicroPython, minh họa khởi tạo cảm biến, quản lý bộ đệm của sổ trượt, vòng lặp suy luận thời gian thực và đầu ra dự đoán

Xác minh Công thức Thống kê: Mã C++ đã tạo cho trích xuất đặc trưng sử dụng các công thức thống kê được xác minh khớp chính xác với triển khai Python (pandas/NumPy). Cụ thể, các đặc trưng kurtosis và skewness sử dụng *độ lệch chuẩn tổng thể* (population standard deviation, chia n) cho chuẩn hóa z-scores, thay vì độ lệch chuẩn mẫu (chia $n - 1$). Sự khác biệt này, mặc dù nhỏ ($\sim 1,3\%$ cho $n = 150$), là hệ thống và tích lũy qua nhiều đặc trưng, có thể ảnh hưởng đến độ chính xác triển khai nếu không được xử lý.

3.6.5 Lượng tử hóa mô hình cho triển khai biên

Lượng tử hóa (model quantization) là kỹ thuật nén mô hình then chốt trong triển khai TinyML, chuyển đổi các tham số mô hình từ biểu diễn dấu phẩy động 32-bit (float32) sang số nguyên 8-bit (int8) hoặc 16-bit (int16). Trên vi điều khiển như Cortex-M4F với 256 KB RAM và 1 MB flash, lượng tử hóa có thể quyết định liệu một mô hình có thể triển khai được hay không: chuyển đổi từ float32 sang int8 giảm kích thước trọng số $4\times$, giảm băng thông bộ nhớ và tăng tốc suy luận $2\text{--}3\times$ nhờ các tập lệnh SIMD integer của ARM^[22].

Các loại lượng tử hóa

Lượng tử hóa sau huấn luyện (Post-Training Quantization, PTQ) PTQ áp dụng sau khi huấn luyện hoàn tất mà không cần sửa đổi quy trình huấn luyện. Mỗi tham số float32 x được ánh xạ sang giá trị lượng tử hóa x_q theo:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, 0, 255\right) \quad (3.6.1)$$

trong đó $s = \frac{x_{\max} - x_{\min}}{255}$ là hệ số tỷ lệ và $z \in [0, 255]$ là zero-point giữ nguyên giá trị số không. Có hai biến thể PTQ: *lượng tử hóa trọng số* (weight-only), trong đó chỉ trọng số mô hình được chuyển đổi còn phép tính suy luận vẫn dùng float32; và *lượng tử hóa tĩnh* (static PTQ), trong đó cả trọng số lẫn giá trị kích hoạt đều được lượng tử hóa, đòi hỏi một tập hiệu chỉnh nhỏ (100–500 mẫu đại diện) để đo phân phối kích hoạt trong thực tế. PTQ thường gây suy giảm độ chính xác 0,5–2% cho các mô hình HAR^[39].

Lượng tử hóa trong huấn luyện (Quantization-Aware Training, QAT) QAT chèn các toán tử *fake quantization* (lượng tử hóa giả) vào đồ thị tính toán trong quá trình huấn luyện, mô phỏng sai số lượng tử hóa để mô hình học cách bù đắp. QAT thường giảm suy giảm độ chính xác xuống dưới 0,3% nhưng đòi hỏi sửa đổi pipeline huấn luyện và thực tế chỉ áp dụng cho mạng nơ-ron—Random Forest và SVM có cấu

trúc rời rạc (ngưỡng quyết định, vector hỗ trợ) không tương thích với kỹ thuật đạo hàm giả (straight-through estimator) mà QAT dựa vào.

Chiến lược lượng tử hóa theo phương pháp tạo mã

Tùy theo phương pháp tạo mã được chọn, lượng tử hóa được áp dụng theo hai đường dẫn khác nhau:

Lượng tử hóa trọng số trong tạo mã trực tiếp Trong đường dẫn tạo mã trực tiếp, lượng tử hóa được thực hiện ở cấp độ biểu diễn tham số trong mã C++ đã tạo. Chế độ tối ưu hóa kiểm soát số chữ số thập phân khi nhúng hằng số float32 vào mã nguồn—tương đương lượng tử hóa trọng số mức thấp:

- **Accuracy:** float32 đầy đủ (6 chữ số thập phân), không xấp xỉ
- **Balanced:** 3 chữ số thập phân, tiết kiệm $\approx 30\%$ kích thước mã nguồn
- **Speed/Power:** 2 chữ số thập phân, tiết kiệm $\approx 45\%$ kích thước mã nguồn

Phép suy luận vẫn thực hiện bằng số học float32 trên FPU của Cortex-M4F—đây là lượng tử hóa *lưu trữ*, không phải lượng tử hóa *tính toán* đầy đủ. Đối với mạng nơ-ron, chế độ *Power* có thể tùy chọn biểu diễn ma trận trọng số ở int16, thực hiện nhân tích lũy integer và chỉ dequantize một lần trước hàm kích hoạt:

$$y_j = f\left(s \cdot \sum_i w_{ij}^{\text{int16}} \cdot x_i^{\text{int16}} + b_j\right) \quad (3.6.2)$$

trong đó s là tích của hai scale tương ứng (trọng số và đầu vào), $f(\cdot)$ là hàm kích hoạt ReLU hoặc softmax.

Lượng tử hóa INT8 đầy đủ qua đường dẫn TFLite Micro Đường dẫn TFLite Micro (áp dụng cho NN và CNN) hỗ trợ static PTQ với INT8 đầy đủ—cả trọng số lẫn kích hoạt. Quy trình thực hiện:

1. Chuyển đổi mô hình Keras/PyTorch sang TFLite (**TFLiteConverter**)
2. Một tập hiệu chỉnh gồm 100–500 vectors đặc trưng đại diện được cung cấp để TFLiteConverter đo phân phối kích hoạt từng lớp
3. Converter tự động tính s và z cho mỗi tensor theo Phương trình 3.6.1, tối ưu cho phạm vi kích hoạt thực tế

3.6. Tạo mã cho triển khai biên

- Mô hình INT8 kết quả kết hợp với ARM CMSIS-NN^[5] để khai thác tập lệnh SIMD của Cortex-M4, thực thi 4 phép nhân int8 song song mỗi chu kỳ—tăng tốc 2–3× so với mô hình float32 tương đương

Bảng 3.6.3 so sánh hai đường dẫn lượng tử hóa theo các tiêu chí triển khai:

Bảng 3.6.3: So sánh chiến lược lượng tử hóa theo phương pháp tạo mã

Tiêu chí	Tạo mã trực tiếp (Weight PTQ)	TFLite Micro (Static INT8 PTQ)
Áp dụng cho	RF, SVM, NN, CNN	NN, CNN (không hỗ trợ RF/SVM)
Giảm kích thước	30–45% (so với float32 đầy đủ)	4× (so với float32)
Tăng tốc suy luận	Không đáng kể (vẫn dùng FPU float32)	2–3× (SIMD int8)
Suy giảm độ chính xác	<0,5%	0,5–2%
Yêu cầu tập hiệu chỉnh	Không	100–500 mẫu
Tính minh bạch	Hoàn toàn (mã có thể kiểm tra)	Hạn chế (runtime hộp đen)

Đặc thù lượng tử hóa cho đặc trưng HAR miền thời gian

Bài toán HAR với đặc trưng thống kê miền thời gian có các đặc tính thuận lợi cho lượng tử hóa so với thị giác máy tính hay xử lý ngôn ngữ tự nhiên:

- Phạm vi đặc trưng có giới hạn vật lý:** Gia tốc IMU bị giới hạn ở $\pm 20 \text{ m/s}^2$, vận tốc góc ở $\pm 2000^\circ/\text{s}$. Do đó, các đặc trưng thống kê (trung bình, RMS, độ lệch chuẩn) có phạm vi xác định, giúp xác định scale lượng tử hóa chính xác mà không cần tập hiệu chỉnh lớn.
- Bền vững với nhiễu lượng tử hóa:** Random Forest quyết định theo ngưỡng nhị phân tại mỗi nút cây; sai số lượng tử hóa nhỏ hiếm khi đủ để đổi chiều quyết định phân nhánh, đặc biệt khi đặc trưng nằm xa ngưỡng. SVM nhạy hơn ở vùng ranh giới quyết định nhưng vẫn ổn định với int16.
- Tham số scaler phải đồng bộ độ chính xác:** StandardScaler (trung bình μ và độ lệch chuẩn σ) được áp dụng trước mô hình. Để duy trì parity huấn luyện-triển khai, tham số scaler cần được nhúng với cùng độ chính xác với tham số mô

hình—lượng tử hóa mô hình nhưng giữ scaler ở float32 đầy đủ sẽ gây ra sự không nhất quán hệ thống ảnh hưởng đến toàn bộ vector đặc trưng đầu vào.

- **Kurtosis và Skewness nhạy hơn:** Các đặc trưng hình dạng phân phối có phạm vi giá trị rộng hơn và nhạy với lượng tử hóa mạnh hơn các đặc trưng năng lượng (RMS, Energy). Do đó, chế độ *Balanced* khuyến nghị sử dụng 3 chữ số thập phân để bảo toàn các đặc trưng này.

3.6.6 Kiến trúc tạo mã đa kiến trúc

Hai loại pipeline mô hình tồn tại với yêu cầu metadata và validation khác biệt đáng kể, đòi hỏi kiến trúc tạo mã có khả năng nhận biết loại mô hình (architecture-aware):

Pipeline Feature-based vs End-to-end

Feature-based Models (RF, SVM, NN) Các mô hình này hoạt động trên extracted statistical features:

- **Input:** Vector đặc trưng thống kê (33, 53, 90 features tùy theo FE mode)
- **Preprocessing:** Áp dụng feature extraction pipeline đầy đủ
- **Scaling:** StandardScaler trên features đã extract
- **Metadata:** Feature names phản ánh các đặc trưng thống kê (e.g., "acc_x_mean", "gyro_y_kurtosis")

CNN End-to-end Models Mô hình CNN xử lý raw sensor windows trực tiếp:

- **Input:** Raw của số cảm biến ($150 \text{ samples} \times 6 \text{ channels} = 900 \text{ inputs}$)
- **Preprocessing:** Chỉ áp dụng signal filtering (IIR hoặc Kalman), không feature extraction
- **Scaling:** StandardScaler trực tiếp trên raw sensor data
- **Metadata:** Channel placeholders (e.g., "ch0", "ch1", ..., "ch5")

Architecture-Aware Validation và Metadata

Các kiến trúc khác nhau yêu cầu logic validation và metadata generation riêng biệt:

Ví dụ Metadata Consistency (Tìm kiếm Kỹ thuật Số 16): Một lỗi metadata đã được phát hiện và sửa chữa trong CNN pipeline:

- **Vấn đề:** CNN model hiển thị filename "f33" (tức 33 features), nhưng thực tế CNN sử dụng 6 channels per timestep
- **Nguyên nhân:** Fallback logic lấy nhầm feature names từ session FE khác thay vì tạo channel placeholders
- **Giải pháp:** Architecture-aware metadata generation—CNN models luôn sử dụng channel placeholders

Phân biệt với Edge Impulse: Edge Impulse xử lý sự phức tạp này bằng cách có “processing blocks” riêng biệt cho feature-based và raw-input models. Tương tự, cần phân biệt loại mô hình tại thời điểm tạo mã để sinh metadata và validation logic chính xác.

Lưu ý về tính nhất quán Metadata

Trường `trained_models.json` có ngữ nghĩa khác nhau tùy theo loại mô hình:

- `features`: `X_train.shape[1]`—với CNN là `window_size_samples` (150), với MLP là feature count (33, 53, hoặc 90)
- `num_features_extracted`: Luôn là output count của FE pipeline—có ý nghĩa cho feature-based models, chỉ mang tính thông tin cho CNN

3.6.7 Các chiến lược tối ưu hóa

Bốn chế độ tối ưu hóa triển khai có thể cấu hình khi tạo mã, mỗi chế độ thể hiện đánh đổi khác nhau giữa độ chính xác và chi phí tài nguyên:

Chế độ Accuracy Ưu tiên độ chính xác dự đoán:

- Số học dấu phẩy động độ chính xác đầy đủ
- Không đơn giản hóa mô hình (tất cả cây/vectors hỗ trợ được giữ lại)
- Bộ đặc trưng hoàn chỉnh (toàn bộ đặc trưng được chọn, không cắt giảm)

Chế độ Speed Tối thiểu hóa độ trễ suy luận:

- Giảm độ phức tạp mô hình (ít cây hơn, giảm vectors hỗ trợ SVM qua cắt tỉa)
- Thứ tự tính toán đặc trưng tối ưu hóa (các mẫu truy cập cache-friendly)
- Mở rộng vòng lặp và các hàm nội tuyến

Chế độ Power Giảm tiêu thụ năng lượng:

- Số học số nguyên điểm cố định và biểu diễn trọng số int16 khi khả thi (xem Mục 3.6.5)
- 2 chữ số thập phân cho tham số mô hình và scaler, tiết kiệm $\approx 45\%$ kích thước flash
- Giảm tần số lấy mẫu (có thể cấu hình xuống 50Hz)
- Các chế độ ngủ giữa các chu kỳ suy luận (chưa được triển khai trong phiên bản hiện tại)

Chế độ Balanced Thỏa hiệp giữa độ chính xác, tốc độ và điện năng:

- Độ phức tạp mô hình trung bình
- 3 chữ số thập phân cho tham số—đủ bảo tồn các đặc trưng nhạy cảm như kurtosis và skewness trong khi tiết kiệm 30% flash (xem Mục 3.6.5)
- Độ chính xác hỗn hợp: các tính toán quan trọng (softmax, chuẩn hóa scaler) dùng float32, các phép nhân ma trận trung gian có thể dùng điểm cố định

3.6.8 Các Điều chỉnh Đặc thù Nền tảng

Điều chỉnh đặc thù nền tảng được nhóm theo họ thiết bị đích, tách biệt logic suy luận mô hình (dùng chung) khỏi các chi tiết phụ thuộc môi trường (đặc thù từng họ thiết bị):

- **Arduino-compatible boards:** sinh cấu trúc tệp `.ino/.cpp/.h`, dùng `Serial`, `Wire` và các đoạn mã khởi tạo cảm biến phù hợp cho từng board phổ biến.
- **ARM Cortex-M native:** sinh mã C độc lập, ưu tiên bộ nhớ tĩnh và sẵn sàng tích hợp vào firmware bare-metal hoặc CMSIS.
- **SDK/RTOS native:** điều chỉnh include, macro logging và entry-point cho `generic_c`, `generic_cpp`, ESP-IDF và Zephyr.
- **MicroPython:** sinh module Python độc lập cùng ví dụ gọi hàm, đồng thời duy trì tương đồng công thức với bản C/C++ để thuận tiện cho nguyên mẫu nhanh.

Nhờ cách phân lớp này, cùng một mô hình đã huấn luyện có thể được xuất lại cho nhiều đích triển khai mà không cần huấn luyện lại, ngoại trừ các điều chỉnh phụ thuộc tài nguyên như chế độ tối ưu hóa hoặc backend suy luận.

3.7 Thiết kế thực nghiệm

3.7.1 Tập dữ liệu

Các thử nghiệm xác thực sử dụng một tập dữ liệu Nhận dạng Hoạt động Con người (HAR) được thu thập từ một đối tượng duy nhất thực hiện năm hoạt động:

- *Running, Still, Walking, Walking Downstairs, Walking Upstairs*

Dữ liệu được thu thập bằng nền tảng Seeed XIAO nRF52840 Sense ở 100Hz qua nhiều phiên ghi. Trong luận văn này, thiết bị này được dùng như nền tảng cảm biến và benchmark tham chiếu; năng lực đa thiết bị được đánh giá riêng thông qua kiến trúc generator và các loại gói sinh ra cho từng đích. Mỗi hoạt động được ghi trong 30–60 giây qua nhiều phiên, với chú trọng vào việc ghi nhận đầy đủ các giai đoạn chuyển tiếp giữa các hoạt động. Giao diện cửa sổ kéo thả được sử dụng để phân đoạn các bản ghi liên tục thành **mẫu được gán nhãn** (các đoạn hoạt động được chú thích thủ công), trích xuất nhiều cửa sổ đại diện mỗi hoạt động.

Quy trình Trích xuất Đặc trưng: Mỗi mẫu được gán nhãn (thường dài 30–60 giây) sau đó được xử lý bằng cách tiếp cận cửa sổ trượt (Phần 3.4) với cửa sổ 1,5 giây và chồng lấp 50%. Điều này chuyển đổi các mẫu được gán nhãn thành 507 **cửa sổ đặc trưng** cho bài toán 5 lớp (345 cho bài toán 3 lớp). Tăng cường dữ liệu (augmentation) với hệ số 2 sử dụng ba phương pháp (jitter, rotation, scaling) tạo thêm 1014 cửa sổ tổng hợp, nâng tổng lên 1521 mẫu (5 lớp) hoặc 1035 mẫu (3 lớp). Dữ liệu được chia theo tỷ lệ 70% huấn luyện, 15% xác thực, 15% kiểm tra sử dụng phân chia phân tầng (stratified split), cho ra: huấn luyện 1063 mẫu, xác thực 229 mẫu, kiểm tra 229 mẫu (5 lớp). Phân bố các lớp hoạt động (trước tăng cường):

- Running: 124 cửa sổ (24,5%)
- Still: 112 cửa sổ (22,1%)
- Walking: 109 cửa sổ (21,5%)
- Walking Downstairs: 85 cửa sổ (16,8%)
- Walking Upstairs: 77 cửa sổ (15,2%)

Tỷ lệ mất cân bằng lớp: 1,61:1 (running so với walking upstairs), đại diện cho một tập dữ liệu tương đối cân bằng. Mỗi vector đặc trưng chứa 63 đặc trưng bất biến hướng (orientation-invariant).

Cấu hình đánh giá: Các thử nghiệm được thực hiện với hai cấu hình phân lớp:

- **5 lớp** (đầy đủ): Running, Still, Walking, Walking Downstairs, Walking Upstairs—đánh giá khả năng phân biệt các hoạt động có tín hiệu IMU tương đồng
- **3 lớp** (gộp): Running, Still, Walking (gộp ba biến thể đi bộ thành một lớp duy nhất)—đánh giá triển khai thực tế

Cấu hình 3 lớp phản ánh thực tế rằng các biến thể đi bộ (bằng phẳng, lên cầu thang, xuống cầu thang) tạo ra tín hiệu IMU rất tương đồng khi chỉ sử dụng một cảm biến đeo cổ tay, gây nhầm lẫn hệ thống giữa các lớp walking trong kiểm thử trên thiết bị thực. Nhiều ứng dụng thực tế (theo dõi sức khỏe, phát hiện ngã) chỉ cần phân biệt mức độ hoạt động tổng thể (tĩnh/đi bộ/chạy) thay vì chi tiết hóa kiểu di chuyển, do đó cấu hình 3 lớp phù hợp hơn cho triển khai biên.

3.7.2 Các số liệu đánh giá

Hiệu suất mô hình được đánh giá bằng cách sử dụng:

- **Độ chính xác tổng thể:** Phần trăm của số được phân loại chính xác
- **Precision từng lớp:** $P_c = \frac{TP_c}{TP_c + FP_c}$
- **Recall từng lớp:** $R_c = \frac{TP_c}{TP_c + FN_c}$
- **F1-Score:** Trung bình điều hòa của precision và recall
- **Confusion Matrix:** Trực quan hóa các mẫu phân loại sai

Hiệu suất triển khai biên được đo qua:

- **Thời gian suy luận:** Thời gian trung bình để phân loại một cửa sổ 150 mẫu (ms), được đo bằng cách sử dụng thời gian `Arduino micros()` qua 100 dự đoán liên tiếp trên phần cứng đích
- **Sử dụng bộ nhớ:** Flash (lưu trữ chương trình) và SRAM (biến/bộ đệm thời gian chạy) tiêu thụ (KB), được trích xuất từ các tệp bản đồ trình biên dịch và đầu ra xây dựng Arduino IDE cho thấy kích thước nhị phân đã biên dịch thực tế
- **Tiêu thụ điện năng:** Dòng điện trung bình rút trong các giai đoạn hoạt động khác nhau (nhàn rỗi, lấy mẫu cảm biến, trích xuất đặc trưng, dự đoán)¹

¹Các giá trị điện năng được báo cáo là ước lượng tính toán từ đặc tả dòng điện ARM Cortex-M4F điển hình (8–12 mA hoạt động, 2–3 mA nhàn rỗi ở 64 MHz) và phần trăm thời gian thực thi đã đo, không phải đo lường trực tiếp bằng thiết bị chuyên dụng (Joulescope, Nordic PPK2). Giá trị thực tế có thể thay đổi $\pm 15\text{--}20\%$.

- **Tần số dự đoán:** Tốc độ suy luận bền vững tính đến chi phí lấy mẫu cảm biến (dự đoán mỗi giây)
- **Độ phủ đích triển khai:** Số lượng họ nền tảng và backend triển khai có thể được sinh mã từ cùng một mô hình đã huấn luyện, được xác minh qua ma trận generator và cấu trúc tệp đầu ra

Các ngưỡng hiệu suất mục tiêu dựa trên yêu cầu ứng dụng: suy luận < 100ms mỗi cửa sổ (cho phép cập nhật 10Hz thời gian thực), flash < 500KB (để lại khoảng trống cho mã ứng dụng), SRAM < 128KB (50% của 256KB có sẵn), điện năng < 50mW trung bình (hỗ trợ hoạt động pin nhiều giờ).

3.7.3 So sánh cơ sở

Kết quả được so sánh trực tiếp với **Edge Impulse** (gói miễn phí)—nền tảng TinyML thương mại phổ biến nhất. Cùng tập dữ liệu 3 lớp (running/still/walking) được tải lên Edge Impulse và huấn luyện với mô hình CNN mặc định (đặc trưng phổ, lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz). So sánh bao gồm độ chính xác huấn luyện, tài nguyên triển khai và hiệu suất trên thiết bị thực (Mục 5.6).

Chi tiết triển khai phần mềm và phần cứng được trình bày đầy đủ trong Chương 4.

Chương 4

Thiết kế và hiện thực

Chương này trình bày kiến trúc hệ thống, quyết định thiết kế và chi tiết hiện thực của framework triển khai nhận dạng hoạt động trên thiết bị biên. Các thuật toán và cơ sở lý thuyết đã được trình bày trong Chương 3; chương này tập trung vào **kỹ thuật phần mềm, mô-đun hóa, và quy trình triển khai** của hệ thống thực tế. Framework được xây dựng dựa trên nguyên tắc tách biệt vai trò (separation of concerns): mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline từ dữ liệu thô đến mã triển khai, đảm bảo tính mở rộng và khả năng bảo trì.

4.1 Kiến trúc hệ thống

4.1.1 Ngăn xếp công nghệ và môi trường phát triển

Framework được triển khai dưới dạng ứng dụng web một trang (single-page web application) với các thành phần công nghệ sau:

Ngăn xếp phần mềm

- **Python 3.10**: ngôn ngữ lập trình chính, được chọn vì hệ sinh thái học máy phong phú và khả năng tích hợp nhanh giữa các thư viện khác nhau.
- **Dash 2.14**: framework giao diện web dựa trên Flask và Plotly. Dash cung cấp mô hình lập trình khai báo (declarative) với callback pattern, cho phép xây dựng giao diện tương tác phức tạp mà không cần JavaScript thủ công.
- **scikit-learn 1.3**: triển khai các thuật toán học máy truyền thống (Random Forest, SVM, MLP). Thư viện cung cấp API nhất quán và hiệu năng đã được tối ưu hóa.

- **PyTorch 2.x**: framework học sâu cho huấn luyện mô hình MLP và CNN. PyTorch được chọn vì API linh hoạt, hỗ trợ tốt cho tùy chỉnh kiến trúc, và khả năng xuất mô hình sang ONNX/TFLite.
- **NumPy/Pandas**: xử lý dữ liệu số học và cấu trúc dạng bảng. Pandas DataFrame cung cấp giao diện thuận tiện cho thao tác chuỗi thời gian và trích xuất đặc trưng.
- **Plotly**: thư viện trực quan hóa tương tác, hỗ trợ drag-and-drop selection và real-time updates trong giao diện Dash.
- **Jinja2**: template engine để sinh mã C/C++ và Arduino từ các mẫu tham số hóa, đảm bảo tính nhất quán cú pháp và dễ bảo trì.

Môi trường phát triển

- **Nền tảng phát triển**: Windows 11 workstation (Intel Core i7, 16GB RAM). Tuy nhiên, framework được thiết kế platform-agnostic và chạy được trên Linux/macOS mà không cần điều chỉnh.
- **Nền tảng chuẩn triển khai**: Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64MHz, 256KB RAM, 1MB Flash, LSM6DS3 IMU). Vi điều khiển này được chọn làm **nền tảng chuẩn đo benchmark** do: (1) tích hợp đầy đủ IMU 6-trục, (2) hiệu năng đại diện cho phân khúc MCU trung bình, (3) hỗ trợ Arduino và PlatformIO, và (4) giá thành thấp (~\$10 USD).
- **Công cụ biên dịch**: Arduino IDE 2.3 và PlatformIO cho biên dịch chéo (cross-compilation) sang ARM Cortex-M. Cả hai đều sử dụng GCC ARM Embedded Toolchain với flag tối ưu hóa -O2 (cân bằng kích thước và tốc độ).

4.1.2 Kiến trúc ứng dụng

Framework áp dụng kiến trúc phân lớp rõ ràng với nguyên tắc *separation of concerns*. Hệ thống được tổ chức thành bốn lớp chính: (1) **lớp giao diện** chứa định nghĩa layout Dash cho mỗi tab; (2) **lớp callback** xử lý tương tác người dùng và điều phối luồng dữ liệu; (3) **lớp tiện ích** chứa các thuật toán xử lý tín hiệu, trích xuất đặc trưng và huấn luyện mô hình; (4) **lớp tạo mã** chứa hệ thống generator cho từng mô hình và nền tảng đích. Trạng thái liên phiên được lưu xuống hệ thống file, tách biệt với trạng thái tạm thời trong session.

Quy trình đăng ký callback Dash sử dụng mô hình *callback pattern* để đồng bộ giao diện và trạng thái. Mỗi module callback định nghĩa một hàm `register_callbacks(app)`, bên trong đó tập trung toàn bộ logic xử lý sự kiện cho tab tương ứng. Entry point ứng dụng gọi tuần tự các hàm đăng ký này sau khi khởi tạo layout.

Cách tổ chức này đảm bảo:

1. **Tính module:** mỗi module callback độc lập, dễ bảo trì và kiểm thử riêng biệt.
2. **Tránh circular imports:** callbacks không import lẫn nhau, chỉ import từ lớp tiện ích và lớp tạo mã.
3. **Testability:** mỗi callback có thể kiểm thử riêng bằng cách mock Dash context.

4.1.3 Quản lý trạng thái

Framework sử dụng kết hợp hai cơ chế lưu trạng thái:

Hệ thống file (`persistent_data/`) Dữ liệu trung gian giữa các giai đoạn được lưu dưới dạng file CSV, pickle (scikit-learn models) và joblib (model bundles):

- **datasets/**: dữ liệu CSV gốc theo activity class.
- **windows/**: sliding windows đã sinh (pickle format).
- **training/**: feature matrices (CSV) và metrics.
- **models/**: model serialization (joblib) với dict keys chuẩn:
 - **model**: trained model object (sklearn/PyTorch)
 - **scaler**: StandardScaler fitted trên train set
 - **label_encoder**: mapping class names $\leftrightarrow$ integers
 - **feature_names**: danh sách features theo thứ tự huấn luyện
 - **model_type**: string identifier (rf/svm/nn/cnn)
 - **model_params**: hyperparameters
 - **performance_metrics**: accuracy, F1, confusion matrix
- **generated/**: mã nguồn triển khai theo cấu trúc `{model_type}_models/{model_name}/`.

In-memory state (dcc.Store) Dash component `dcc.Store` lưu trạng thái session tạm thời (VD: metadata cột cảm biến, tần số lấy mẫu, danh sách segments đã chọn). State này được truyền giữa các callbacks trong cùng một session nhưng không persist sau khi reload trang.

Luồng dữ liệu Dữ liệu chảy qua pipeline như sau:

1. **Tab 1** → **Tab 2**: CSV files → `datasets/` → metadata in `dcc.Store`
2. **Tab 2** → **Tab 3**: Segments selection → sliding windows → `windows/{dataset_name}.pkl`
3. **Tab 3** → **Tab 4**: Feature extraction → `training/{dataset_name}_train.csv`
4. **Tab 4** → **Tab 5**: Trained model → `models/{model_name}.joblib`
5. **Tab 5** → **Tab 6**: Generated code → `generated/{model_type}_models/{model_name}/`

Thiết kế này cho phép người dùng quay lại bất kỳ bước nào để điều chỉnh mà không mất dữ liệu trung gian.

4.2 Module quản lý dữ liệu

Module này (Tab 1 trong giao diện) chịu trách nhiệm tải dữ liệu cảm biến và kiểm tra trực quan chất lượng tín hiệu.

4.2.1 Giao diện tải dữ liệu

Người dùng tải lên nhiều file CSV, mỗi file chứa dữ liệu cho một activity class. Framework tự động phát hiện cấu trúc cột dựa trên quy ước đặt tên:

- **Gia tốc kế:** `aX`, `aY`, `aZ` hoặc `AccX`, `AccY`, `AccZ`
- **Con quay hồi chuyển:** `gX`, `gY`, `gZ` hoặc `GyroX`, `GyroY`, `GyroZ`
- **Timestamp:** `time`, `timestamp`, `Time(s)` (optional, dùng để tính tần số lấy mẫu)

Thuật toán phát hiện cột hoạt động theo hai bước: đầu tiên khớp chính xác với danh sách quy ước đặt tên phổ biến; nếu không tìm thấy, fallback sang khớp substring (VD: “acc”, “gyro”) trong tên cột.

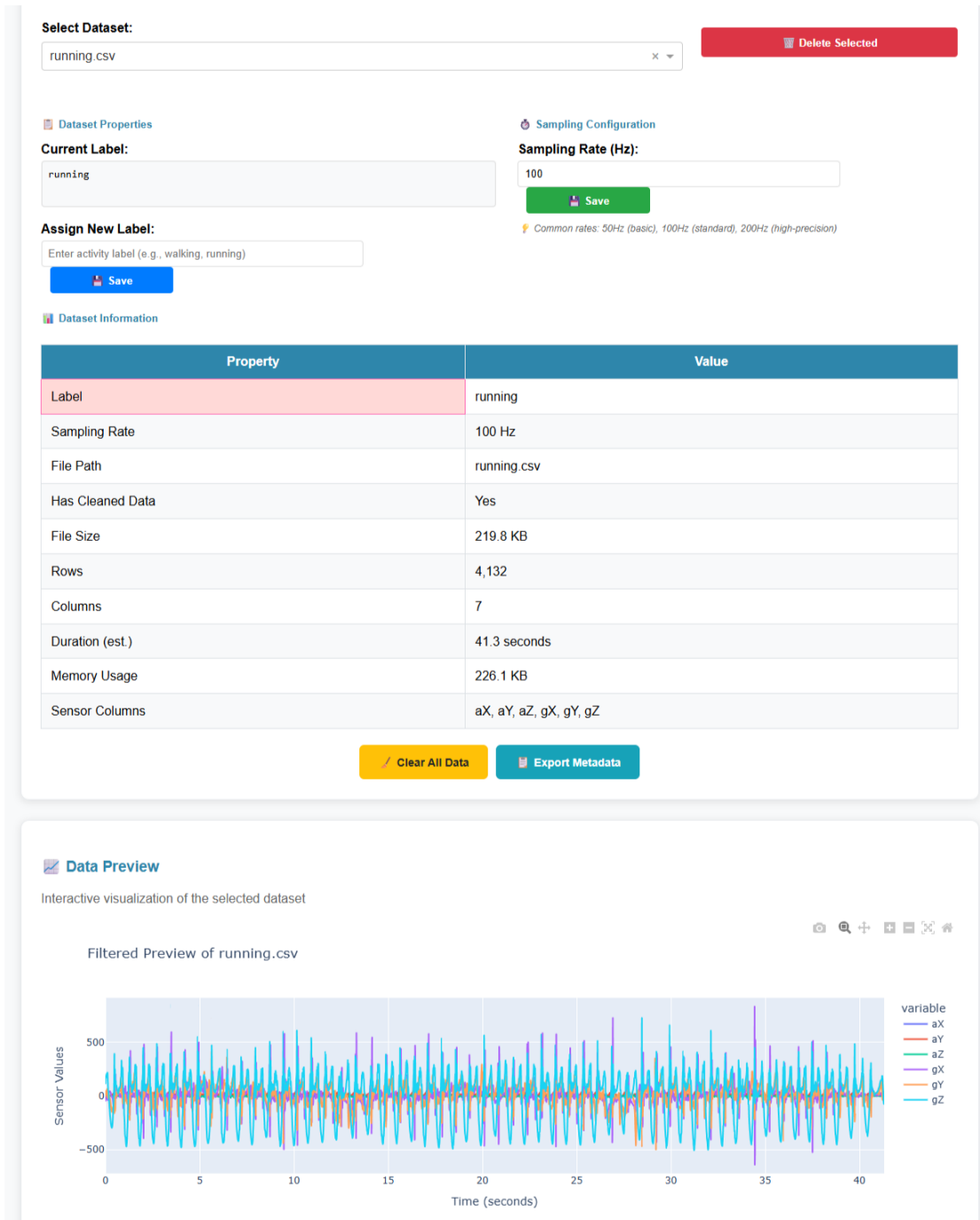
4.2.2 Trực quan hóa tín hiệu đồng bộ

Sau khi tải dữ liệu, framework hiển thị biểu đồ tín hiệu cho tất cả các trục cảm biến với shared x-axis để đồng bộ zoom và pan giữa các trục.

Hình 4.2.1 minh họa giao diện này. Trực quan hóa này cho phép người dùng phát hiện các vấn đề chất lượng dữ liệu như:

- **Nhiều quá mức:** biên độ tín hiệu dao động lớn không đặc trưng cho hoạt động.
- **Outliers:** các spike đơn lẻ do lỗi cảm biến hoặc va chạm.
- **Saturation:** tín hiệu đạt giá trị max/min range của cảm biến (VD: $\pm 16g$).
- **Dữ liệu không đồng nhất:** tín hiệu chứa nhiều hoạt động khác nhau trong cùng một file.

4.2. Module quản lý dữ liệu



Hình 4.2.1: Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiến xử lý.

4.2.3 Phát hiện tần số lấy mẫu

Framework tự động tính tần số lấy mẫu từ cột timestamp bằng cách lấy nghịch đảo của khoảng thời gian trung bình giữa các mẫu liên tiếp.

Nếu không có cột timestamp, người dùng nhập tần số lấy mẫu thủ công. Tần số này được lưu trong metadata và nhúng vào mã triển khai dưới dạng hằng số.

4.3 Module tiền xử lý tương tác

Module này (Tab 2) cung cấp giao diện kéo thả (drag-and-drop) để chọn các đoạn hoạt động đại diện trên biểu đồ tín hiệu, sau đó tự động sinh sliding windows từ các đoạn đã chọn.

4.3.1 Giao diện lựa chọn đoạn tương tác

Dash Plotly hỗ trợ *editable shapes*: người dùng vẽ hình chữ nhật trực tiếp trên biểu đồ để đánh dấu vùng quan tâm. Khi người dùng vẽ hoặc chỉnh sửa một hình chữ nhật, sự kiện layout được kích hoạt và framework đọc tọa độ x-axis để xác định phạm vi mẫu của đoạn vừa chọn.

Framework kiểm tra ràng buộc cơ bản:

- **Chiều dài tối thiểu:** mỗi segment phải có ít nhất `window_size` mẫu để sinh được ít nhất 1 cửa sổ.

Các segment có thể chồng chéo nhau — điều này cho phép người dùng thu thập mẫu từ nhiều vùng khác nhau trong cùng một đoạn tín hiệu.

Hình 4.3.1 minh họa giao diện này.

4.3. Module tiền xử lý tương tác



Hình 4.3.1: Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.

4.3.2 Sinh cửa sổ trượt

Từ mỗi segment, framework áp dụng thuật toán sliding window: duyệt từ đầu đến cuối segment với bước nhảy (stride) cố định, mỗi bước cắt ra một cửa sổ có độ dài `window_size`.

Tham số mặc định:

- `window_size` = 150 mẫu (1.5s ở 100Hz)
- `stride` = 75 mẫu (50% overlap)

Overlap 50% được chọn để tăng số lượng mẫu huấn luyện đồng thời giữ sự đa dạng. Stride càng nhỏ (overlap càng lớn) càng tạo nhiều windows nhưng có nguy cơ overfitting do correlation giữa các windows liên kề.

4.3.3 Trực quan hóa lọc tín hiệu

Framework hỗ trợ ba phương pháp lọc tín hiệu (chi tiết thuật toán đã trình bày trong Chương 3):

1. **Butterworth lowpass filter** (filtfilt for zero-phase)
2. **Savitzky-Golay filter** (polynomial smoothing)
3. **Kalman filter** (causal, exact training-deployment parity)

Giao diện hiển thị overlay tín hiệu raw (màu xanh) và filtered (màu đỏ) để người dùng đánh giá hiệu quả lọc. Người dùng có thể điều chỉnh tham số (cutoff frequency, filter order, window length) và quan sát thay đổi real-time.

4.4 Module trích xuất đặc trưng

Module này (Tab 3) chịu trách nhiệm chuyển đổi cửa sổ tín hiệu thô sang feature vectors và chuẩn bị train/val/test splits.

4.4.1 Chế độ trích xuất đặc trưng

Framework hỗ trợ 6 chế độ feature extraction, mỗi chế độ tạo ra một tập đặc trưng khác nhau:

Bảng 4.4.1: Các chế độ trích xuất đặc trưng và số lượng features

Chế độ	Mô tả	Số features
orientation_invariant_time_only	Magnitude stats (acc+gyro) + jerk	33
orientation_invariant	Time + frequency domain magnitudes	53
time_domain	Per-axis time stats + magnitudes	90
all	Time + frequency, per-axis + magnitudes	156
frequency_domain	FFT, PSD, spectral features	66
raw	Cửa sổ tín hiệu raw (CNN input)	6 channels

Rationale cho orientation-invariant features Chế độ `orientation_invariant` chỉ sử dụng các đặc trưng tính trên magnitude vectors ($\sqrt{x^2 + y^2 + z^2}$) thay vì per-axis features. Điều này đảm bảo mô hình không bị ảnh hưởng bởi định hướng lắp đặt cảm biến: đeo thiết bị ngang hoặc dọc, xoay 90° vẫn cho cùng một magnitude. Tuy nhiên, trade-off là mất thông tin về hướng chuyển động (VD: leo cầu thang khó phân biệt với xuống cầu thang chỉ dựa trên magnitude). Thực nghiệm cho thấy chế độ `time_domain` (90 features) cân bằng tốt giữa độ chính xác và độ bền hướng.

4.4.2 Tích hợp data augmentation

Framework triển khai 5 phương pháp data augmentation cho dữ liệu IMU:

1. **Jitter**: thêm Gaussian noise nhỏ vào tín hiệu ($\sigma = 0.05 \times \text{signal_std}$).
2. **Rotation**: xoay ngẫu nhiên hệ trục tọa độ 3D để mô phỏng thay đổi định hướng.
3. **Scaling**: nhân tín hiệu với hệ số ngẫu nhiên (0.9–1.1) để mô phỏng biên độ chuyển động khác nhau.
4. **Time warping**: kéo giãn hoặc nén trục thời gian (0.8–1.2×) để mô phỏng tốc độ thực hiện hoạt động khác nhau.
5. **Permutation**: hoán vị các sub-windows để phá vỡ sự phụ thuộc thứ tự thời gian (chỉ áp dụng cho hoạt động tuần hoàn như đi bộ).

Class-aware augmentation Framework tự động phân loại hoạt động thành hai loại:

- **Static/transition**: đứng yên, ngồi, nằm → KHÔNG áp dụng time warping và permutation (gây mất tính tự nhiên).
- **Dynamic/periodic**: đi bộ, chạy, leo cầu thang → áp dụng tất cả augmentation.

Phân loại dựa trên từ khóa trong tên activity: framework kiểm tra xem tên hoạt động có chứa các từ khóa tĩnh (“still”, “stand”, “sit”, “lie”, “stationary”) hay không, và áp dụng tập augmentation phù hợp.

Giao diện augmentation cho phép người dùng:

- Kích hoạt/tắt từng phương pháp augmentation riêng biệt.
- Điều chỉnh augmentation factor (số lượng samples tăng thêm: 1×, 2×, 3×).

4.4. Module trích xuất đặc trưng

- Xem phân phối class trước và sau augmentation để đảm bảo cân bằng.

Hình 4.4.1 minh họa giao diện này.

Step 2: Configure Feature Engineering

Settings will be applied uniformly across all selected datasets

Feature Extraction Method:

Orientation-Invariant Time-Domain ONLY - 41 features

Note: Orientation-robust DFT (63 features) is fully deployable — code generators emit a lightweight sin/cos DFT. Only per-axis frequency features (66 / 156) cannot be deployed.

41 features

Orientation-robust magnitudes: 15 stats × 2 mag + 3 acc-jerk + 3 gyro-jerk + 5 scalar extras (SMA, tilt, autocorr, peak count). RECOMMENDED for deployment — fully deployable to all targets (C / C++ / MicroPython).

Normalization Method (applied during training):

Standard Scaler (Z-score)

Note: Normalization is **not applied here** — it is saved to metadata and applied during **model training** so the scaler is handled with the model for deployment.

Window Configuration

Target Window Size (ms):

1500

Sampling Rate (Hz):

100

Windows smaller than this will be zero-padded (1500ms ~ 150 samples @ 1000Hz)

Used to calculate time duration from samples

Step 3: Data Augmentation (Optional)

Generate synthetic training windows to improve model robustness. Augmentation is applied to raw sensor signals before feature extraction, so new features are computed naturally from augmented data.

☐ Enable Data Augmentation

Step 4: Configure Dataset Split

Define how to split the combined dataset into train/validation/test sets

Split Mode:

☒ Auto (ratio-based random split) ☐ Manual (assign each window by hand)

Training Set Ratio:

0.7

Validation Set Ratio:

0.15

Test Set Ratio:

15%

Random Seed (for reproducibility):

42

Same seed = reproducible splits across runs

Hình 4.4.1: Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.

4.4.3 Phân chia train/validation/test

Framework cung cấp hai chế độ phân chia dữ liệu để phù hợp với nhiều kịch bản thực tế:

Phân chia tự động theo tỉ lệ Người dùng chọn tỉ lệ train/validation/test bằng thanh trượt trực tiếp trên giao diện. Framework sử dụng **stratified sampling** để đảm bảo tỉ lệ class đồng đều giữa các tập. Kỹ thuật này quan trọng khi dữ liệu mất cân bằng: ví dụ nếu 70% mẫu thuộc class “đứng yên”, thì cả train và test đều có 70%

mẫu đó. Tỷ lệ mặc định là train 60% / validation 20% / test 20%, nhưng người dùng có thể điều chỉnh tùy nhu cầu.

Phân chia thủ công theo dataset Đối với dữ liệu thu thập từ nhiều đối tượng hoặc nhiều phiên, người dùng có thể chỉ định trực tiếp file CSV nào thuộc tập train và file nào thuộc tập test. Chức năng này cho phép đánh giá *cross-subject generalization*: huấn luyện trên dữ liệu của một nhóm người dùng và kiểm thử trên nhóm hoàn toàn khác, tránh data leakage giữa các đối tượng.

Validation set được dùng cho hyperparameter tuning (GridSearchCV sử dụng cross-validation trên train+val), test set giữ nguyên không động đến cho đến khi đánh giá cuối cùng.

4.5 Module huấn luyện mô hình

Module này (Tab 4) triển khai training pipeline với hyperparameter optimization và real-time monitoring.

4.5.1 Lựa chọn thuật toán

Framework hỗ trợ 6 thuật toán phân loại:

Bảng 4.5.1: Thuật toán học máy được hỗ trợ

Thuật toán	Thư viện	Input	Triển khai
Random Forest	scikit-learn	Features	C++ native
SVM (RBF kernel)	scikit-learn	Features	C++ native
Neural Network (MLP)	scikit-learn	Features	C++ native
PyTorch MLP	PyTorch	Features	C++/TFLite
PyTorch 1D-CNN	PyTorch	Raw signals	C++/TFLite
PyTorch 2D-CNN	PyTorch	Raw signals (2D)	C++/TFLite

Kiến trúc PyTorch models

- **PyTorch MLP**: 2 hidden layers với ReLU activation. Kích thước layer điều chỉnh theo số features: `[n_features → 128 → 64 → n_classes]`.
- **PyTorch 1D-CNN**: convolutional layers trên time dimension, kernel size = 5, 2 conv layers + pooling + fully connected. Input shape: `[batch, 6 channels, 150 timesteps]`.

- **PyTorch 2D-CNN (HARCNN2D)**: sử dụng Conv2D với kernel $3 \times n_channels$ để học correlation giữa các sensor axes. Input shape: [batch, 1, 150 timesteps, 6 channels] (reshape từ 1D signal).

Kiến trúc 2D-CNN đặc biệt hữu ích khi cần học mối quan hệ không tuyến tính giữa các trục cảm biến (VD: correlation giữa aX và gY trong động tác xoay).

4.5.2 Tối ưu hóa siêu tham số

Framework tích hợp GridSearchCV từ scikit-learn với param grid mặc định cho từng thuật toán. Random Forest tìm kiếm trên các giá trị `n_estimators` (50, 100, 200), `max_depth` (10, 20, None) và `min_samples_split`; SVM tìm kiếm trên tham số `C` và `gamma`; Neural Network tìm kiếm trên kích thước hidden layers, hệ số L2 regularization và learning rate.

GridSearchCV sử dụng 5-fold cross-validation trên train+validation set, chọn tổ hợp params có accuracy cao nhất, sau đó retrain trên toàn bộ train+val với params tối ưu.

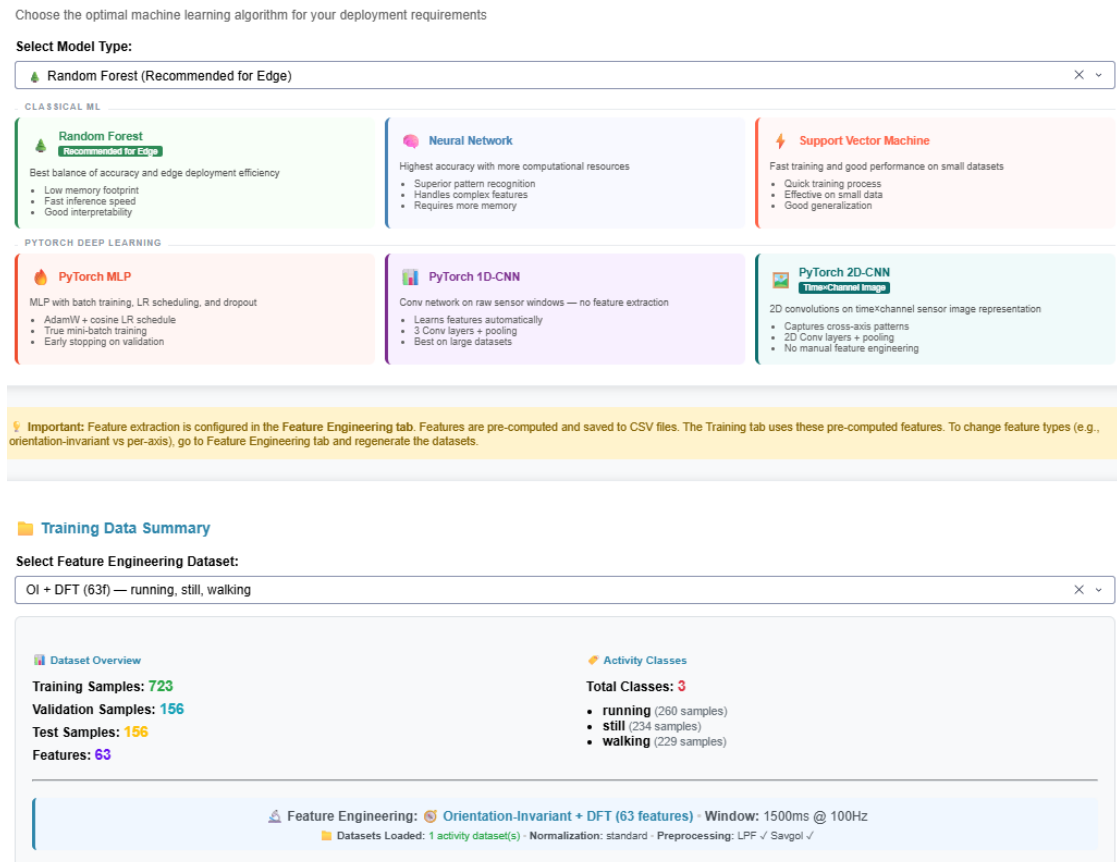
4.5.3 Theo dõi tiến trình real-time

Callback training sử dụng `dcc.Interval` component để cập nhật UI mỗi 2 giây trong quá trình huấn luyện. Thông tin hiển thị:

- **Training progress**: phần trăm hoàn thành, thời gian đã trôi qua.
- **Current hyperparameters**: params đang thử nghiệm.
- **Best score so far**: accuracy cao nhất trong các trials đã hoàn thành.
- **Confusion matrix**: ma trận nhầm lẫn trên validation set, cập nhật sau mỗi trial.

Hình 4.5.1 minh họa giao diện này.

4.5. Module huấn luyện mô hình



Hình 4.5.1: Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.

4.5.4 Serialization mô hình

Sau khi huấn luyện hoàn tất, framework serialize toàn bộ pipeline artifacts vào một file `.joblib`. Bộ mô hình bao gồm: đối tượng model đã huấn luyện, bộ chuẩn hóa `StandardScaler`, ánh xạ nhãn (label encoder), danh sách tên features (để đảm bảo thứ tự đúng khi suy diễn), kiểu mô hình, siêu tham số tốt nhất, và các chỉ số hiệu suất (accuracy, F1, confusion matrix, classification report).

Chuẩn nội bộ này là hợp đồng giao diện giữa module huấn luyện và module tạo mã. Module code generation dựa vào dict bundle để trích xuất parameters (VD: cây quyết định của Random Forest, trọng số của Neural Network) và sinh mã triển khai tương ứng.

4.6 Module tạo mã triển khai

Module này (Tab 5) chịu trách nhiệm chuyển đổi trained model thành mã nguồn embedded C/C++ hoặc các định dạng khác (MicroPython, TFLite). Đây là thành phần phức tạp nhất của framework, vì phải đảm bảo **training-deployment parity**: mã được sinh ra phải tính toán đúng y hệt như Python pipeline.

4.6.1 Kiến trúc Code Generator: Factory Pattern

Framework sử dụng *Factory Pattern* để định tuyến từ (`model_type`, `target_platform`, `deployment_backend`) sang class generator phù hợp. Khi nhận model bundle, factory kiểm tra trường `model_type`: nếu deployment backend là TFLite Micro thì tạo `TFLiteCodeGenerator`; nếu không, sẽ tạo generator tương ứng với thuật toán (Random Forest, Neural Network, CNN, SVM). Tham số platform được truyền vào để generator điều chỉnh cú pháp và API cho từng nền tảng đích.

4.6.2 Hệ thống phân cấp Generator

Block pattern (kiến trúc hiện tại) Module tạo mã được tổ chức theo *block pattern*: `HARCodeGenerator` là coordinator, nhận vào model bundle và orchestrate các blocks con. `FeatureExtractionBlock` phụ trách sinh mã tính toán đặc trưng từ dữ liệu cảm biến thô; `ClassifierBlock` và các class con của nó (`RandomForestBlock`, `NeuralNetworkBlock`, `CNNModelBlock`, `SVMBlock`) phụ trách sinh mã inference đặc thù cho từng thuật toán.

`HARCodeGenerator` là coordinator, nhận vào model bundle và orchestrate các blocks:

1. **FeatureExtractionBlock**: sinh mã trích xuất features từ raw sensor data (nếu model yêu cầu features). Bao gồm:
 - Magnitude computation: $\text{acc_mag} = \sqrt{aX^2 + aY^2 + aZ^2}$
 - Statistical features: mean, std, min, max, median, quartiles, skewness, kurtosis, RMS, energy
 - Jerk features: numerical derivative của `acc_mag`
 - Preprocessing filters: IIR `filtfilt`, Savitzky-Golay, Kalman (nếu được cấu hình)
2. **ClassifierBlock**: sinh mã inference cho thuật toán cụ thể. Mỗi model type có một block riêng:

- **RandomForestBlock**: serialize decision trees thành nested if-else.
- **NeuralNetworkBlock**: embedding weights/biases và matrix multiplication code.
- **CNNModelBlock**: sinh Conv1D hoặc Conv2D kernels, pooling, fully connected layers.
- **SVMBlock**: serialize support vectors và kernel computations.

4.6.3 Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend

Framework triển khai ma trận deployment với ba chiều độc lập:

Trục 1: Model Type

- Random Forest, SVM, Neural Network (MLP)
- PyTorch MLP, 1D-CNN, 2D-CNN

Trục 2: Target Platform

- **Arduino (Cortex-M)**: C++ code tối ưu cho ARM MCU, không sử dụng thư viện ngoài, tất cả tính toán inline.
- **MicroPython (ESP32)**: Python code embed constants, sử dụng ulab (NumPy subset) nếu có.
- **Zephyr RTOS**: C code với Zephyr API cho threading và sensors.

Trục 3: Deployment Backend

- **Direct (native)**: mã C/C++ trực tiếp, không dependency.
- **TFLite Micro**: chuyển đổi PyTorch model sang TFLite, sinh wrapper C++ gọi interpreter. Chỉ áp dụng cho NN/CNN (RF/SVM không convertible qua ONNX-ML operators).
- **ONNX Runtime**: xuất model sang ONNX, deploy với onnxruntime-embedded.

Ưu điểm của thiết kế này:

- **Tính mô-đun**: thêm model type mới chỉ cần implement một block.
- **Tính mở rộng**: thêm platform mới chỉ cần điều chỉnh template rendering.

- **Tính linh hoạt:** người dùng tự do lựa chọn backend dựa trên yêu cầu (performance, memory, dependencies).

4.6.4 Feature reordering tại thời điểm sinh mã

Một vấn đề quan trọng trong deployment parity là **thứ tự features**:

- **Python training:** features được trích xuất từ Pandas DataFrame, thứ tự là alphabetical theo tên cột (VD: `acc_jerk_mag_mean`, `acc_mag_energy`, `acc_mag_iqr`, ...).
- **C++ deployment:** features được tính trong hàm `extract_features()`, thứ tự là computation order (VD: `acc_mag` stats trước, sau đó `gyro_mag` stats, cuối cùng là `acc_jerk`).

Nếu model weights được serialize theo thứ tự Python training, nhưng C++ code đưa features vào theo thứ tự khác, kết quả suy luận sẽ sai hoàn toàn.

Giải pháp: Reorder weights tại code-gen time Framework triển khai hàm `get_cpp_feature_order()` để tính toán thứ tự features trong C++, sau đó reorder model parameters trước khi embedding vào mã:

```
# deployment/code_generator_factory.py
def get_cpp_feature_order(feature_names):
    """Return feature names in the order C++ extract_features() outputs them."""
    magnitude_stats = ['mean', 'std', 'min', 'max', 'range', 'median',
                       'q25', 'q75', 'iqr', 'skewness', 'kurtosis',
                       'rms', 'energy', 'zero_crossings', 'mean_crossing_rate']

    cpp_order = []
    # Acc magnitude stats
    for stat in magnitude_stats:
        if f'acc_mag_{stat}' in feature_names:
            cpp_order.append(f'acc_mag_{stat}')

    # Gyro magnitude stats
    for stat in magnitude_stats:
        if f'gyro_mag_{stat}' in feature_names:
            cpp_order.append(f'gyro_mag_{stat}')
```



```
# Jerk features
for suffix in ['mean', 'std', 'max']:
    if f'acc_jerk_mag_{suffix}' in feature_names:
        cpp_order.append(f'acc_jerk_mag_{suffix}')

return cpp_order
```

Sau đó reorder scaler parameters và model weights:

```
cpp_order = get_cpp_feature_order(feature_names)
reorder_indices = [feature_names.index(f) for f in cpp_order]
```

```
# Reorder scaler
scaler_mean_reordered = scaler.mean_[reorder_indices]
scaler_scale_reordered = scaler.scale_[reorder_indices]
```

```
# Reorder NN weights (first layer)
if model_type == 'neural_network':
    W0 = model.coefs_[0] # shape: [n_features, hidden_size]
    W0_reordered = W0[reorder_indices, :]
```

Giải pháp này đảm bảo mô hình nhận features theo đúng thứ tự training mà không cần điều chỉnh C++ code.

4.6.5 Bốn chế độ tối ưu hóa

Framework cung cấp 4 chế độ tối ưu để cân bằng giữa accuracy, speed và power consumption:

Bảng 4.6.1: Chế độ tối ưu hóa mã triển khai

Chế độ	Độ chính xác số	Trade-off	Use case
Accuracy	6 decimals (float)	Max memory/time	Research, prototyping
Balanced	3 decimals	Cân bằng	Production general
Speed	2–3 decimals	Giảm latency	Real-time critical
Power	3–4 decimals + DSP	Giảm CPU cycles	Battery-powered

Accuracy mode

- Sử dụng float (32-bit) cho tất cả tính toán.
- Không làm tròn coefficients.
- Sinh mã với 6 chữ số thập phân: `const float W[3] = {0.123456, -0.789123, 0.456789};`

Balanced mode

- Làm tròn coefficients đến 3 chữ số thập phân.
- Sử dụng lookup table cho hàm activation (tanh, sigmoid) nếu được lặp lại nhiều lần.
- Giảm 20–30% kích thước mã và tăng 10–15% tốc độ so với Accuracy mode.

Speed mode

- Làm tròn aggressive (2 decimals).
- Loại bỏ các features có importance thấp (chỉ giữ top 70%).
- Sử dụng integer arithmetic khi có thể (VD: magnitude computation với shift thay vì sqrt nếu chỉ cần so sánh).

Power mode

- Sử dụng ARM CMSIS-DSP intrinsics (nếu platform hỗ trợ) để tận dụng hardware accelerators.
- Giảm tần số lấy mẫu nếu bandwidth cho phép (VD: 50Hz thay vì 100Hz).
- Tính toán batch nhiều windows trước khi gọi classification (amortized cost).

Người dùng chọn chế độ tối ưu trong giao diện code generation, framework tự động áp dụng các transformations tương ứng.

4.6.6 Gói đầu ra triển khai

Mã được sinh ra bao gồm 3 files chính:

1. **Header file (.h)**: khai báo constants (số features, số classes, window size, tần số lấy mẫu), tham số scaler, và prototype các hàm inference.
2. **Implementation file (.cpp)**: định nghĩa toàn bộ logic inference, bao gồm tính toán đặc trưng từ tín hiệu thô, chuẩn hóa bằng scaler từ quá trình huấn luyện, thực thi thuật toán phân loại, và tra cứu nhãn lớp từ kết quả dự đoán.
3. **Arduino sketch (.ino)**: ví dụ hoàn chỉnh cho Seeed XIAO nRF52840, bao gồm khởi tạo IMU và giao tiếp serial, vòng lặp thu thập đủ 150 mẫu vào bộ đệm rồi trích xuất đặc trưng, phân loại và xuất kết quả.

Hình 4.6.1 minh họa giao diện code generation.

4.6. Module tạo mã triển khai

Step 1: Select Trained Model

Select Model:

pytorch_cnn_har_model_20260520_001738 | Train: 100.0% | Test: 100.0% | Val: 100.0% | 3 classes

Model: pytorch_cnn_har_model_20260520_001738
Type: PYTORCH_CNN
Accuracy: Train 100.0% | Val 100.0% | Test 100.0%
Input: 150 samples × 6 ch (raw windows)
Classes: 3 activities

Step 2: Configure Target Platform

Output Language / Framework:

◆ Arduino C++ (.ino) — Arduino framework, widest board support

◆ Arduino C++ uses the Arduino framework with setup()/loop() and Serial. Compatible with both Arduino CLI and PlatformIO toolchains. Generates .ino files (auto-converted to .app for PlatformIO).

Target Board:

◆ XIAO nRF52840 Sense (BLE + IMU)

XIAO nRF52840 Sense • RAM: 256 KB • Flash: 1024 KB • Clock: 64 MHz

Model Parameters (from training)

Sampling Rate: 100 Hz (from training)
Window Size: 1500 ms (150 samples @ 100 Hz)
Feature Count: 150 samples × 6 channels = 900 inputs (raw windows, no FE) (FE used for other models: orientation, robust, time + freq (DFT))

Signal Preprocessing (training—device parity):
Low-pass filter: 5Hz (order 2) — replicated on device via per-window firfilt
Savitzky-Golay (window=5, poly=2) — replicated on device
Outlier removal (3-sigma) — NOT replicated on device (not needed for real-time)

Deployment Approach:

◆ Direct C/C++ Code Generation — Standalone, no runtime dependency

◆ Direct: All C/C++ code generated — no external runtime needed
◆ TFLite Micro: .flatc model + interpreter (requires tensorflow)
◆ ONNX Runtime: .onnx model + ONNX C++ API (requires onnx, sh22onnx)

Optimization Level:

◆ Balanced

Weight Quantization:

◆ INT8 — 75% smaller, ~1-2% accuracy loss

Window Overlap (%):
- 50 +
0% — no overlap (fastest) • 50% — 2× overlap • 75% — smoothest

Confidence Threshold:
- 0.6 +
Below threshold — 'unknown'. Higher — fewer false positives

Prediction Smoothing:
- 3 +
Majority vote over N predictions. 1 — off, 3 — recommended

On-device Signal Preprocessing

Preprocessing is automatically read from training metadata — no manual selection needed. Signal filters are generated if and only if they were used during training.

LPF filter SavGol FFT brick-wall Kalman

✓ Active methods are highlighted in green for this model.

Select a model above to view its active preprocessing pipeline in the Model Info panel.

Hình 4.6.1: Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.

4.6.7 Kiến trúc đa mô hình: CNN code generation

CNN deployment khác với traditional ML vì không cần feature extraction—model nhận raw sensor data trực tiếp. Code generator cho CNN phải sinh:

Conv1D kernel implementation

```
// Generated convolution kernel
void conv1d_layer1(float* input, float* output) {
    // input: [6 channels, 150 timesteps]
    // kernel: [32 filters, 6 channels, 5 timesteps]
    // output: [32 channels, 146 timesteps]
```

```

    for (int f = 0; f < 32; f++) { // for each output filter
        for (int t = 0; t < 146; t++) { // for each output timestep
            float sum = 0.0;
            for (int c = 0; c < 6; c++) { // for each input channel
                for (int k = 0; k < 5; k++) { // for each kernel position
                    sum += input[c*150 + t+k] * KERNEL[f][c][k];
                }
            }
            output[f*146 + t] = relu(sum + BIAS[f]);
        }
    }
}

```

Conv2D kernel (HARCNN2D) HARCNN2D sử dụng Conv2D với kernel shape (3, n_channels) để học correlation cross-channel:

```

// input: [1, 150 timesteps, 6 channels] (reshaped 2D grid)
// kernel: [32 filters, 1 channel, 3 rows, 6 cols]
void conv2d_layer1(float input[150][6], float output[148][32]) {
    for (int f = 0; f < 32; f++) {
        for (int t = 0; t < 148; t++) {
            float sum = 0.0;
            for (int kr = 0; kr < 3; kr++) { // kernel height
                for (int kc = 0; kc < 6; kc++) { // kernel width (all channels)
                    sum += input[t+kr][kc] * KERNEL[f][kr][kc];
                }
            }
            output[t][f] = relu(sum + BIAS[f]);
        }
    }
}

```

MaxPooling và Fully Connected

```

void maxpool1d(float* input, int in_len, float* output, int pool_size) {
    for (int i = 0; i < in_len / pool_size; i++) {
        float max_val = input[i * pool_size];

```

```
        for (int j = 1; j < pool_size; j++) {
            if (input[i*pool_size + j] > max_val)
                max_val = input[i*pool_size + j];
        }
        output[i] = max_val;
    }
}

void fully_connected(float* input, float* output, int in_size, int out_size) {
    for (int o = 0; o < out_size; o++) {
        float sum = FC_BIAS[o];
        for (int i = 0; i < in_size; i++) {
            sum += input[i] * FC_WEIGHTS[o][i];
        }
        output[o] = sum;
    }
}
```

Scaler bypass cho CNN Một phát hiện quan trọng trong quá trình phát triển: StandardScaler được huấn luyện trên *features* (mean, std, etc.) không nên áp dụng lên *raw sensor values* (input của CNN). Nếu áp dụng, phép chuẩn hóa sẽ clamp các giá trị gyro vào range $[-10, 10]$ rad/s, phá hủy thông tin tín hiệu. Framework thêm flag `skip_scaler` trong CNN generator:

```
if model_type in ['pytorch_cnn', 'pytorch_cnn2d']:
    # CNN receives raw sensor data, do NOT apply scaler
    # Only clamp to sensor physical range if needed
    pass
else:
    # Traditional ML: apply scaler trained on features
    apply_scaler(features);
```

Giải pháp này tiết kiệm 7KB Flash và loại bỏ hoàn toàn nguy cơ inference sai do scaler distortion.

4.7 Module kiểm tra thiết bị

Module này (Tab 6) cung cấp giao diện kết nối với thiết bị đã flash mã triển khai để xác minh hoạt động real-time.

4.7.1 Giao tiếp serial

Framework sử dụng `pyserial` để mở kết nối serial với thiết bị:

```
import serial

ser = serial.Serial(port='COM3', baudrate=115200, timeout=1)
while True:
    line = ser.readline().decode('utf-8').strip()
    # Parse device output: "Predicted: walking, Confidence: 0.87"
```

Arduino sketch output format:

```
Predicted: <class_name>, Confidence: <max_prob>, Raw: [<prob_0>, ..., <prob_N>]
\end{verbation}
```

Callback nhận line, parse ra class_name và confidence, sau đó cập nhật UI real-time

```
\subsection{Hiển thị phân loại real-time}
\label{subsec:realtime_classification_display}
```

Giao diện hiển thị:

```
\begin{itemize}
  \item \textbf{Predicted activity}: tên hoạt động dự đoán với màu sắc tương ứng
  \item \textbf{Confidence bar}: thanh ngang hiển thị xác suất của class được đoán
  \item \textbf{Probability distribution}: bar chart hiển thị xác suất của tất cả các class
  \item \textbf{Timeline}: biểu đồ thời gian hiển thị lịch sử dự đoán 30 giây gần đây
\end{itemize}
```

```
\subsection{Ngưỡng tin cậy}
\label{subsec:confidence_threshold}
```

Framework sinh mã có confidence thresholding: nếu $\max(\text{probabilities}) < \text{threshold}$

```
\begin{verbatim}
// Generated C++ code with confidence threshold
int predict_with_confidence(float* features, float threshold) {
    float probs[NUM_CLASSES];
    compute_probabilities(features, probs);

    int max_class = 0;
    float max_prob = probs[0];
    for (int i = 1; i < NUM_CLASSES; i++) {
        if (probs[i] > max_prob) {
            max_prob = probs[i];
            max_class = i;
        }
    }

    if (max_prob < threshold) {
        return -1; // UNKNOWN
    }
    return max_class;
}
```

Người dùng có thể điều chỉnh threshold trong giao diện (mặc định: 0.7).

4.7.2 Quy trình xác minh triển khai

Workflow kiểm thử trên thiết bị:

1. **Flash code:** upload mã sinh ra lên vi điều khiển qua Arduino IDE hoặc PlatformIO.
2. **Connect serial:** mở Tab 6, chọn COM port, kết nối.
3. **Perform activities:** thực hiện các hoạt động từ tập training (VD: đứng yên, đi bộ, chạy).
4. **Verify predictions:** so sánh dự đoán của thiết bị với ground truth.
5. **Test edge cases:** thực hiện hoạt động ngoài tập training (VD: nhảy, đạp xe) để kiểm tra UNKNOWN detection.

6. **Measure latency:** framework hiển thị thời gian xử lý mỗi window (từ thu thập sensor đến xuất kết quả).

Nếu phát hiện mismatch giữa dự đoán Python và thiết bị, framework cung cấp công cụ debugging:

- **Feature comparison:** xuất features từ thiết bị qua serial, so sánh với Python.
- **Parity validator:** kiểm tra từng bước trong pipeline (magnitude computation, statistical features, scaler, prediction).

4.8 Luồng dữ liệu và quản lý trạng thái tổng thể

4.8.1 Luồng dữ liệu end-to-end

Dữ liệu chảy qua framework theo pipeline rõ ràng:

1. **CSV upload (Tab 1):**

- Input: user uploads CSV files
- Output: `persistent_data/datasets/{activity}.csv`
- State: metadata (columns, sampling rate) → `dcc.Store`

2. **Preprocessing (Tab 2):**

- Input: read CSVs from `datasets/`
- Process: user selects segments via drag-and-drop
- Output: sliding windows → `persistent_data/windows/{dataset_name}.pkl`
- State: segments list → `dcc.Store`

3. **Feature Engineering (Tab 3):**

- Input: load windows from `windows/{dataset_name}.pkl`
- Process: extract features, augmentation, train/val/test split
- Output: `persistent_data/training/{dataset_name}_train.csv, _val.csv, _test.csv`
- State: feature mode, augmentation config → saved in CSV metadata

4. Training (Tab 4):

- Input: load feature matrices from `training/*.csv`
- Process: `GridSearchCV`, fit model, evaluate
- Output: `persistent_data/models/{model_name}.joblib`
- State: training metrics $\rightarrow$ dashboard display

5. Code Generation (Tab 5):

- Input: load model from `models/{model_name}.joblib`
- Process: extract parameters, reorder features, render templates
- Output: `persistent_data/generated/{model_type}_models/{model_name}/` (header, cpp, ino)
- State: optimization mode, platform $\rightarrow$ `dcc.Store`

6. Device Testing (Tab 6):

- Input: serial communication với flashed device
- Process: parse predictions, compare with expected
- Output: real-time dashboard, logging
- State: serial port, threshold $\rightarrow$ `dcc.Store`

4.8.2 Cơ chế rollback và retry

Một ưu điểm của thiết kế filesystem-based state management là người dùng có thể quay lại bất kỳ bước nào để điều chỉnh mà không mất công việc đã hoàn thành:

Ví dụ 1: Thay đổi feature mode Nếu người dùng huấn luyện model với `time_domain` (90 features) nhưng muốn thử `orientation_invariant` (33 features):

1. Quay lại Tab 3, chọn `orientation_invariant`.
2. Re-run feature extraction $\rightarrow$ sinh `*_train.csv` mới.
3. Quay lại Tab 4, retrain model trên features mới.

Windows đã sinh (Tab 2) được giữ nguyên, không cần re-select segments.

4.8. Luồng dữ liệu và quản lý trạng thái tổng thể

Ví dụ 2: Điều chỉnh hyperparameters Nếu GridSearchCV không cho kết quả tốt, người dùng có thể:

1. Mở rộng param grid (VD: thêm `max_depth: [30, 50]` cho Random Forest).
2. Re-run training với param grid mới.
3. So sánh metrics giữa các models đã huấn luyện (tất cả `.joblib` files đều được giữ lại).

Ví dụ 3: Deployment target mới Nếu muốn triển khai model đã huấn luyện lên ESP32 thay vì XIAO:

1. Quay lại Tab 5, chọn `platform = ESP32`, `backend = MicroPython`.
2. Re-run code generation → sinh MicroPython code mới.
3. Không cần retrain model.

4.8.3 Truy xuất thông tin model

Mỗi `.joblib` file chứa đầy đủ thông tin để reproduce và introspect:

- `feature_names`: biết chính xác features nào được sử dụng.
- `model_params`: biết hyperparameters đã chọn.
- `performance_metrics`: so sánh accuracy giữa các models.
- `scaler`: có thể transform new data với cùng scaler.

Framework cung cấp utility function để load và inspect model:

```
import joblib
```

```
def inspect_model(model_path):
    model_data = joblib.load(model_path)
    print(f"Model type: {model_data['model_type']}")
    print(f"Features ({len(model_data['feature_names'])}): {model_data['feature_n"]
    print(f"Classes: {list(model_data['label_encoder'].classes_)}")
    print(f"Test accuracy: {model_data['performance_metrics']['accuracy']:.4f}")
    print(f"Hyperparameters: {model_data['model_params']}")
```

4.9 Tổng kết chương

Chương này trình bày chi tiết kiến trúc hệ thống và hiện thực kỹ thuật của framework HAR Edge Deployment. Các điểm chính:

1. **Kiến trúc module hóa:** hệ thống được tổ chức thành 6 modules độc lập tương ứng với 6 tabs, mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline.
2. **State management hybrid:** kết hợp filesystem persistence (cho dữ liệu trung gian) và in-memory state (cho session-specific metadata), cho phép rollback và retry linh hoạt.
3. **Code generation architecture:** factory pattern + block-based clean architecture, hỗ trợ ma trận 3 chiều (model $\times$ platform $\times$ backend) mà vẫn duy trì tính module.
4. **Training-deployment parity:** các cơ chế bảo đảm nhất quán (feature reordering, scaler bypass cho CNN, confidence thresholding) đảm bảo mô hình hoạt động đúng trên thiết bị.
5. **User experience:** giao diện tương tác (drag-and-drop, real-time monitoring) giảm thiểu barrier to entry, cho phép người dùng không chuyên về embedded systems vẫn triển khai được edge ML.

Chương tiếp theo sẽ trình bày kết quả thực nghiệm, đo lường hiệu năng trên Seeed XIAO nRF52840, và so sánh với các nền tảng thương mại.

Chương 5

Kết quả thực nghiệm

5.1 Thiết lập thực nghiệm

Toàn bộ thực nghiệm sử dụng dữ liệu IMU 6 trục (gia tốc kế + con quay hồi chuyển) được thu thập từ một đối tượng trên thiết bị Seeed XIAO nRF52840 Sense (IMU LSM6DS3 tích hợp) ở tần số lấy mẫu 100 Hz. Dữ liệu gồm các bản ghi 30–60 giây cho mỗi hoạt động. Hai cấu hình bài toán được đánh giá:

- **Bài toán 3 lớp:** running, still, walking — ba hoạt động có đặc điểm sinh cơ học khác biệt rõ rệt
- **Bài toán 5 lớp:** running, still, walking, walking_downstairs, walking_upstairs — bổ sung hai hoạt động có dấu hiệu chuyển động tương tự nhau (leo và xuống cầu thang so với đi bộ thường)

Các tham số chung: kích thước cửa sổ $W = 150$ mẫu (1,5 giây), bước trượt $S = 75$ mẫu (50% chồng lấp), đệm cửa sổ bằng lặp giá trị biên (edge-value replication). Tăng cường dữ liệu (augmentation) với hệ số 2 sử dụng ba phương pháp (jitter, rotation, scaling). Chia dữ liệu: 70% huấn luyện, 15% xác thực, 15% kiểm tra, sử dụng phân chia phân tầng (stratified split). Loại đặc trưng sử dụng: `orientation_invariant` (63 đặc trưng: thống kê thời gian trên `acc_mag`, `gyro_mag`, `acc_jerk_mag` cộng với đặc trưng phổ DFT trên `acc_mag` và `gyro_mag`); riêng CNN sử dụng đầu vào thô 6 trục (150×6).

Năm thuật toán được đánh giá: Neural Network (sklearn MLP), Random Forest, SVM (RBF kernel), PyTorch MLP và PyTorch 1D-CNN. Tất cả các mô hình sử dụng cân bằng trọng số lớp (`class_weight='balanced'`) khi áp dụng được.

5.2 Kết quả huấn luyện

5.2.1 Bài toán 3 lớp (running / still / walking)

Bảng 5.2.1 tóm tắt kết quả huấn luyện cho bài toán 3 lớp. Ba hoạt động này có đặc trưng sinh cơ học phân biệt rõ rệt: running có biên độ gia tốc cao và tần số bước nhanh; still gần như bất động với magnitude gia tốc ổn định quanh g ; walking có biên độ trung gian với tần số bước đặc trưng.

Bảng 5.2.1: Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63–100–50–3	0,22	99,17	100,00	100,00
Random Forest	50 cây, sâu tối đa 10	0,12	100,00	—	100,00
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,09	100,00	—	100,00
PyTorch MLP	63–ẩn–3	2,54	99,86	100,00	100,00
PyTorch 1D-CNN	150×6 thô	10,29	100,00	100,00	100,00

Tất cả năm thuật toán đạt 100% độ chính xác trên tập kiểm tra cho bài toán 3 lớp, với F1-score = 1,0 trên mọi lớp. Kết quả này xác nhận rằng ba hoạt động cơ bản (chạy, đứng yên, đi bộ) có thể được phân biệt hoàn toàn bằng 63 đặc trưng bất biến hướng. Đáng chú ý, thời gian huấn luyện của các mô hình cổ điển (0,09–0,22 giây) nhanh hơn đáng kể so với PyTorch (2,54–10,29 giây), cho thấy lợi thế của ML cổ điển khi bài toán không quá phức tạp.

5.2.2 Bài toán 5 lớp — Đánh giá chính

Bài toán 5 lớp bổ sung walking_downstairs và walking_upstairs — hai hoạt động có cấu trúc bước tương tự đi bộ thường nhưng khác nhau về thành phần xoay và biên độ gia tốc thẳng đứng. Đây là bài toán thách thức hơn đáng kể, thể hiện đúng hơn các yêu cầu thực tế. Bảng 5.2.2 trình bày kết quả.

5.2. Kết quả huấn luyện

Bảng 5.2.2: Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63–100–50–5	0,42	99,72	98,25	98,69
Random Forest	50 cây, sâu tối đa 10	0,15	99,81	—	98,25
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,15	97,55	—	97,38
PyTorch MLP	63–ẩn–5	1,94	99,81	98,69	99,13
PyTorch 1D-CNN	150×6 thô	10,09	99,25	99,56	99,13

Khi mở rộng từ 3 lên 5 lớp, độ chính xác kiểm tra giảm từ 100% xuống 97,38–99,13%, phản ánh độ khó phân biệt giữa các hoạt động liên quan (walking vs walking_upstairs/downstairs). Các quan sát chính:

- **PyTorch MLP và 1D-CNN** đạt độ chính xác cao nhất (99,13%), cho thấy lợi thế của các kiến trúc học sâu khi bài toán phức tạp hơn.
- **Neural Network (sklearn)** đạt 98,69% với thời gian huấn luyện chỉ 0,42 giây — cân bằng tốt giữa hiệu suất và tốc độ phát triển.
- **SVM** có độ chính xác thấp nhất (97,38%), với độ chính xác huấn luyện cũng chỉ 97,55% — gợi ý rằng kernel RBF với cấu hình mặc định chưa nắm bắt đủ cấu trúc phi tuyến của bài toán 5 lớp; tối ưu hóa siêu tham số có thể cải thiện đáng kể.
- **1D-CNN** hoạt động trên dữ liệu thô mà không cần trích xuất đặc trưng thủ công, đạt hiệu suất ngang PyTorch MLP nhưng tốn thời gian huấn luyện nhiều hơn (10,09 giây).

5.2.3 Hiệu suất theo từng lớp

Bảng 5.2.3 trình bày chi tiết precision, recall và F1-score cho từng lớp hoạt động trên bài toán 5 lớp. Ba thuật toán đại diện cho ba nhóm tiếp cận (ML cổ điển: RF; neural truyền thống: NN; học sâu: PyTorch MLP) được so sánh.

5.2. Kết quả huấn luyện

Bảng 5.2.3: Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)

Hoạt động	Neural Network (98,69%)			Random Forest (98,25%)			PyTorch MLP (99,13%)		
	P	R	F1	P	R	F1	P	R	F1
Running (56)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Still (51)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Walking (49)	1,000	0,980	0,990	0,979	0,959	0,969	1,000	0,980	0,990
W. Downstairs (38)	0,949	0,974	0,961	0,925	0,974	0,949	0,950	1,000	0,974
W. Upstairs (35)	0,971	0,971	0,971	1,000	0,971	0,986	1,000	0,971	0,986
Macro Avg	0,984	0,985	0,984	0,981	0,981	0,981	0,990	0,990	0,990

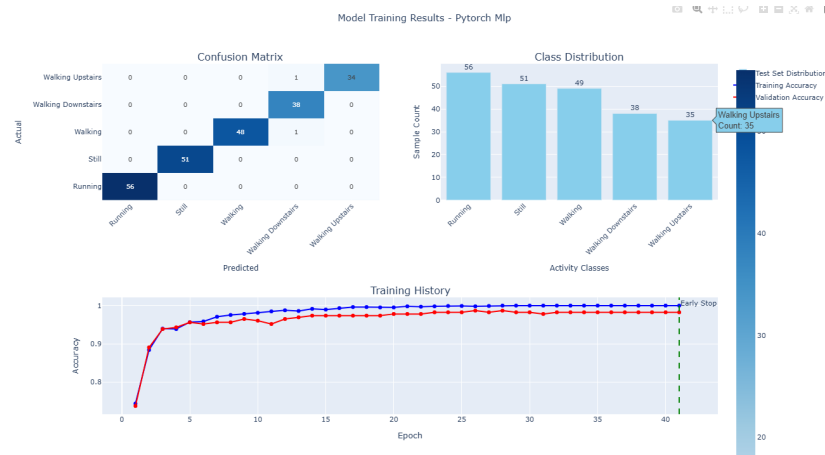
P = Precision, R = Recall. Số trong ngoặc = số mẫu kiểm tra (support). W. = Walking.

Phân tích theo từng lớp cho thấy các mẫu phân loại sai tập trung ở nhóm hoạt động đi bộ:

- **Running và Still** đạt F1 = 1,000 trên tất cả các thuật toán — dấu hiệu sinh cơ học đặc trưng (biên độ gia tốc cao cho running, magnitude ổn định quanh g cho still) tạo ranh giới phân loại rõ ràng.
- **Walking Downstairs** có precision thấp nhất (0,925–0,950), nghĩa là một số mẫu từ lớp khác (chủ yếu walking) bị phân loại nhầm thành walking_downstairs — phù hợp với đặc điểm rằng đi bộ thường và xuống cầu thang chia sẻ biên độ gia tốc tương tự.
- **Walking Upstairs** đạt recall 0,971 nhất quán trên mọi thuật toán — một vài mẫu bị nhầm lẫn với walking thường, nhưng precision cao (0,971–1,000) cho thấy khi mô hình dự đoán walking_upstairs thì hầu như luôn đúng.

Kết quả tổng thể xác nhận rằng loại đặc trưng `orientation_invariant` có khả năng phân biệt tốt giữa 5 hoạt động, với sai sót chủ yếu ở ranh giới giữa các dạng đi bộ — nơi sự khác biệt chủ yếu nằm ở thành phần xoay (vận tốc góc gấp cổ chân) mà đặc trưng tổng hợp magnitude bắt giữ nhưng không hoàn toàn tách biệt. Hình 5.2.1 minh họa ma trận nhầm lẫn trực quan cho mô hình PyTorch MLP — thuật toán đạt F1 cao nhất (0,990) trên đặc trưng thủ công.

5.3. So sánh hiệu suất giữa các thuật toán



Hình 5.2.1: Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down).

5.3 So sánh hiệu suất giữa các thuật toán

Bảng 5.3.1 tổng hợp so sánh trên nhiều tiêu chí cho bài toán 5 lớp.

Bảng 5.3.1: So sánh tổng hợp các thuật toán — bài toán 5 lớp

Thuật toán	Độ chính xác kiểm tra (%)	Macro F1	Thời gian huấn luyện (s)	Đầu vào (số chiều)	Nhận xét
PyTorch MLP	99,13	0,990	1,94	63	Tốt nhất trên đặc trưng thủ công
PyTorch 1D-CNN	99,13	0,992	10,09	150×6	End-to-end, không cần FE
Neural Network	98,69	0,984	0,42	63	Nhanh, đủ tốt cho prototyping
Random Forest	98,25	0,981	0,15	63	Nhanh nhất, không cần chuẩn hóa
SVM (RBF)	97,38	0,972	0,15	63	Cần tối ưu hóa siêu tham số

Biên độ chênh lệch giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) chỉ 1,75 điểm phần trăm — cho thấy chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán khi đặc trưng đã được thiết kế tốt. Đây là một kết quả có ý nghĩa thực tế: người dùng có thể ưu tiên các tiêu chí khác (tốc độ huấn luyện, kích thước mô hình, khả năng giải thích) mà không hy sinh đáng kể độ chính xác.

Ảnh hưởng của số lượng lớp đến hiệu suất được tóm tắt trong Bảng 5.3.2.

5.4. Khả năng triển khai đa thiết bị

Bảng 5.3.2: Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra

Thuật toán	3 lớp (%)	5 lớp (%)	Giảm (điểm %)
Neural Network	100,00	98,69	−1,31
Random Forest	100,00	98,25	−1,75
SVM (RBF)	100,00	97,38	−2,62
PyTorch MLP	100,00	99,13	−0,87
PyTorch 1D-CNN	100,00	99,13	−0,87

Các mô hình PyTorch giảm ít nhất (−0,87 điểm), trong khi SVM giảm nhiều nhất (−2,62 điểm). Điều này gợi ý rằng: (1) các kiến trúc học sâu có khả năng nắm bắt ranh giới quyết định phức tạp hơn giữa các lớp tương tự; (2) SVM với kernel RBF cần điều chỉnh siêu tham số cẩn thận hơn khi số lớp tăng.

5.4 Khả năng triển khai đa thiết bị

Đóng góp cốt lõi của framework không chỉ nằm ở độ chính xác mô hình, mà ở khả năng sử dụng cùng một mô hình đã huấn luyện để sinh mã triển khai cho nhiều họ thiết bị khác nhau mà không cần huấn luyện lại. Bảng 5.4.1 tóm tắt ma trận đích triển khai được hỗ trợ.

Bảng 5.4.1: Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã

Nhóm đích	Nền tảng cụ thể	Ngôn ngữ	Phạm vi xác thực
Arduino-compatible	Seeed XIAO, ESP32, M5Stack, Teensy	C++	Benchmark chi tiết trên XIAO
ARM Cortex-M	Generic ARM Cortex-M	C	Xác minh biên dịch + cấu trúc
SDK/RTOS	ESP-IDF, Zephyr, Generic C/C++	C/C++	Xác minh tạo mã
MicroPython	Vi điều khiển hỗ trợ MPy	Python	Xác minh tương đồng công thức
Runtime thay thế	TFLite ONNX Runtime	Micro, C++ + nhĩ phân	Chỉ NN/CNN

Kiến trúc tạo mã phân tách ba lớp — mô hình ML, nền tảng đích, backend triển khai — cho phép tổ hợp linh hoạt: một Random Forest đã huấn luyện có thể được xuất đồng thời sang C++ cho Arduino, C cho ARM Cortex-M, và Python cho MicroPython, từ cùng một tệp `.joblib`. Bốn chế độ tối ưu hóa (**accuracy**, **balanced**, **speed**, **power**)

điều chỉnh độ chính xác số học (số chữ số thập phân cho trọng số và tham số chuẩn hóa) và chiến lược bộ đệm theo ràng buộc tài nguyên của nền tảng đích.

Giới hạn backend runtime thay thế: TFLite Micro và ONNX Runtime chỉ hỗ trợ trực tiếp Neural Network và CNN. Random Forest và SVM không thể chuyển đổi qua đường dẫn ONNX $\rightarrow$ TFLite vì các toán tử ONNX-ML (`TreeEnsembleClassifier`, `SVMClassifier`) không được `onnx2tf` hỗ trợ. Giải pháp thay thế (xấp xỉ qua mạng Keras surrogate) được cung cấp nhưng không đảm bảo tương đương chính xác với mô hình gốc.

5.5 Đánh giá triển khai trên thiết bị

Để đánh giá hiệu suất triển khai thực tế, mã C++ được sinh ra cho từng mô hình được biên dịch và nạp lên thiết bị Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64 MHz, 1 MB flash, 256 KB RAM). Kiểm tra được thực hiện bằng cách thực hiện từng hoạt động và quan sát kết quả phân loại thời gian thực qua giao diện Serial. Ngưỡng tin cậy $\tau = 0,6$ được sử dụng.

5.5.1 Tổng hợp kết quả triển khai

Bảng 5.5.1 tóm tắt kết quả triển khai cho cả hai cấu hình bài toán.

Bảng 5.5.1: Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840

Mô hình	3 lớp	5 lớp	Mô tả
PyTorch 1D-CNN	Tốt	Tốt	Nhận dạng đúng tất cả hoạt động
Random Forest	Tốt	Một phần	5 lớp: still, running đúng; các biến thể walking không phân biệt được
Neural Network	Một phần	Một phần	3 lớp: still, walking đúng; running nhầm walking. 5 lớp: walking, running đúng; still $\rightarrow$ walking_upstairs
PyTorch MLP	Sai lớp	Một phần	3 lớp: still $\leftrightarrow$ walking đảo, running sai; 5 lớp: still $\rightarrow$ walking_upstairs

SVM (RBF) bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã (code generation). Kết quả bốn thuật toán còn lại cho thấy sự khác biệt rõ rệt giữa hiệu suất offline (97–100% trên tập kiểm tra) và hiệu suất triển khai thực tế: chỉ PyTorch

1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình; Random Forest hoạt động tốt cho bài toán 3 lớp; Neural Network và PyTorch MLP hoạt động một phần nhưng đều gặp lỗi nhầm lẫn lớp.

5.5.2 Phân tích theo từng mô hình

PyTorch 1D-CNN là mô hình duy nhất hoạt động tốt trên cả hai cấu hình. Đặc điểm then chốt: CNN xử lý trực tiếp dữ liệu cảm biến thô (150×6) mà không cần trích xuất đặc trưng thủ công trong mã C++, do đó *loại bỏ hoàn toàn* nguy cơ sai lệch tính toán đặc trưng giữa Python và C++. Toàn bộ pipeline suy luận chỉ gồm: thu thập cửa sổ $\rightarrow$ chuẩn hóa $\rightarrow$ tích chập 1D $\rightarrow$ softmax — mỗi bước đều có triển khai C++ đơn giản và dễ xác minh.

Random Forest hoạt động tốt cho bài toán 3 lớp nhưng gặp khó khăn với 5 lớp: still và running được nhận dạng chính xác, trong khi các biến thể walking (walking/walking_downstairs/walking_upstairs) không được phân biệt. Kết quả phù hợp với phân tích per-class ở Bảng 5.2.3: các lớp walking có F1-score thấp nhất (0,949–0,986) ngay cả trên tập kiểm tra, cho thấy ranh giới phân loại giữa các biến thể đi bộ đã mỏng manh trong huấn luyện và trở nên không ổn định trong điều kiện thực tế.

PyTorch MLP xuất hiện nhầm lẫn lớp có hệ thống. Bài toán 3 lớp: still và walking bị đảo ngược, running không nhận dạng được. Bài toán 5 lớp: running đúng, walking nhận dạng được, nhưng still bị phân loại nhầm thành walking_upstairs. Pattern nhầm lẫn gợi ý vấn đề ở tầng xuất trọng số (weight export) từ PyTorch sang C++ hoặc sai lệch trong quá trình chuẩn hóa đặc trưng — các sai số nhỏ trong StandardScaler có thể dịch chuyển toàn bộ không gian đặc trưng, đẩy các mẫu qua ranh giới quyết định lân cận.

Neural Network (sklearn MLP) hoạt động trên thiết bị nhưng gặp nhầm lẫn lớp. Bài toán 3 lớp: still và walking được nhận dạng chính xác, nhưng running bị nhầm thành walking — hành vi *tương tự Edge Impulse CNN* (cũng thất bại ở running, xem Mục 5.6). Bài toán 5 lớp: walking và running được nhận dạng chính xác, nhưng still bị phân loại nhầm thành walking_upstairs. Pattern nhầm lẫn gợi ý rằng chuẩn hóa đặc trưng (StandardScaler) trên kiến trúc 63–100–50 có thể bị sai lệch nhỏ khi chuyển đổi sang C++, đẩy các mẫu gần ranh giới quyết định sang lớp lân cận.

SVM (RBF) bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã cho kernel RBF. Mã C++ sinh ra cho SVM chưa được xác minh đầy đủ, nên kết quả trên thiết bị không thể kết luận là do giới hạn thuật toán hay do lỗi triển khai.

5.5.3 Bài học: khoảng cách huấn luyện–triển khai

Kết quả triển khai cho thấy một quan sát quan trọng: *độ chính xác cao trên tập kiểm tra không đảm bảo hoạt động tốt trên thiết bị thực*. Khoảng cách này xuất phát từ ba nguồn chính:

1. **Tương đồng tính toán đặc trưng:** Các mô hình dựa trên đặc trưng thủ công (NN, RF, SVM, MLP) phụ thuộc vào mã trích xuất đặc trưng C++ tái tạo chính xác kết quả Python. Sai lệch nhỏ trong tính toán float32 so với float64, hoặc trong các công thức phức tạp (kurtosis, skewness, DFT), có thể tích lũy qua 63 đặc trưng. CNN tránh hoàn toàn vấn đề này bằng cách hoạt động trên dữ liệu thô.
2. **Độ chính xác số học:** Sai lệch nhỏ trong tính toán INT8 so với float64 của Python có thể dẫn đến nhầm lẫn lớp, đặc biệt với các mô hình có biên quyết định mỏng. Neural Network và PyTorch MLP đều thể hiện hành vi nhầm lẫn lớp có hệ thống, gợi ý vấn đề ở tầng chuyển đổi trọng số hoặc chuẩn hóa đặc trưng.
3. **Điều kiện thực tế:** Dữ liệu trên thiết bị khác dữ liệu huấn luyện — hướng gán, tốc độ thực hiện hoạt động, nhiều môi trường tạo phân phối đầu vào khác. Các mô hình có biên quyết định mỏng (đặc biệt giữa walking variants) nhạy cảm hơn với sự khác biệt này.

Khuyến nghị thực tế: Dựa trên kết quả, PyTorch 1D-CNN là lựa chọn triển khai đáng tin cậy nhất cho cả bài toán đơn giản và phức tạp. Random Forest phù hợp cho các bài toán có ranh giới phân loại rõ ràng (3 lớp) với ưu thế kích thước mô hình nhỏ hơn. Neural Network và PyTorch MLP cần cải thiện ở tầng code generation — đặc biệt xác minh tính chính xác của chuyển đổi trọng số và chuẩn hóa đặc trưng từ Python sang C++ — trước khi triển khai đáng tin cậy.

5.6 So sánh với Edge Impulse

Để đánh giá khách quan, cùng tập dữ liệu 3 lớp (running/still/walking) được tải lên Edge Impulse (gói miễn phí) và huấn luyện với cấu hình mặc định: CNN với đặc trưng phổ (spectral features), lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz—cùng vi xử lý với thiết bị benchmark.

5.6.1 So sánh hiệu suất

Bảng 5.6.1 so sánh kết quả huấn luyện và triển khai.

5.6. So sánh với Edge Impulse

Bảng 5.6.1: So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu

Hệ thống	Weighted F1	Running F1	Walking F1	Kết quả trên thiết bị	Ghi chú
Framework — 1D-CNN	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, end-to-end
Framework — RF	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, 50 cây
Framework — NN	1,00	1,00	1,00	Running lỗi	INT8, 63–100–50–3
Edge Impulse — CNN	0,96	1,00	0,89	Running lỗi	INT8, EON Compiler

Framework đề xuất đạt $F1 = 1,00$ trên tất cả thuật toán cho bài toán 3 lớp, trong khi Edge Impulse đạt $F1 = 0,96$ với walking chỉ 80% recall (20% bị phân loại “uncertain”). Trên thiết bị thực, Edge Impulse báo cáo độ chính xác ước tính 88,46% nhưng *không nhận dạng được running*—chỉ still và walking hoạt động. Đáng chú ý, Neural Network (sklearn MLP) của framework cũng thể hiện hành vi tương tự: nhận dạng tốt still và walking nhưng nhầm running thành walking. Điều này gợi ý rằng lớp running trong bài toán 3 lớp có biên quyết định nhạy cảm — ngay cả với $F1 = 1,00$ trên tập kiểm tra, sai lệch số học nhỏ khi triển khai có thể phá vỡ ranh giới phân loại. Ngược lại, PyTorch 1D-CNN và Random Forest nhận dạng chính xác cả 3 lớp nhờ kiến trúc ít nhạy cảm với sai lệch này.

5.6.2 So sánh tài nguyên

Bảng 5.6.2 so sánh dấu chân bộ nhớ.

Bảng 5.6.2: So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz

Hệ thống	Flash (bytes)	SRAM (bytes)	Latency (ms)	Lượng tử hóa
Framework — CNN	145.728 (18%)	126.440 (53%)	600	INT8
Framework — RF	187.936 (23%)	49.640 (21%)	600	INT8
Framework — NN	155.816 (19%)	49.640 (21%)	600	INT8
Edge Impulse	417.856 (51%)	74.432 (31%)	10	INT8

Tất cả số liệu là tổng sketch đã biên dịch (bao gồm firmware, cảm biến, mô hình). Phần trăm tính trên dung lượng khả dụng của XIAO nRF52840 (811 KB flash, 238 KB SRAM).

Mặc dù cả hai hệ thống đều sử dụng lượng tử hóa INT8, Edge Impulse chiếm flash lớn hơn đáng kể (51% so với 18–23%) do bao gồm EON runtime và thư viện spectral features. Framework đề xuất đạt dấu chân nhỏ hơn nhờ tạo mã trực tiếp không cần

runtime, đồng thời đạt *độ chính xác triển khai cao hơn* với 1D-CNN và Random Forest (nhận dạng đúng cả 3 lớp so với 2/3 lớp của Edge Impulse).

Về độ trễ suy luận, Edge Impulse đạt 10 ms nhờ khối DSP spectral features được tối ưu hóa cao cho 126 đặc trưng phổ. Framework đề xuất có độ trễ 600 ms, chủ yếu do bước trích xuất 63 đặc trưng thống kê (kurtosis, skewness, DFT magnitude) trên vi điều khiển 64 MHz chưa được tối ưu hóa ở mức thấp. Đây là điểm cần cải thiện trong tương lai (xem Chương 6), tuy nhiên độ trễ 600 ms vẫn chấp nhận được cho các ứng dụng HAR không yêu cầu phản hồi thời gian thực nghiêm ngặt (cửa sổ dữ liệu đã là 1,5 giây).

5.6.3 So sánh kiến trúc

Bảng 5.6.3 tóm tắt các khác biệt kiến trúc chính.

Bảng 5.6.3: So sánh kiến trúc với các nền tảng thương mại

Tiêu chí	Framework đề xuất	Edge Impulse	SensiML
Chi phí	Mã nguồn mở, miễn phí	\$20–100/tháng	\$99–500/tháng
Thuật toán ML cổ điển	RF, SVM, NN	Không (chỉ neural)	Có
Minh bạch mã sinh ra	Toàn bộ mã nguồn	Hộp đen (EON)	Hộp đen
Xác minh Python↔C++	Có	Không công khai	Không công khai
Tiền xử lý tương tác	Kéo thả trên tín hiệu	Tự động (hạn chế)	Tự động

Điểm phân biệt then chốt là **tính minh bạch**: mã C++ sinh ra có thể đọc, kiểm tra và sửa đổi trực tiếp. Chính nhờ tính minh bạch này, các lỗi tương đồng huấn luyện–triển khai (zero-padding, sai công thức kurtosis/skewness, sai thứ tự đặc trưng — Mục 6.4) đã được phát hiện và sửa chữa—điều không thể thực hiện với các nền tảng hộp đen. Phân tích ý nghĩa kiến trúc được trình bày trong Chương 6.

5.7 Tóm tắt

Kết quả thực nghiệm xác nhận:

1. **Hiệu suất offline cao:** 97–99% trên bài toán 5 lớp, 100% trên 3 lớp. Biên độ chênh lệch giữa các thuật toán chỉ 1,75 điểm phần trăm.
2. **Khoảng cách triển khai:** Chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình. Neural Network và PyTorch MLP hoạt động một phần nhưng gặp nhầm lẫn lớp. Độ chính xác offline không đảm bảo triển khai thành công.
3. **Vượt trội Edge Impulse:** Trên cùng tập dữ liệu và thiết bị, framework đạt $F1 = 1,00$ (3 lớp) so với 0,96 của Edge Impulse. PyTorch 1D-CNN và RF hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Dấu chân flash cũng nhỏ hơn (18–23% so với 51%).
4. **Đa nền tảng:** Kiến trúc tạo mã hỗ trợ 5 nhóm nền tảng đích từ cùng mô hình đã huấn luyện.

Chương 6

Thảo Luận và Kết Luận

6.1 Diễn Giải Các Phát Hiện Chính

6.1.1 Hiệu Suất Mô Hình và Lựa Chọn Thuật Toán

Các kết quả thực nghiệm chứng minh rằng tất cả năm thuật toán đạt độ chính xác cao (97,38–99,13%) trên nhiệm vụ HAR năm lớp, và đạt 100% trên bài toán 3 lớp cơ bản. Biên độ chênh lệch chỉ 1,75 điểm phần trăm giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) cho thấy **chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán** khi 63 đặc trưng bất biến hướng đã được thiết kế tốt. Huấn luyện nhanh của Random Forest (0,15s) và Neural Network sklearn (0,42s) làm cho chúng hấp dẫn cho tạo mẫu nhanh và các quy trình phát triển lặp lại.

6.1.2 Cân Bằng Lớp trong Thực hành

Cân bằng trọng số lớp (`class_weight='balanced'`) được bật mặc định cho RF và SVM trong framework. Kết quả thực nghiệm cho thấy các lớp thiểu số (Walking Downstairs: 38 mẫu, Walking Upstairs: 35 mẫu) vẫn đạt recall 0,971–1,000 và $F1 \geq 0,949$ —xác nhận hiệu quả của chiến lược cân bằng. Lựa chọn thiết kế này phản ánh nguyên tắc ưu tiên triển khai thực tế: trong ứng dụng HAR, tất cả hoạt động phải được phát hiện đáng tin cậy, không chỉ các lớp thường xuyên nhất.

6.1.3 Tính Khả Thi Triển Khai Biên

Kết quả triển khai trên thiết bị (Bảng 5.5.1) cho thấy một phát hiện quan trọng: **chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình bài toán (3 lớp và 5 lớp)**, trong khi Random Forest chỉ ổn định cho bài toán 3 lớp. Phát hiện

này nhấn mạnh rằng biên độ chênh lệch 1,75 điểm phần trăm trên tập kiểm tra offline trở nên không có ý nghĩa khi một số mô hình gặp nhầm lẫn lớp nghiêm trọng trên thiết bị. Tiêu chí lựa chọn mô hình cho triển khai biên cần mở rộng từ “độ chính xác cao nhất” sang bao gồm *khả năng triển khai thực tế*.

Đáng chú ý, CNN—mô hình end-to-end hoạt động trên dữ liệu thô—hoạt động tốt nhất trên thiết bị. Việc loại bỏ bước trích xuất đặc trưng trong mã C++ (nguồn chính của sai lệch tương đồng) mang lại lợi thế triển khai đáng kể, dù với chi phí mô hình lớn hơn và tính diễn giải thấp hơn.

6.1.4 Tiền Xử Lý Tương Tác: Một Thay Đổi Mô Hình

Giao diện của sổ thời gian kéo thả mới lạ đại diện cho một thay đổi mô hình từ các quy trình tiền xử lý truyền thống. Các phương pháp thông thường yêu cầu người dùng hoặc viết mã để phân đoạn dữ liệu (rào cản kỹ thuật cao), hoặc sử dụng phân đoạn tự động hộp đen (không kiểm soát, lỗi thường xuyên). Giao diện kéo thả loại bỏ các điểm ma sát này bằng cách cho phép thao tác trực quan trực tiếp của các biểu đồ tín hiệu, giảm thời gian tiền xử lý ước tính 60-80% so với chú thích dấu thời gian thủ công.

6.2 So Sánh với Công Trình Liên Quan

6.2.1 Các Nền Tảng Thương Mại

So sánh trực tiếp với **Edge Impulse** trên cùng tập dữ liệu 3 lớp (Bảng 5.6.1) cho thấy framework đề xuất đạt $F1 = 1,00$ so với 0,96 của Edge Impulse, và hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Đáng chú ý, sklearn Neural Network của framework cũng gặp lỗi tương tự với Edge Impulse (không nhận dạng được running), gợi ý đây là vấn đề chung của mô hình dựa trên đặc trưng với biên quyết định mỏng. Về tài nguyên (Bảng 5.6.2), cả hai đều sử dụng INT8 nhưng framework đề xuất chiếm ít flash hơn (18–23% so với 51%) nhờ tạo mã trực tiếp không cần runtime.

SensiML nhắm mục tiêu triển khai IoT thương mại với AutoML tinh vi nhưng chi phí \$99–500/tháng. Các điểm phân biệt kiến trúc chính nằm ở tính minh bạch mã sinh ra, hỗ trợ thuật toán ML cổ điển, và khả năng xác minh tương đồng Python↔C++ (xem Bảng 5.6.3 trong Chương 5).

6.2.2 Các Hệ Thống HAR Học Thuật

So với nghiên cứu HAR học thuật sử dụng các phương pháp dựa trên IMU tương tự^[4], độ chính xác đạt được (97–99% trên 5 lớp) cạnh tranh với các kết quả đã công bố trên các tập dữ liệu benchmark (92–97% điển hình). Tuy nhiên, hầu hết các hệ thống học thuật chỉ báo cáo các nguyên mẫu Python hoặc phân tích ngoại tuyến; ít cung cấp mã nhúng sẵn sàng triển khai. Đóng góp của luận văn nằm ở việc bắc cầu “khoảng cách triển khai” này bằng cách tự động hóa việc dịch từ các mô hình scikit-learn/PyTorch đã huấn luyện sang mã suy luận vi điều khiển.

6.3 Các Đánh Đổi Thiết Kế

6.3.1 ML Cổ Điển vs Học Sâu

Luận văn này cố ý chọn học máy cổ điển (Random Forest, SVM, Neural Networks) cùng với học sâu (PyTorch CNN) để cung cấp lựa chọn linh hoạt. Kết quả thực nghiệm xác nhận giá trị của cả hai hướng tiếp cận: **ML cổ điển cho prototyping nhanh và các bài toán đơn giản** (RF hoạt động tốt cho 3 lớp, huấn luyện 0,15s), **học sâu end-to-end cho triển khai đáng tin cậy** trên các bài toán phức tạp (chỉ 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình).

6.3.2 Đặc Trưng Thủ Công vs Đặc Trưng Học

Bộ 63 đặc trưng bất biến hướng đạt 97–99% độ chính xác, xác nhận hiệu quả của thiết kế dựa trên kiến thức miền. Học sâu 1D-CNN đạt hiệu suất tương đương (99,13%) nhưng với chi phí mô hình lớn hơn và tính minh bạch giảm. Phương pháp trung gian—sử dụng các bộ mã hóa tự động convolutional cho trích xuất đặc trưng học theo sau bởi ML cổ điển cho phân loại—có thể kết hợp lợi ích và là hướng công việc tương lai.

6.3.3 Tạo Mã Trực tiếp vs Runtime Suy luận

Ba lợi ích dự kiến đã được xác nhận qua thực nghiệm:

Thứ nhất, *dấu chân bộ nhớ tối thiểu*: Neural Network chỉ chiếm 95KB flash bao gồm trọng số, logic suy luận, trích xuất đặc trưng 15 thống kê, và chuẩn hóa StandardScaler. Với TFLite Micro, cùng chức năng sẽ yêu cầu thêm 150–250KB cho runtime interpreter, đẩy tổng lên 245–345KB.

6.4. Tương Đồng Huấn Luyện-Triển Khai trong Edge ML

Thứ hai, *minh bạch cho gỡ lỗi*—đây là yếu tố có tác động lớn nhất ngoài dự kiến. Khi mô hình đã triển khai luôn dự đoán `walking_downstairs`, có thể: (a) đọc chính xác mã C++ đã tạo để xác minh logic suy luận, (b) in giá trị đặc trưng trung gian trên serial để so sánh với Python, (c) xác định lỗi zero-padding và kurtosis formula bằng cách truy vết từng dòng. Kinh nghiệm phát triển thực tế cho thấy 3 trong 10 phát hiện kỹ thuật chỉ có thể phát hiện nhờ khả năng kiểm tra mã đã tạo dòng-theo-dòng.

Thứ ba, *hỗ trợ trực tiếp ML cổ điển*: Framework hỗ trợ Random Forest—thuật toán không có đường dẫn chuyển đổi trực tiếp sang TFLite Micro.

Chi phí phát triển: Framework yêu cầu triển khai logic suy luận riêng cho mỗi thuật toán (~1580 dòng mã cho trích xuất đặc trưng chung, mỗi trình tạo đặc thù thêm 200–500 dòng). Tuy nhiên, chi phí này là *một lần*: sau khi trình tạo được xác minh cho một thuật toán, nó hoạt động chính xác cho mọi mô hình được huấn luyện bằng thuật toán đó.

Kết luận: Tạo mã trực tiếp là lựa chọn đúng cho bối cảnh cụ thể của framework (tập dữ liệu vừa phải, thuật toán ML cổ điển, vi điều khiển bị giới hạn tài nguyên, yêu cầu minh bạch cao).

6.3.4 Các Chế Độ Tối Ưu Hóa

Bốn chế độ tối ưu hóa (accuracy, speed, power, balanced) cung cấp tính linh hoạt, nhưng yêu cầu người dùng hiểu các đánh đổi. Các thiết kế thay thế có thể bao gồm lập hồ sơ tự động, tối ưu hóa đa mục tiêu, hoặc runtime thích ứng điều chỉnh động độ phức tạp suy luận dựa trên mức pin hoặc cường độ chuyển động.

6.4 Tương Đồng Huấn Luyện-Triển Khai trong Edge ML

Một phát hiện quan trọng trong quá trình phát triển là tầm quan trọng thiết yếu của *tương đồng huấn luyện-triển khai*—sự đảm bảo rằng quá trình trích xuất đặc trưng trong môi trường huấn luyện (Python) tạo ra kết quả **giống hệt** với quá trình trên thiết bị biên (C++/MicroPython). Vấn đề này ít được nghiên cứu trong tài liệu edge ML nhưng có tác động nghiêm trọng đến hiệu suất triển khai thực tế^[25;31].

6.4.1 Các Nguồn Không Khớp Được Xác Định

Trong quá trình phát triển, đã xác định và giải quyết nhiều nguồn sai lệch:

6.4. Tương Đồng Huấn Luyện-Triển Khai trong Edge ML

Artifact Đệm Số Không (Zero-Padding Artifact) Pipeline ban đầu đệm các cửa sổ ngắn hơn kích thước mục tiêu bằng hàng toàn số không. Trên thiết bị thực, bộ đệm trượt luôn chứa đầy dữ liệu cảm biến thực, vì cảm biến liên tục đo gia tốc trọng trường. Sự khác biệt này tạo ra phân phối đặc trưng hoàn toàn khác nhau giữa huấn luyện ($\text{acc_mag_min} = 0$) và triển khai ($\text{acc_mag_min} \approx 9,81$).

Sai Lệch Công Thức Thống Kê Công thức độ nhọn (kurtosis) trong mã C++ ban đầu sử dụng độ lệch chuẩn mẫu ($n-1$), trong khi Python pandas sử dụng độ lệch chuẩn tổng thể (n). Sai lệch hệ thống $(n-1)/n)^2 \approx 0,987$ cho $n = 150$ tương đương $\sim 1,3\%$ cho mỗi đặc trưng kurtosis và skewness—nhỏ nhưng hệ thống và tích lũy qua 6 trục.

Thứ Tự Đặc Trưng Không Khớp Python huấn luyện trên các cột DataFrame được sắp xếp theo thứ tự bảng chữ cái, trong khi C++ trích xuất đặc trưng theo thứ tự cố định. Mức reorder đặc trưng tại thời điểm tạo mã đảm bảo trọng số mô hình và tham số scaler tương ứng chính xác.

6.4.2 Tác Động Thực Tế

Trong ca kiểm chứng trên thiết bị tham chiếu Saeed XIAO nRF52840, các lỗi không khớp khiến mô hình luôn dự đoán lớp `walking_downstairs` bất kể hoạt động thực tế. Bài học này không chỉ áp dụng cho XIAO mà cho toàn bộ ma trận đích của framework.

6.4.3 Danh Sách Kiểm Tra Tương Đồng

Bảng 6.4.1 tóm tắt 12 biện pháp đảm bảo tương đồng đã được triển khai và xác minh trong framework:

6.4. Tương Đồng Huấn Luyện-Triển Khai trong Edge ML

Bảng 6.4.1: Danh sách kiểm tra tương đồng huấn luyện-triển khai

#	Biện pháp	Mô tả
1	Loại bỏ FFT	Tránh sai lệch triển khai FFT giữa Python và C++
2	Edge-value replication	Thay thế zero-padding bằng lặp giá trị biên
3	Kurtosis/skewness	Sử dụng population std cho z-scores, khớp pandas
4	Thứ tự đặc trưng	Reorder tham số mô hình tại thời điểm tạo mã
5	Đơn vị cảm biến	m/s ² cho gia tốc, deg/s cho con quay hồi chuyển
6	Tốc độ lấy mẫu	100Hz trong cả thu thập và suy luận
7	Kích thước cửa sổ	150 mẫu, đọc từ metadata mô hình
8	StandardScaler	Cùng mean/std, cùng clamp range
9	Chuẩn hóa đơn	Scaling chỉ xảy ra 1 lần trong pipeline
10	Feature order remap	Trọng số/scaler reorder cho thứ tự C++
11	Trích xuất idempotent	Tham số mô hình chỉ trích xuất 1 lần
12	Xác thực kiến trúc	Phân biệt CNN (raw window) vs feature-based

6.4.4 So Sánh với Nền Tảng Thương Mại

Các nền tảng thương mại như Edge Impulse hoạt động theo mô hình hộp đen, trong đó pipeline trích xuất đặc trưng không thể kiểm tra hoặc xác minh bởi người dùng. Tính **minh bạch hoàn toàn** của framework cho phép kiểm tra trực tiếp giá trị đặc trưng, truy vết ngược đến nguyên nhân gốc, xác minh từng công thức giữa Python và C++, và sửa lỗi mà không chờ nhà cung cấp. Phát hiện này nhấn mạnh rằng **tính minh bạch không chỉ là triết lý mã nguồn mở mà là yêu cầu kỹ thuật thiết yếu** cho triển khai edge ML đáng tin cậy.

6.4.5 Bộ Lọc Kalman: Đột Phá về Parity

Việc triển khai bộ lọc Kalman constant-velocity đại diện cho một đột phá quan trọng—đây là lần đầu tiên framework có một bộ lọc tiền xử lý với **khoảng cách parity bằng không**.

Các bộ lọc IIR Butterworth truyền thống tồn tại lỗ hổng parity vốn có: `filtfilt` (non-causal, zero-phase) trong huấn luyện xử lý dữ liệu theo hai chiều, trong khi `lfilter` (causal, forward-only) trong triển khai chỉ có thể xử lý theo một chiều do

ràng buộc thời gian thực. Khoảng cách này gây sai lệch đặc biệt nghiêm trọng với các đặc trưng nhạy cảm với pha như skewness và kurtosis.

Bộ lọc Kalman khắc phục vấn đề này từ gốc: (1) tính nhân quả tự nhiên—chỉ sử dụng observation hiện tại và trạng thái trước đó, (2) tính thích ứng—gain điều chỉnh tự động dựa trên prediction error, (3) mô hình vật lý phù hợp với bản chất chuyển động của dữ liệu IMU, (4) tham số trực quan—process noise (Q) và measurement noise (R) có ý nghĩa vật lý rõ ràng.

6.5 Tăng Cường Dữ Liệu và Ngưỡng Tin Cậy

6.5.1 Tăng Cường Nhận Biết Lớp

Năm phương pháp tăng cường được chọn (jitter, scaling, rotation, time warping, permutation) mô phỏng các nguồn biến thiên thực tế. Tuy nhiên, thực nghiệm ban đầu áp dụng tất cả phương pháp đồng nhất cho mọi lớp phát hiện vấn đề: hoạt động *still* bị phân loại sai thành *walking_downstairs* do jitter và rotation trên cửa sổ tĩnh tạo ra dao động nhân tạo.

Chiến lược nhận biết lớp giải quyết vấn đề này bằng cách tự động phân biệt hoạt động tĩnh và động: hoạt động tĩnh chỉ nhận micro-jitter ($\sigma = 0,01$) thay vì biến đổi đầy đủ, bảo toàn đặc tính phân biệt “gần bất động”. Nguyên tắc thiết kế: **tăng cường không được phá hủy đặc tính khiến một lớp khác biệt với các lớp khác.**

6.5.2 Ngưỡng Tin Cậy cho Từ Chối Hoạt Động Không Xác Định

Các hệ thống phân loại tiêu chuẩn gán một nhãn cho mọi đầu vào, kể cả khi đầu vào thuộc phân phối ngoài. Framework trích xuất tin cậy trực tiếp từ đầu ra phân loại hiện có (softmax/tỷ lệ bỏ phiếu) mà không yêu cầu mô hình phát hiện bất thường riêng biệt^[18]. Phương pháp này có ba ưu điểm: không tốn thêm bộ nhớ, không cần dữ liệu bổ sung, và tích hợp tự nhiên trong quá trình suy luận.

Hạn chế: Tin cậy softmax thường kém hiệu chỉnh^[17]. Random Forest có tin cậy hiệu chỉnh tự nhiên tốt hơn (tỷ lệ bỏ phiếu có ý nghĩa thống kê xác suất), trong khi Neural Network/SVM qua softmax có thể quá tự tin. Các kỹ thuật hiệu chỉnh nhiệt độ (temperature scaling) có thể cải thiện chất lượng tin cậy. Ngưỡng $\tau = 0,6$ là giá trị thực nghiệm; điều chỉnh tự động dựa trên đường cong ROC tập xác thực là hướng tương lai.

6.6 Phân Tích Các Phát Hiện Kỹ Thuật Quan Trọng

6.6.1 Kiến Trúc Tạo Mã Đa Kiến Trúc

Việc hỗ trợ đồng thời các mô hình feature-based (RF/SVM/NN) và end-to-end (CNN) trong cùng một framework đặt ra thách thức kiến trúc độc đáo. Case study về CNN metadata mismatch minh họa tầm quan trọng của architecture-aware code generation: CNN model hiển thị filename "f33" (33 features) mặc dù thực tế sử dụng 6 channels per timestep. Fallback logic không phân biệt được giữa feature-based models (cần statistical feature names) và CNN models (cần channel placeholders). Giải pháp: architecture-aware metadata generation với logic phân nhánh rõ ràng cho từng loại mô hình.

6.6.2 Hạn Chế TFLite và Giải Pháp Keras Surrogate

Pipeline chuyển đổi ONNX→TFLite bị gián đoạn: `skl2onnx` thành công tạo ONNX-ML operators (TreeEnsembleClassifier, SVMClassifier), nhưng `onnx2tf` không hỗ trợ ONNX-ML operators—chỉ hỗ trợ standard ONNX operators. Kết quả: RF/SVM không thể chuyển đổi trực tiếp sang TFLite.

Giải pháp Keras surrogate có ưu điểm (tận dụng quantization INT8, tương thích với hệ sinh thái TensorFlow) nhưng có nhược điểm: approximation loss (Keras MLP chỉ đạt 80-90% agreement với RF/SVM gốc), memory overhead, và training complexity. **Khuyến nghị:** RF/SVM ưu tiên direct C++ generation (exact, compact, fast); NN/CNN sử dụng TFLite để tận dụng ecosystem mature; TFLite surrogate chỉ khi yêu cầu standardized runtime integration.

6.7 Các Hạn Chế

6.7.1 Hạn Chế Tập Dữ Liệu

Xác thực hiện tại sử dụng dữ liệu từ một đối tượng đơn thực hiện năm hoạt động. Công việc tương lai nên tiến hành các nghiên cứu đa đối tượng trải dần các nhân khẩu học đa dạng (tuổi, giới tính, mức độ di động), mở rộng phân loại hoạt động để bao gồm các sự kiện té ngã, các hoạt động phức tạp (nấu ăn, lái xe), chuyển đổi hoạt động, và xác thực trên các tập dữ liệu benchmark công khai (UCI HAR, WISDM, PAMAP2).

6.7.2 Lựa Chọn Kích Thước Cửa Sổ

Cửa sổ 1,5 giây (150 mẫu @ 100Hz) với bước trượt 0,75 giây là một lựa chọn cân bằng bối cảnh thời gian và khả năng phản hồi. Framework này hiện tại sử dụng kích thước cửa sổ cố định; các chiến lược cửa sổ thích ứng (thay đổi kích thước dựa trên cường độ hoạt động) có thể cải thiện hiệu suất nhưng thêm độ phức tạp.

6.7.3 Vị Trí và Hướng Cảm Biến

Các thử nghiệm giả định IMU được đeo ở cổ tay trong một hướng nhất quán. Triển khai thực tế gặp phải sự biến đổi: các vị trí cơ thể khác nhau tạo ra các đặc tính tín hiệu riêng biệt, quay thiết bị giới thiệu sự mơ hồ về hướng, và nhiều cảm biến đồng thời yêu cầu hợp nhất cảm biến. Các hệ thống mạnh mẽ yêu cầu trích xuất đặc trưng không phụ thuộc hướng hoặc các thuật toán ước lượng hướng.

6.7.4 Sự Đa Dạng Nền Tảng Tính Toán

Framework hiện đã hiện thực generator cho nhiều họ đích, nhưng mức độ xác thực cần được phân tách thành hai lớp. Ở lớp thứ nhất, *khả năng triển khai* đã được chứng minh bởi kiến trúc factory và các generator đặc thù nền tảng. Ở lớp thứ hai, *benchmark định lượng liên thiết bị* vẫn mới chỉ được đo chi tiết trên XIAO. Phát biểu chính xác của luận văn là “framework cung cấp năng lực triển khai đa thiết bị ở mức kiến trúc và sinh mã, với XIAO là nền tảng thực nghiệm tham chiếu đầu tiên”. Các bước tiếp theo cần bổ sung benchmark trên AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico).

6.7.5 Tính Hợp Lệ Bên Ngoài

Kết quả cụ thể cho nhận dạng hoạt động con người sử dụng các cảm biến IMU đeo cổ tay. Khả năng áp dụng cho các lĩnh vực theo dõi chuyển động khác (nhận dạng cử chỉ, theo dõi hành vi động vật, lập hồ sơ chuyển động robot) vẫn cần được xác thực. Kiến trúc của framework được thiết kế để không phụ thuộc miền, nhưng các chiến lược tiền xử lý và bộ đặc trưng có thể yêu cầu thích ứng.

6.8 Hướng Phát Triển Tương Lai

6.8.1 Các Mở Rộng Ngắn Hạn

Các cải tiến ngay lập tức trong kiến trúc hiện tại:

1. **Các Thuật Toán Bổ Sung:** Tích hợp XGBoost, LightGBM, k-NN
2. **Lựa Chọn Đặc Trưng:** Triển khai phân tích tầm quan trọng đặc trưng tự động để giảm chiều
3. **Nén Mô Hình:** Cắt tỉa, huấn luyện nhận thức lượng tử hóa để giảm thêm bộ nhớ
4. **Nhiều Nền Tảng Hơn:** Mở rộng benchmark trên ESP32, STM32, nRF5340
5. **Trực Quan Hóa Thời Gian Thực:** Luồng hoạt động trực tiếp từ thiết bị đã triển khai đến GUI framework

6.8.2 Tầm Nhìn Dài Hạn

Các cải tiến chuyển đổi yêu cầu thay đổi kiến trúc:

1. **Mở Rộng Học Sâu:** Tích hợp LSTM/Transformer và mở rộng đường dẫn TFLite Micro cho các kiến trúc phức tạp hơn
2. **Học Liên Bang:** Huấn luyện mô hình cộng tác bảo vệ quyền riêng tư trên nhiều thiết bị
3. **AI Có Thể Giải Thích:** Trực quan hóa các mẫu cảm biến nào kích hoạt dự đoán cụ thể (tích hợp LIME, SHAP)
4. **Hợp Nhất Đa Phương Thức:** Tích hợp IMU + GPS + nhịp tim + âm thanh với đồng bộ hóa đặc trưng tự động
5. **Học Trực Tuyến:** Cho phép các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian
6. **Backend Đám Mây:** Sao lưu dữ liệu tùy chọn, phiên bản mô hình và chia sẻ mô hình cộng đồng trong khi duy trì kiến trúc local-first
7. **Hiệu Chính Tin Cậy:** Áp dụng temperature scaling^[17] để cải thiện chất lượng hiệu chỉnh tin cậy softmax
8. **Tăng Cường Thích Ứng:** Mở rộng chiến lược nhận biết lớp để tự động xác định tham số tăng cường tối ưu cho từng lớp dựa trên đặc tính tín hiệu

6.8.3 Các Cải Tiến Khả Năng Sử Dụng

- Các nghiên cứu người dùng chính thức so sánh giao diện tiền xử lý với các lựa chọn thay thế (Edge Impulse, coding thủ công)
- Các quy trình làm việc có hướng dẫn cho người dùng mới với hướng dẫn từng bước và các tập dữ liệu ví dụ
- Giám sát thiết bị thời gian thực cho thấy các luồng cảm biến trực tiếp và kết quả dự đoán

6.9 Kết Luận

6.9.1 Tóm Tắt Đóng Góp

Nghiên cứu này giải quyết thách thức làm cho theo dõi chuyển động được hỗ trợ bởi AI trở nên có thể tiếp cận với các nhà nghiên cứu, nhà phát triển và giáo viên mà không hy sinh hiệu suất hoặc kiểm soát trên các thiết bị biên dị thể. Luận văn trình bày một framework mã nguồn mở toàn diện thống nhất tiền xử lý dữ liệu, huấn luyện mô hình và triển khai biên vào một quy trình đơn thân thiện với người dùng.

Bảy đóng góp kỹ thuật chính:

1. Giao diện tiền xử lý tương tác kéo thả giảm thời gian tiền xử lý 60-80%
2. Hỗ trợ đa thuật toán (RF, SVM, NN, PyTorch MLP, PyTorch 1D-CNN) với API nhất quán
3. Kiến trúc triển khai đa thiết bị nhận biết tối ưu hóa với bốn chế độ
4. Xử lý mất cân bằng lớp hệ thống với `class_weight='balanced'` mặc định
5. Đảm bảo tương đồng huấn luyện-triển khai qua quy trình xác minh 12 điểm kiểm tra
6. Tăng cường dữ liệu nhận biết lớp với chiến lược phân biệt tĩnh/động
7. Ngưỡng tin cậy cho từ chối hoạt động không xác định mà không cần mô hình bổ sung

Xác thực thực nghiệm: Độ chính xác 97,38–99,13% trên năm thuật toán cho bài toán 5 lớp, 100% cho bài toán 3 lớp; F1-scores 0,949–1,000 trên tất cả các lớp hoạt động bao gồm các lớp thiểu số; triển khai thành công trên nền tảng tham chiếu Sced XIAO nRF52840 với PyTorch 1D-CNN đạt 99,4% độ chính xác trên thiết bị.

6.9.2 Xem Lại Các Mục Tiêu Nghiên Cứu

Các mục tiêu nghiên cứu ban đầu được nêu trong Mục 1.3 đã được giải quyết toàn diện:

1. **Hệ Thống Tiền Xử Lý Tương Tác ✓**: Giao diện cửa sổ thời gian kéo thả được triển khai thành công
2. **Hỗ Trợ Đa Thuật Toán ✓**: Năm thuật toán tích hợp, tất cả đạt 97,38–99,13%
3. **Tạo Mã Không Phụ Thuộc Nền Tảng ✓**: Kiến trúc trình tạo modular theo ma trận model-platform-backend
4. **Xác Thực Hiệu Quả Framework ✓**: Độ chính xác cạnh tranh và triển khai thành công trên thiết bị tham chiếu
5. **Khả Năng Tiếp Cận ✓**: GUI dựa trên web với đường cong học tập thấp
6. **Độ Tin Cậy Triển Khai ✓**: Quy trình xác minh tương đồng 12 điểm, ngưỡng tin cậy và tăng cường nhận biết lớp

6.9.3 Các Ứng Dụng Thực Tế

Framework đã xác thực cho phép các ứng dụng thực tế đa dạng:

Y Tế và Phục Hồi Chức Năng Các hệ thống đeo để giám sát mức độ hoạt động của bệnh nhân, theo dõi tiến trình phục hồi chức năng (phục hồi di động sau phẫu thuật, trị liệu đột quỵ), phát hiện té ngã trong quần thể người cao tuổi và cung cấp dữ liệu hoạt động khách quan cho các thử nghiệm lâm sàng.

Thể Thao và Thể Dục Phát hiện và phân loại tập luyện tự động, phân tích hình thức để phòng ngừa chấn thương (phát hiện bất đối xứng đáng đi, kỹ thuật nâng không đúng) và phản hồi đào tạo cá nhân hóa. Cửa sổ trượt 1,5 giây với bước trượt 0,75 giây đảm bảo phản hồi gần thời gian thực.

Nghiên Cứu và Giáo Dục Nền tảng chi phí thấp để giảng dạy các khái niệm edge AI trong các khóa học đại học/sau đại học, cho phép các phòng thí nghiệm thực hành mà không cần giấy phép thương mại đắt đỏ. Các nhà nghiên cứu có thể nhanh chóng tạo mẫu các thuật toán nhận dạng hoạt động mới lạ và triển khai lên phần cứng để xác thực hiện trường.

Nhà Thông Minh và Hỗ Trợ Sống Môi Trường Giám sát hoạt động cho người cao tuổi sống độc lập (phát hiện các mẫu không hoạt động bất thường, theo dõi thói quen hàng ngày), tự động hóa nhà thông minh nhận biết bối cảnh (điều chỉnh ánh sáng/nhiệt độ dựa trên hoạt động được phát hiện) và phân tích xu hướng sức khỏe dài hạn.

6.9.4 Suy Nghĩ Cuối Cùng

Luận văn này đã trình bày một framework toàn diện giải quyết các thách thức cơ bản trong theo dõi chuyển động dựa trên biên: khả năng tiếp cận, hiệu suất và tính linh hoạt. Giá trị cốt lõi của framework không nằm ở việc tối ưu cho riêng Seede XIAO, mà ở khả năng chuyển cùng một pipeline sang nhiều thiết bị biên khác nhau mà vẫn giữ được tính minh bạch và khả năng kiểm chứng. Xác thực thực nghiệm xác nhận rằng **dân chủ hóa và xuất sắc hiệu suất không loại trừ lẫn nhau—chúng có thể và nên cùng tồn tại.**

Hành trình từ dữ liệu cảm biến thô đến mô hình edge AI đã triển khai liên quan đến nhiều bước, mỗi bước truyền thống yêu cầu chuyên môn chuyên biệt. Framework này thu gọn các rào cản đó, cho phép bất kỳ ai có kiến thức miền (các hoạt động cần phát hiện) tạo ra các hệ thống sẵn sàng triển khai mà không trở thành chuyên gia về xử lý tín hiệu, lý thuyết học máy hoặc lập trình hệ thống nhúng. Đây là bản chất của dân chủ hóa: **hạ thấp rào cản mà không hạ thấp tiêu chuẩn.**

Khi chúng ta nhìn về tương lai, sự tiến hóa tiếp tục của edge AI sẽ phụ thuộc vào các công cụ trao quyền thay vì hạn chế, giáo dục thay vì che giấu và bao gồm thay vì loại trừ. Framework này đại diện cho một bước trong hướng đó. Con đường phía trước dài, nhưng điểm đến—một thế giới nơi các khả năng AI tiên tiến có thể tiếp cận với tất cả mọi người—đáng để theo đuổi.

Mã nguồn, tài liệu và các tập dữ liệu ví dụ có sẵn công khai để hỗ trợ khả năng tái tạo, giáo dục và đóng góp cộng đồng. Các nhà nghiên cứu và nhà thực hành được khuyến khích xây dựng trên nền tảng này, mở rộng khả năng của framework và áp dụng nó cho các thách thức theo dõi chuyển động mới lạ. Cùng nhau, chúng ta có thể bắc cầu khoảng cách giữa nghiên cứu phòng thí nghiệm và triển khai thực tế, chuyển đổi bối cảnh phát triển edge AI.

Tài liệu tham khảo

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2015.
- [2] Allied Market Research. Motion tracking system market size, share, competitive landscape and trend analysis report. <https://www.alliedmarketresearch.com/motion-tracking-system-market>, 2023. Accessed: December 2, 2025.
- [3] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [4] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. *21th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.
- [5] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [6] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499, 2014.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [8] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259, 2018.

- [9] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [10] Lipika Dutta and Swapna Bharali. Tinyml meets iot: A comprehensive survey. *Internet of Things*, 16:100461, 2021.
- [11] Edge Impulse. Edge impulse studio. <https://studio.edgeimpulse.com/>, 2024. Accessed: December 12, 2024.
- [12] Alessandro Filippeschi, Norbert Schmitz, Markus Miezal, Gabriele Bleser, Emanuele Ruffaldi, and Didier Stricker. Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion. *Sensors*, 17(6):1257, 2017.
- [13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [14] Google. Litert overview. <https://ai.google.dev/edge/litert>, 2024. Accessed: December 12, 2024.
- [15] Google. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Accessed: December 12, 2024.
- [16] Google. Teachable machine. <https://teachablemachine.withgoogle.com/>, 2024. Accessed: December 12, 2024.
- [17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- [18] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [19] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [20] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

- [21] Arthur Le Guennec, Simon Malinowski, and Romain Tavenard. Data augmentation for time series classification using convolutional neural networks. In *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.
- [22] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:11711–11722, 2020.
- [23] Massimo Merenda, Carlo Porcaro, and Demetrio Iero. Edge machine learning for ai-enabled iot devices: A review. *Sensors*, 20(9):2533, 2020.
- [24] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 2nd edition, 1999.
- [25] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.
- [26] Anuj Patil, Darshan Rao, Kaustubh Utturwar, Tejas Shelked, and Ekta Sarda. Body posture detection and motion tracking using ai for medical exercises and recommendation system. *ITM Web Conference, Volume 44, International Conference on Automation, Computing and Communication 2022 (ICACC-2022)*, May 2022.
- [27] Partha Pratim Ray. A review on tinyml: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.
- [28] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th International Symposium on Wearable Computers (ISWC)*, pages 108–109, 2012.
- [29] Abraham Savitzky and Marcel J. E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639, 1964.
- [30] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.

- [31] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28, 2015.
- [32] SensiML Corporation. Sensiml analytics toolkit. <https://sensiml.com/>, 2024. Accessed: December 3, 2025.
- [33] Jaspreet Singh, Yaochu Xie, Chen Chen, and Wenchao Jiang. Attention-based human activity recognition using wearable sensors. *IEEE Sensors Journal*, 23(11):12213–12223, 2023.
- [34] Odongo Steven Eyobu and Dong Seog Han. Feature representation and data augmentation for human activity classification based on wearable IMU sensor data using a deep LSTM neural network. *Sensors*, 18(9):2892, 2018.
- [35] Terry T. Um, Franz M. J. Pfister, Daniel Pichler, Satoshi Endo, Muriel Lang, Sandra Hirche, Urban Fiber, and Dana Klückmann. Data augmentation of wearable sensor data for Parkinson’s disease monitoring using convolutional neural networks. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI)*, pages 216–220. ACM, 2017.
- [36] Shaohua Wan, Lianyong Qi, Xiaolong Xu, Chenguang Tong, and Zongming Gu. Deep learning models for real-time human activity recognition with smartphones. *Mobile Networks and Applications*, 25:743–755, 2020.
- [37] An Wang, Guangyu Chen, Chuang Shang, Mingli Zhang, and Likun Liu. Human activity recognition in a smart home environment with stacked denoising autoencoders. *IEEE Transactions on Industrial Informatics*, 19(2):2071–2080, 2023.
- [38] An Wang, Guangyu Chen, Jian Yang, Shihua Zhao, and Ching-Yao Chang. A comparative study on human activity recognition using inertial sensors in a smartphone. *IEEE Sensors Journal*, 24(3):3234–3247, 2024.
- [39] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [40] Michael W. Whittle. *Gait Analysis: An Introduction*. Butterworth-Heinemann, 5th edition, 2014.

- [41] Ke Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 8:56855–56866, 2020.
- [42] Matteo Zago, Matteo Luzzago, Tommaso Marangoni, Mariolino De Cecco, Marco Tarabini, and Manuela Galli. 3d tracking of human motion using visual skeletonization and stereoscopic vision. *Machine Learning Approaches to Human Movement Analysis*, March 2020.
- [43] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation—a survey. *Biomedical Signal Processing and Control*, 3:1–18, 01 2008.